Karlsruher Institut für Technologie

Transformerbasierte Fusion von Luft- und Bodensensordaten zur Erstellung von HD-Kartenmaterial

Bachelorarbeit

Adrian Noah Wimmer

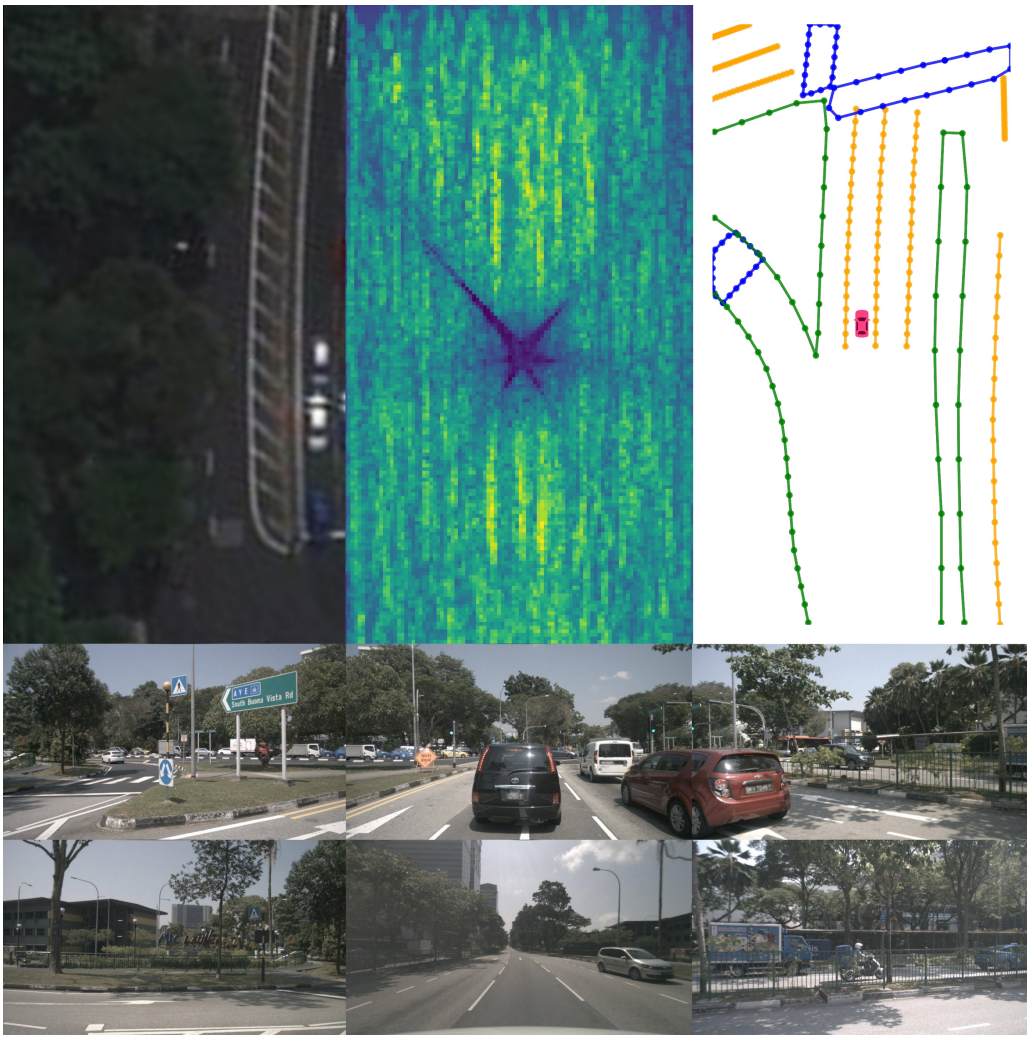
KIT-Fakultät für Wirtschaftswissenschaften
Institut für Angewandte Informatik und Formale Beschreibungsverfahren (AIFB)
und
FZI Forschungszentrum Informatik

Erstgutachter:	Prof. Dr. J. Marius Zöllner
Zweitgutachter:	Prof. Dr. Andreas Oberweis
Betreuender Mitarbeiter:	Rupert Polley, M.Sc.

Bearbeitungszeit: 10. Juli 2024 – 10. Januar 2025

Transformerbasierte Fusion von Luft- und Bodensensordaten zur Erstellung von HD-Kartenmaterial

von
Adrian Noah Wimmer



Bachelorarbeit
im Januar 2025

FZI

Bachelorarbeit, FZI
KIT-Fakultät für Wirtschaftswissenschaften, 2025
Gutachter: Prof. Dr. J. Marius Zöllner, Prof. Dr. Andreas Oberweis

Abteilung Technisch kognitive Systeme
FZI Forschungszentrum Informatik

Erklärung

Ich versichere wahrheitsgemäß, die Arbeit selbstständig verfasst, alle benutzten Hilfsmittel vollständig und genau angegeben und alles kenntlich gemacht zu haben, was aus Arbeiten anderer unverändert oder mit Abänderungen entnommen wurde sowie die Satzung des KIT zur Sicherung guter wissenschaftlicher Praxis in der jeweils gültigen Fassung beachtet zu haben.

Karlsruhe, *Adrian Noah Wimmer*
im Januar 2025

Kurzfassung

Die Arbeit beschäftigte sich mit der Konzeption, Umsetzung und Evaluation einer transformerbasierten Netzwerkarchitektur, welche Luftbilder und Kamerabilder aus Bodenperspektive eines Fahrzeugs fusioniert und daraus online eine HD-Karte der Umgebung erzeugt. Im Fokus stand die Erweiterung der existierenden MapTR-Architektur um ein Fusionsmodell, das die Datenquellen fusioniert. Es wurden statische und dynamische Fusionsmodelle konzeptioniert und implementiert, dabei konnte die Genauigkeit der existierenden MapTR-Architektur übertroffen werden. Die Evaluierung der unterschiedlichen Fusionsansätze zeigte, dass vor allem Modelle, die Feature Scaling implementieren und auf einer pre-trained Netzwerkarchitektur basieren die besten Ergebnisse erzielen.

Inhaltsverzeichnis

1 Einleitung **1**
1.1 Ausgangssituation 1
1.2 Aufgabenstellung 1

2 Grundlagen **3**
2.1 Grundlagen Neuronaler Netzwerke 3
2.1.1 Multi-Layer-Perceptron 3
2.1.2 Lernen in Neuronalen Netzwerken 5
2.1.3 Lossfunktionen in Neuronalen Netzwerken 6
2.2 Klassifikation und deren Bewertung 7
2.3 Convolutional Neural Networks 9
2.4 Feature-Pyramid-Networks 12
2.5 Mini-Batch-Normalisierung 13
2.6 Layer-Normalisierung 14
2.7 Die Transformer-Architektur 14
2.7.1 Der Encoder 15
2.7.2 Der Decoder 16
2.7.3 Attention 16
2.7.4 Position-Wise Fully-Connected Feed-Forward Network 18
2.7.5 Positional Encoding 18

3 Verwandte Arbeiten **21**
3.1 Grundlagen und Übersicht zur HD-Kartenerstellung für autonome Fahrzeuge 21
3.2 MapTR 22
3.2.1 Architektur 22
3.2.2 Permutationsäquivalente Modellierung 23
3.2.3 Hierarchisches Matching 24
3.2.4 Design der Lossfunktion im MapTR Training 26
3.3 2D-zu-BEV-Transformationsmethoden 27
3.4 Weitere Ansätze zur Erstellung von HD-Karten 30

4 Konzept **33**
4.1 Netzwerkarchitektur 33
4.2 Fusionsstrategien 34
4.2.1 Statische Fusionsstrategien 34

4.2.2 Dynamische Fusionsstrategien 34

5 Umsetzung **37**
5.1 Fusionsmodelle 37
5.1.1 Statische Fusionsmodelle 38
5.1.2 Dynamische Fusionsmodelle 39
5.2 Der Optimierungsalgorithmus AdamW 41
5.3 Konfiguration der Lernrate im Trainingsprozess 43
5.4 Berechnungsinfrastruktur und Cluster-Details 44
5.5 Weiteres Hyperparameter-Tuning 44
5.6 Datensatz 46
5.6.1 Der nuScenes-Datensatz 46
5.6.2 Apple Maps als ergänzende Datenquelle 48
5.7 Gewichtsinitialisierung Xavier 50

6 Evaluation **51**
6.1 Validierungsmetrik 51
6.2 Ergebnisse 52
6.2.1 Übersicht und Vergleich der statischen Fusionensmodelle 52
6.2.2 Übersicht und Vergleich der dynamsichen Fusionmodelle 55
6.3 Schlussfolgerungen aus den Ergebnissen 56
6.3.1 Qualitative Analyse der Fusionsstrategien 57

7 Zusammenfassung und Ausblick **61**

A Abbildungsverzeichnis **63**

B Tabellenverzeichnis **65**

C Literaturverzeichnis **67**

1 Einleitung

1.1 Ausgangssituation

Hochaufgelöste Karten (oder HD-Karten) sind eine entscheidende Komponente des autonomen Fahrsystems, da sie dem System als detaillierte Repräsentation ein Verständnis der Umgebung und des Fahrkontextes vermitteln. Sie beinhalten typischerweise geometrische, semantische und topologische Informationen über die Fahrumgebung [11].

Traditionell werden HD-Karten mittels SLAM-basierter Methoden (Simultaneous Localization and Mapping) offline erstellt [39, 40, 47]. Die Karten werden vorab generiert und anschließend in die Fahrzeuge geladen, die sie zur Navigation nutzen. Dieser Offline-Ansatz erfordert regelmäßige Updates und Wartungen, um die Karten aktuell zu halten. Diese Prozesse sind oft zeit- und kostenintensiv.

Neuere Ansätze konzentrieren sich jedoch auf die Nutzung der im Fahrzeug verbauten Sensorik, um in Echtzeit eine HD-Karte der unmittelbaren Umgebung zu generieren [27, 31]. Diese Online-Ansätze der HD-Kartenerstellung nutzen dabei die im Fahrzeug eingebauten Sensoren wie Kameras, LIDAR und Radar, um in Echtzeit eine präzise HD-Karte der unmittelbaren Umgebung zu generieren. Dieser Online-Ansatz bringt zahlreiche Vorteile mit sich, darunter die erhebliche Vereinfachung der Lokalisierung des Fahrzeugs innerhalb der Karte und die Vermeidung veralteter Informationen, die die Entscheidungsfindung beeinträchtigen könnten.

Luftbilder von Flugzeugen, Drohnen oder Satelliten sind geeignet, um Kartenmaterial zu erzeugen, allerdings führt die niedrige räumliche Auflösung und veraltetes Material zu suboptimalen Ergebnissen. Durch die inhärente Perspektive haben sie jedoch den Vorteil einer sehr hohen Reichweite und Verdeckungen durch Verkehrsteilnehmer sind nur auf die unterliegende Bodenebene beschränkt. Weiterhin sind sie für viele Regionen der Welt bereits vorhanden.

In dieser Bachelorarbeit sollen die Vorteile der Bodenperspektive mit den Vorteilen der Vogelperspektive fusioniert werden, um eine verbesserte online HD-Karte zu erzeugen.

1.2 Aufgabenstellung

In dieser Bachelorarbeit werden Fusionsstrategien von Bildfeatures konzeptioniert, implementiert und hinsichtlich in der Arbeit zu definierender Gütekriterien bewertet. Hierfür werden folgende Forschungsfragen als Leitlinien betrachtet:

- Wie können Sensorfeatures aus unterschiedlichen Perspektiven effektiv fusioniert werden?
- Welche Gütekriterien eignen sich für die Evaluation der Netzwerkarchitektur und der Fusion?

- Wie beeinflusst die Fusion die Güte der HD-Karte bezüglich Genauigkeit, Reichweite, Verdeckungen und Lokalisierung?

Als Ausgangsbasis dieser Arbeit soll die transformerbasierte Netzwerkarchitektur MapTR [27] fungieren, welche anhand von sechs Kameraperspektiven eines Versuchsträgers aus Bodenperspektive online eine HD-Karte erzeugt. Diese Architektur wird mit einem Fusionsmodul erweitert, welches passende Features aus Luftbildern mit den Features der Bodenperspektive verbindet. Der nuScenes-Datensatz [2] beinhaltet die benötigten Bilder aus Bodenperspektive, Lokalisierungsinformationen und Kartenmaterial und ist für die Implementierung und Evaluation der Architektur geeignet. Die Luftbilder stammen von Apple Maps. Schlussendlich werden Gütekriterien definiert, welche die Fusion der Features und die Vor- und Nachteile der implementierten Architektur im Vergleich zur MapTR-Baseline bewerten.

2 Grundlagen

Dieses Kapitel führt in die zentralen Konzepte neuronaler Netzwerke ein, die für diese Arbeit von Bedeutung sind. Es behandelt die Struktur von Multi-Layer-Perceptrons, Aktivierungsfunktionen, Lossfunktionen und Optimierungsmethoden sowie fortgeschrittene Architekturen wie Convolutional Neural Networks (CNNs) und Transformer-Modelle. Diese Grundlagen bilden die Basis für die weiteren Untersuchungen und Anwendungen in der Arbeit.

2.1 Grundlagen Neuronaler Netzwerke

Neuronale Netzwerke sind ein fundamentaler Bestandteil der modernen maschinellen Lernverfahren und haben eine breite Anwendung in Bereichen wie Bild- und Spracherkennung, natürliche Sprachverarbeitung und viele andere. Ein grundlegendes und weit verbreitetes Modell ist das Multi-Layer-Perceptron (Mehrschichtiges Perzeptron, MLP), das zu den ersten entwickelten künstlichen neuronalen Netzwerkarchitekturen gehört.

2.1.1 Multi-Layer-Perceptron

Ein Multi-Layer-Perceptron ist ein Netzwerk aus mehreren Layern (Schichten) von Neuronengruppen, die komplexe Muster in Daten erkennen können. Es besteht typischerweise aus drei Hauptteilen: einem Input-Layer (Eingabeschicht), einer oder mehreren Hidden-Layern (verborgenen Schichten) und einer Output-Layer (Ausgabeschicht). Die Neuronen benachbarter Layer sind jeweils miteinander verbunden (wie in Abb. 2.1 dargestellt).

Die Input-Layer eines MLP empfängt die rohen Daten. Jedes Neuron in dieser Layer repräsentiert ein Merkmal der Eingabedaten. Beispielsweise könnten in einem Bildverarbeitungsproblem die Neuronen der Input-Layer die Intensitäten einzelner Pixel eines Bildes darstellen.

Die Hidden-Layer, zentral für die Funktionsweise eines MLP, bestehen aus Neuronen, deren Aktivierung durch die Berechnung gewichteter Summen der Eingaben aus dem vorherigen Layer erfolgt. Diese Berechnung wird mathematisch wie folgt dargestellt:

$$z^{(l)} = W^{(l)} x^{(l-1)} + b^{(l)} \quad , \tag{2.1}$$

$$a^{(l)} = \sigma(z^{(l)}) \quad , \tag{2.2}$$

wobei $W^{(l)}$ die Gewichtsmatrix, $b^{(l)}$ der Bias-Vektor und σ die Aktivierungsfunktion der l-ten Layer sind. $x^{(l-1)}$ ist der Aktivierungsvektor der vorherigen Layer.

Diese Transformationen ermöglichen es dem MLP, komplexe Muster und Zusammenhänge in den Daten zu erkennen und zu modellieren. Die Anzahl der Hidden-Layer korreliert direkt mit

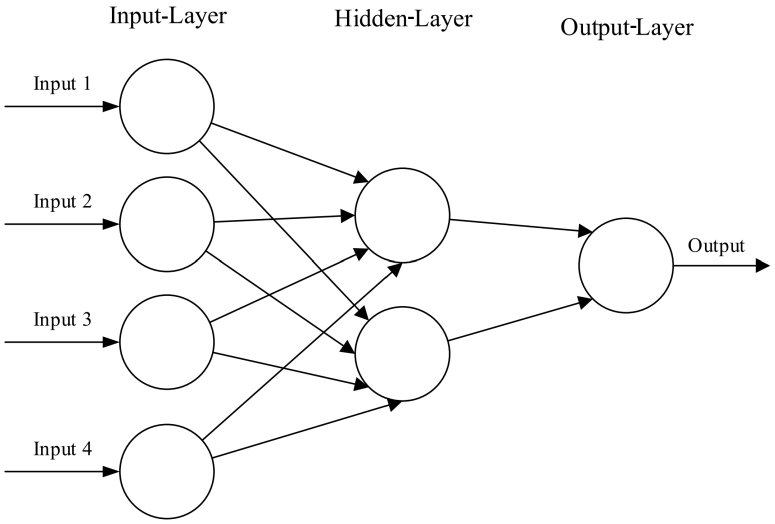


Abb. 2.1: Schematische Darstellung eines Multi-Layer-Perceptron. Zu sehen sind Input-Layer, Hidden-Layer und Output-Layer. Jedes Neuron aus einer Layer ist immer mit jedem Neuron der vorherigen Layer verbunden [36].

der strukturellen Komplexität des Netzwerks, seiner 'Tiefe' und dem Umfang der erforderlichen Rechenkapazität. Mit zunehmender Anzahl an Hidden-Layers verbessert sich potenziell die Fähigkeit des Netzwerks, komplexe Muster und Beziehungen in den Daten zu erkennen und zu lernen, womit jedoch eine höhere Rechenkapazität erforderlich ist.

Die Output-Layer bildet die finalen Prädiktion des Netzwerks. Die Konfiguration dieser Layer, insbesondere die Anzahl und Art der Neuronen, wird von der spezifischen Aufgabenstellung des Netzwerks bestimmt. Für Klassifizierungsaufgaben kann die Output-Layer beispielsweise aus einem Neuron pro Klasse bestehen, wobei jedes Neuron eine Wahrscheinlichkeit ausgibt, die angibt, mit welcher Wahrscheinlichkeit eine Eingabe zu einer bestimmten Klasse gehört. Dafür wird die Softmax-Funktion auf die Neuronen angewendet.

Aktivierungsfunktionen in Neuronalen Netzwerken

Aktivierungsfunktionen spielen eine zentrale Rolle in neuronalen Netzwerken, da sie es ermöglichen, Nichtlinearität in das Modell einzuführen. Ohne sie würden neuronale Netzwerke lediglich lineare Transformationen durchführen, was ihre Fähigkeit, komplexe Muster in den Daten zu lernen, erheblich einschränken würde. Eine Aktivierungsfunktion wird auf ein Neuron angewendet, um die Ausgabe dessen zu bestimmen.

Es gibt mehrere Typen von Aktivierungsfunktionen, die in der Praxis eingesetzt werden:

- **Sigmoid-Funktion:** Definiert als $\sigma(x) = \frac{1}{1+e^{-x}}$, mit einem Output-Bereich von (0, 1). Ein wesentlicher Vorteil dieser Aktivierungsfunktion ist ihre einfache Differenzierbarkeit, während ein Nachteil das Problem des verschwindenden Gradienten darstellt.

- **Hyperbolische Tangensfunktion (tanh):** Eine skalierte Version der Sigmoid-Funktion, definiert als $\tanh(x) = \frac{e^{x}-e^{-x}}{e^{x}+e^{-x}}$. Der Ausgabebereich liegt zwischen (-1 und 1). Leidet wie Sig-

moid unter dem verschwindenden Gradienten.

- **ReLU-Funktion:** Rectified Linear Unit, definiert als $\text{ReLU}(x) = \max(0,x)$. Aktiviert ein Neuron nur, wenn die Eingabe positiv ist, was effiziente und schnelle Konvergenz fördert. Kann jedoch zu "toten Neuronen"führen.

- **Softmax-Funktion:** Wird nur bei der Ausgabe von Netzwerken für Klassifikationsaufgaben mit mehreren Klassen verwendet. Sie normalisiert die Ausgaben zu einer Wahrscheinlichkeitsverteilung über die Klassen, definiert als $\text{Softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$, wobei x_i die Ausgabe des Neurons vor Anwendung der Softmax-Funktion ist.

Durch diese Funktionen wird eine effektive Einführung von Nichtlinearitäten ermöglicht, die für die Modellierung komplexer Funktionszusammenhänge in den Trainingsdaten entscheidend sind.

2.1.2 Lernen in Neuronalen Netzwerken

Das Lernen in Neuronalen Netzen wird durch Backpropagation ermöglicht, meist in Kombination mit einem Optimierungsalgorithmus wie dem Stochastic Gradient Descent (SGD). Diese Methoden ermöglichen es, die Gewichte des Netzwerks systematisch anzupassen, um die Leistung des Modells zu verbessern.

Backpropagation

Backpropagation ist ein Mechanismus zum effizienten Berechnen der Gradienten der Lossfunktion bezüglich aller Gewichte im Netzwerk. Der Prozess beginnt damit, dass am Ende des Netzwerks der Fehler zwischen der tatsächlichen Ausgabe und der vom Netzwerk projizierten Ausgabe berechnet wird. Dieser Fehler wird durch die Lossfunktion quantifiziert. Der berechnete Fehler wird dann verwendet, um die Gradienten der Lossfunktion bezüglich jeder Gewichtung durch die Anwendung der Kettenregel der Differentialrechnung zu berechnen. Für ein Gewicht $W_{ij}^{(l)}$ im Netzwerk wird der Gradient durch folgende Formel berechnet:

$$\frac{\partial L}{\partial W_{ij}^{(l)}} = \frac{\partial L}{\partial a_j^{(l)}} \cdot \frac{\partial a_j^{(l)}}{\partial z_j^{(l)}} \cdot \frac{\partial z_j^{(l)}}{\partial W_{ij}^{(l)}} \quad , \tag{2.3}$$

wobei $a_j^{(l)}$ die Aktivierung, $z_j^{(l)}$ die gewichtete Summe der Eingaben ist und $W_{ij}^{(l)}$ das Gewicht, das die Verbindung von Neuron i in Layer $l-1$ zu Neuron j in Layer l repräsentiert.

Stochastic Gradient Descent (SGD)

Nachdem die Gradienten berechnet wurden, verwendet SGD diese Informationen, um die Gewichte in die Richtung zu aktualisieren, die den Verlust minimiert. Im Gegensatz zum klassischen Gradientenabstieg, der den Durchschnitt der Gradienten über den gesamten Datensatz benötigt,

arbeitet SGD mit einem zufällig ausgewählten Teil des Datensatzes, einem Mini-Batch. Diese Methode reduziert die Rechenlast erheblich und ermöglicht ein effizienteres und schnelleres Training, besonders bei großen Datensätzen. Die Gewichtsaktualisierung erfolgt nach der Formel:

$$W_{ij}^{(l)} = W_{ij}^{(l)} - \eta \frac{\partial L}{\partial W_{ij}^{(l)}} \quad , \tag{2.4}$$

wobei η die Lernrate ist, mit der die Stärke der Gewichtsänderungen bestimmt werden kann. Diese Technik ermöglicht es, dass das Netzwerk durch die zufällige Auswahl von Mini-Batches auch die potenzielle Überanpassung reduziert und eine robustere Generalisierung fördert. Durch wiederholtes Anwenden von Backpropagation und SGD über viele Trainingsdurchläufe (Epochen) hinweg lernt das Netzwerk, die Gewichte so anzupassen, dass der Fehler zwischen den prädizierten und tatsächlichen Ausgaben minimiert wird.

2.1.3 Lossfunktionen in Neuronalen Netzwerken

Die Lossfunktion, auch als Kostenfunktion oder Verlustfunktion bekannt, ist ein zentrales Element im Trainingsprozess neuronaler Netzwerke. Sie misst die Diskrepanz zwischen der Prädiktion des Modells und den tatsächlichen Zielwerten und gibt so ein Maß dafür, wie gut das Modell die Daten approximiert. Die Wahl der Lossfunktion hängt stark vom Typ des Lernproblems ab:

- **Mittlerer quadratischer Fehler (MSE):** Häufig verwendet für Regressionsprobleme, definiert als das Mittel der quadrierten Differenzen zwischen den prädizierten und den tatsächlichen Werten:
$$L_{\text{MSE}} = \frac{1}{n} \sum_{i=1}^{n} (y_i - t_i)^2, \tag{2.5}$$
wobei y_i der prädizierte Wert und t_i der tatsächliche Wert des i-ten Beispiels ist.

- **Mittlerer absoluter Fehler (MAE):** Ebenfalls für Regressionsprobleme genutzt, misst der MAE die durchschnittliche absolute Differenz zwischen den prädizierten Werten und den tatsächlichen Werten:
$$L_{\text{MAE}} = \frac{1}{n} \sum_{i=1}^{n} |y_i - t_i|, \tag{2.6}$$
wobei $|y_i - t_i|$ die absolute Differenz ist. Der MAE ist besonders robust gegenüber Ausreißern.

- **Kreuzentropie:** Vorzugsweise verwendet für Klassifizierungsprobleme, insbesondere wenn die Modellausgaben als Wahrscheinlichkeiten der Zugehörigkeit zu Klassen interpretiert werden können. Sie wird berechnet als:
$$L_{\text{CE}} = -\sum_{i=1}^{C} t_i \log(y_i), \tag{2.7}$$
wobei C die Anzahl der Klassen, t_i das wahre Label der Klasse i als One-Hot-Vektor, und y_i die vom Modell prädizierte Wahrscheinlichkeit ist, dass die Eingabe zur Klasse i gehört.

2.2 Klassifikation und deren Bewertung

In dieser Arbeit spielen Klassifikation eine entscheidende Rolle. Dabei gilt es zu untersuchen, in welchem Maß die Prädiktionen des Modells korrekt oder fehlerhaft einer Klasse zugerdnet werden. Die systematische Bewertung ist deshalb von zentraler Bedeutung, um die angewandten Fusionsstrategien zu bewerten. In diesem Abschnitt werden dafür fundamentale Metriken erörtert.

Confusion Matrix

Die Confusion Matrix (Konfusionsmatrix) stellt die Anzahl der korrekten und fehlerhaften Prädikationen des Modells zu einer Klasse in einer Matrixform dar. Diese Matrix ist in vier Quadranten unterteilt, die jeweils die Anzahlen der wahren Positiven (TP), der wahren Negativen (TN), der falschen Positiven (FP) und der falschen Negativen (FN) anzeigen. Diese Werte sind essenziell für die Berechnung weiterführender Metriken wie Accuracy, Precision, Sensitivity und F1-Score, welche tiefere Einblicke in die Leistungsfähigkeit des Modells ermöglichen.

$$\text{Confusion Matrix} = \begin{array}{|c|c|c|} \hline & \textbf{Prädiktion Positiv} & \textbf{Prädiktion Negativ} \\ \hline \textbf{Tatsächlich Positiv} & \text{TP (True Positives)} & \text{FN (False Negatives)} \\ \hline \textbf{Tatsächlich Negativ} & \text{FP (False Positives)} & \text{TN (True Negatives)} \\ \hline \end{array} \quad [2.8]$$

Metriken

- **Accuracy (Genauigkeit)**: Misst den Prozentsatz der insgesamt korrekt klassifizierten Instanzen (sowohl True Positiv als auch True Negativ) und ist eine der grundlegendsten Validierungsmetriken.
$$\text{Accuracy} = \frac{TP+TN}{TP+TN+FP+FN} \quad . \quad [2.9]$$

- **Precision (Präzision)**: Gibt an, welcher Anteil der als positiv klassifizierten Instanzen tatsächlich positiv ist. Sie ist besonders wichtig in Situationen, in denen die Kosten für falsch positive Ergebnisse hoch sind.
$$\text{Precision} = \frac{TP}{TP+FP} \quad . \quad [2.10]$$

- **Recall (Sensitivität)**: Der Recall misst, welcher Anteil der tatsächlich positiven Instanzen als positiv erkannt wurde. Diese Metrik ist entscheidend, wenn die Kosten für falsch negative Ergebnisse gravierend sind.
$$\text{Recall} = \frac{TP}{TP+FN} \quad . \quad [2.11]$$

- **F1-Score**: Ist das harmonische Mittel von Precision und Recall und gibt ein Maß für die Balance zwischen diesen beiden Metriken. Er wird oft verwendet, wenn ein Gleichgewicht

zwischen Precision und Recall angestrebt wird.

$$\text{F1-Score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad . \tag{2.12}$$

- **Precision-Recall-Kurve** (Abb. 2.2): Zeigt den Trade-off zwischen Präzision und Recall für verschiedene Schwellenwerte (Thresholds) des Klassifikators. Der Schwellenwert bestimmt, ab welchem Wert eine Prädiktion als korrekt klassifiziert wird. Ein hoher Schwellenwert führt zu einer hohen Precision, jedoch auch zu einem niedrigen Recall. Ein niedriger Schwellenwert führt zu einem niedrigen Recall, jedoch zu einem hohen Precision. Eine hohe Fläche unter der Precision-Recall-Kurve bedeutet, dass der Klassifikator sowohl eine hohe Recall- als auch eine hohe Precision erreicht.

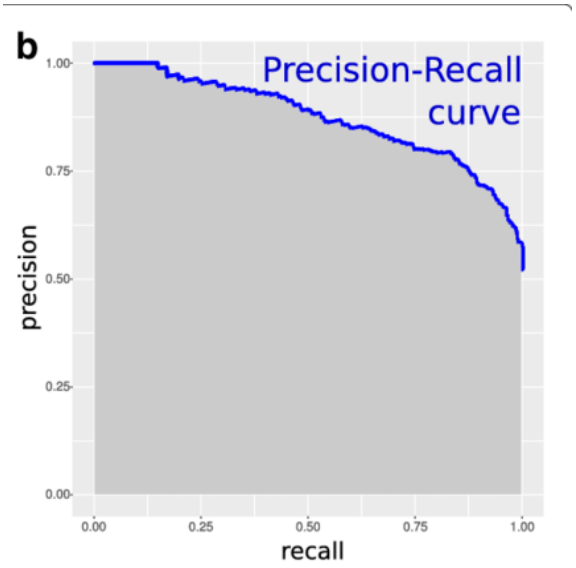


Abb. 2.2: Beispiel einer Precision-Recall-Kurve, wobei die Precision auf der y-Achse und der Recall auf der x-Achse abgetragen sind. Der graue Bereich ist die Fläche unter der Precision-Recall-Kurve [7].

- **Mean-Average-Precision (mAP)**: Ist eine Metrik zur Validierung der Leistung von Modellen, insbesondere in Aufgaben wie der Objekterkennung. Sie wird durch das Mittel der Average-Precision (AP) über alle Klassen berechnet. Für eine Klasse k wird die Average-Precision wie folgt berechnet:

$$\text{AP}_k = \int_0^1 \text{Precision}(r)\, dr \quad , \tag{2.13}$$

wobei r den Recall darstellt. Die mAP wird anschließend durch das Mittel der AP-Werte aller N Klassen berechnet:

$$\text{mAP} = \frac{1}{N} \sum_{k=1}^{N} \text{AP}_k \quad . \tag{2.14}$$

Diese Metriken zusammen geben einen umfassenden Überblick über die Leistung eines Klassifikators und sind entscheidend für das Verständnis seiner Stärken und Schwächen in praktischen Anwendungen.

2.3 Convolutional Neural Networks

Convolutional Neural Networks (CNNs) [10] sind seit den neunziger Jahren in der Entwicklung und wurden ursprünglich für Aufgaben wie die Zeichenerkennung verwendet [21]. Die Popularität von CNNs stieg jedoch signifikant an, als Alex Krizhevsky et al. im Jahr 2012 die ImageNet Challenge gewannen [19]. Diese Netze sind nun zentral für Fortschritte in Bereichen wie Bilderkennung und maschinelles Sehen. Ein wichtiges Beispiel für die Evolution von CNNs ist das Residual Neural Network (ResNet), das 2015 eingeführt wurde und eine fortgeschrittene Netzwerkarchitektur darstellt. ResNet verwendet „Residual Blocks“, um die Trainingsfähigkeit von extrem tiefen Netzwerken zu ermöglichen [14].

Ein typisches Convolutional Neural Network (CNN) setzt sich aus zwei Hauptkomponenten zusammen: der diskreten Convolution und dem Pooling-Layer. Diese Komponenten sind entscheidend für die Leistungsfähigkeit von CNNs in verschiedenen Anwendungen wie in der Bildverarbeitung.

Diskrete Convolution

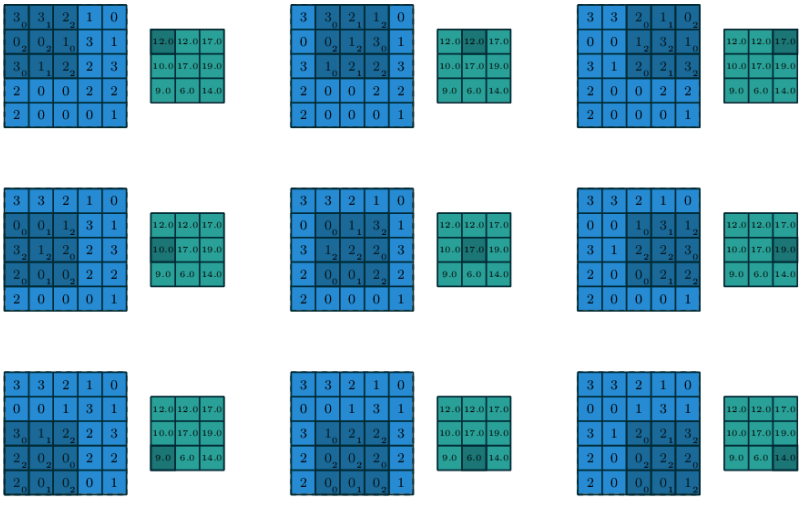


Abb. 2.3: Beispiel einer Convolution. Das hellblaue Gitter stellt die Input-Feature-Map dar. Um die Darstellung einfach zu halten, wird nur eine einzige Input-Feature-Map gezeigt, obwohl es üblich ist, mehrere Feature-Maps übereinander zu stapeln. Ein schattierter Kernel bewegt sich über die Input-Feature-Map. An jeder Position wird das Produkt jedes Elements des Kernels mit dem überlappenden Eingabeelement berechnet und die Ergebnisse werden summiert, um die Ausgabe an der aktuellen Position zu erhalten [10].

Die Grundlage der neuronalen Netze bilden Multi-Layer-Perceptron, siehe Kapitel 2.1.1. Ein Vektor wird als Eingabe mit der jeweiligen Gewichtsmatrix multipliziert (siehe Formel 2.1), um ein Ergebnis zu erzeugen, zu dem in der Regel ein Bias-Vektor hinzugefügt wird, bevor das Ergebnis

durch eine Aktivierungsfunktion (wie in Kapitel 2.1.1 dargelegt) geleitet wird. Diese Transformation ist auf mehrere Input-Modalitäten anwendbar, z.B. Bilder, Text oder Zeitreihen. Unabhängig von ihrer Dimensionalität kann jede Darstellung immer in einen Input-Vektor umgewandelt werden, bevor die Transformation durchgeführt wird. Bilder und viele andere ähnliche Datenarten weisen eine intrinsische Struktur auf. Genauer gesagt teilen sie wichtige Eigenschaften:

- Sie werden als mehrdimensionale Arrays gespeichert.
- Sie besitzen eine oder mehrere Achsen, bei denen die Reihenfolge wichtig ist (zum Beispiel die Breiten- und Höhenachsen bei einem Bild).
- Eine Achse, die als Channel-Achse bezeichnet wird, wird verwendet, um auf unterschiedliche Ansichten der Daten zuzugreifen (zum Beispiel die Rot-, Grün- und Blaukanäle eines Farbbildes).

Diese Eigenschaften werden bei einem Multi-Layer-Perceptron nicht ausgenutzt, tatsächlich werden alle Achsen gleich behandelt und die topologischen Informationen werden nicht berücksichtigt. Dennoch kann das Ausnutzen der impliziten Struktur der Daten bei der Lösung einiger Aufgaben, wie der Computer Vision, sehr nützlich sein. In solchen Fällen ist es vorteilhaft, die zugrunde liegende Struktur zu bewahren. Hierbei kommen diskrete Convolutionen zum Einsatz.

Eine diskrete Convolution ist eine lineare Transformation, die diese Ordnung beibehält. Sie ist sparse (nur wenige Eingabeeinheiten tragen zu einer gegebenen Ausgabeeinheit bei) und verwendet Parameter wieder (dieselben Gewichte werden auf mehrere Stellen in der Eingabe angewendet). Abbildung 2.3 zeigt ein Beispiel für eine diskrete Convolution. Das hellblaue Gitter wird als Input-Feature-Map bezeichnet. Um die Darstellung einfach zu halten, wird nur eine einzige Input-Feature-Map dargestellt, es ist jedoch nicht ungewöhnlich, dass mehrere Feature-Maps übereinander gestapelt werden. Ein Kernel (das schattierte Gebiet) bewegt sich sequenziell über die Input-Feature-Map. An jedem Ort wird das Produkt zwischen jedem Element des Kernels und dem Inputelement, mit dem es sich überschneidet, berechnet und die Ergebnisse werden summiert. Damit wird die Ausgabe an der aktuellen Position zu erhalten. Die endgültigen Ausgaben dieses Verfahrens werden als Output-Feature-Maps bezeichnet. Wenn mehrere Input-Feature-Maps vorliegen, die durch eine zusätzliche Dimension für Channels gekennzeichnet sind, wird eine Vielzahl von Kernels eingesetzt. Die Gesamtheit dieser Kernels bildet den Filter.

Die in Abbildung 2.3 gezeigte Convolution repräsentiert ein Beispiel einer 2-D-Convolution, kann jedoch auf N-D-dimensionale Convolutionen erweitert werden. Zusätzlich ist der Stride definiert, der den Abstand zwischen zwei aufeinanderfolgenden Positionen des Kernels entlang einer Achse festlegt. Der Stride stellt eine Form des Subsampling dar.

Pooling-Layer

Pooling-Layer dienen dazu, die Größe von Feature-Maps zu reduzieren, was nicht nur die Rechenanforderungen für nachfolgende Operationen senkt, sondern auch dazu beiträgt, die räumliche Varianz der Features zu verringern. Pooling ist eine Form des Downsamplings.

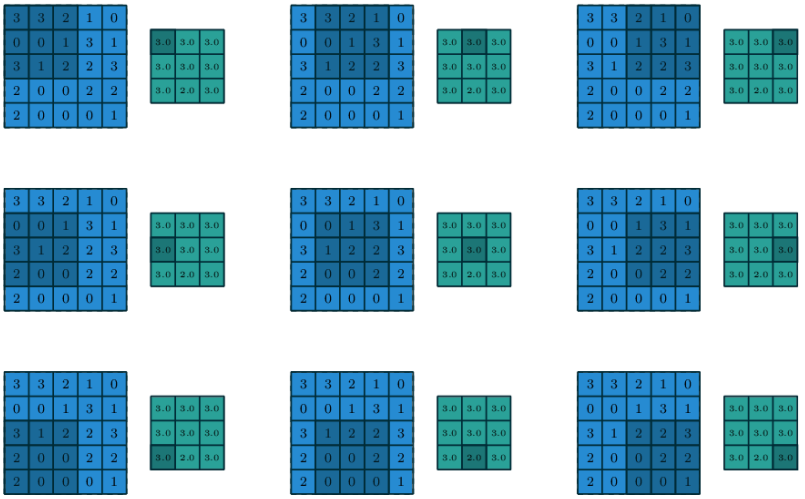


Abb. 2.4: Berechnung der Pooling-Layer einer 3 × 3 Max-Pooling-Operation auf einer 5 × 5 Input-Matrix mit 1 × 1 Strides. Diese Abbildung zeigt, wie das Max-Pooling-Window über die gesamte Input-Feature-Map bewegt sich sequenziell, bei jedem Schritt den maximalen Wert innerhalb des 3 × 3 Bereichs extrahiert und diesen Wert in die entsprechende Position der Output-Feature-Map setzt [10].

In einem typischen Pooling-Prozess wird ein Fenster, auch als Pooling-Window bezeichnet, über die gesamte Input-Feature-Map geschoben. Für jedes Pooling-Window wird eine spezifische Operation durchgeführt, um einen einzelnen Ausgabewert zu erzeugen. Die gängigsten Pooling-Operationen sind das Max-Pooling und das Average-Pooling:

- **Max-Pooling** extrahiert den maximalen Wert aus dem betrachteten Pooling-Window. Dies ist besonders nützlich, um die prägnantesten Features einer Feature-Map hervorzuheben, was das Netzwerk robuster gegenüber kleineren Variationen und Rauschen in der Input-Feature-Map macht.

- **Average-Pooling** berechnet den Durchschnitt aller Werte in dem Pooling-Window. Diese Methode trägt dazu bei, die gesamten Aktivierungen innerhalb des Pooling-Windows zu glätten, was zu einer weniger diskriminativen, aber auch weniger sensitiven Feature-Repräsentation führt.

Durch die Anpassung verschiedener Hyperparameter wie die Anzahl und Größe der Filter, Stride und die Art der Pooling-Operation kann die Leistung eines Convolutional Neural Networks optimiert werden. Jedes Durchlaufen einer Convolution-Operation verkleinert potenziell die Größe der Feature-Map, abhängig von den gewählten Stride- und Pooling-Parametern. Während dieses Prozesses werden semantische Informationen über die Features innerhalb der Feature-Map extrahiert, wodurch wichtige Charakteristika der Eingabedaten hervorgehoben werden. Allerdings führt jede Iteration zu einem Informationsverlust hinsichtlich der exakten Lokalisierung der ursprünglichen Merkmale in der Input-Feature-Map.

2.4 Feature-Pyramid-Networks

Feature-Pyramid-Networks (FPN) [29] sind eine fortschrittliche Netzwerkarchitektur, die speziell für die Aufgabe der Objekterkennung entwickelt wurden. Sie nutzen die intrinsische, mehrstufige, pyramidenförmige Hierarchie von tiefen Convolutional Neural Networks (CNNs, siehe Kapitel 2.3). Diese Architektur ermöglicht es, hochgradig informative Feature-Maps aus verschiedenen Skalenebenen des Netzwerks effizient zu nutzen und zu kombinieren. Durch die Kombination von Bottom-up und Top-down Pathways mit lateralen Verbindungen gelingt es den FPNs, semantisch reiche Feature-Maps auf verschiedenen Skalenebenen zu generieren, ohne signifikante zusätzliche Rechenkosten zu verursachen (wie in Abb. 2.5 dargestellt).

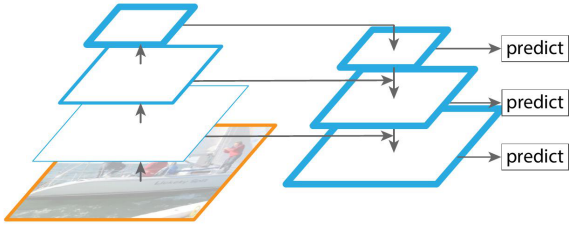


Abb. 2.5: Schematische Darstellung eines Feature-Pyramid-Networks (FPN). Die Architektur nutzt sowohl einen Bottom-up (links) als auch einen Top-down Pathway (rechts), um Feature-Maps verschiedener Skalenebenen zu generieren und zu verfeinern. Die lateralen Verbindungen kombinieren Merkmale aus beiden Richtungen, um semantisch reichhaltigere und räumlich präzisere Vorhersagen zu ermöglichen. Diese Methode verbessert die Objekterkennung über verschiedene Größenskalen hinweg [29].

Der Bottom-up Pathway in einem FPN ist der Vorwärtsdurchlauf durch das Backbone des CNNs, welches eine Hierarchie von Feature-Maps auf verschiedenen Skalierungsstufen erzeugt (wie links in Abb. 2.5 dargestellt). Durch das Runterskalieren mit jeder weiteren Stufe im Netzwerk reduziert sich die räumliche Größe der Feature-Maps, was typisch für die zunehmende Semantik in tiefen CNNs ist. Für jede Stufe des Netzwerks definiert das FPN eine Pyramidenebene. Der tiefste Layer jeder Stufe wird ausgewählt, um als Referenzset für die Feature-Maps zu dienen, die anschließend aggregiert werden, um die Pyramide zu konstruieren. Dies reflektiert die natürliche Wahl, da die tiefsten Layer die stärksten Features aufweisen sollten. Speziell für ResNet-Architekturen werden die Aktivierungen des letzten Residual-Blocks jeder Stufe genutzt.

Der Top-down konstruiert höher aufgelöste Feature-Maps durch das Hochskalieren der räumlich gröberen, aber semantisch stärkeren Feature-Maps der höheren Pyramidenebenen (wie rechts in Abb. 2.5 dargestellt). Diese hochskalierten Features werden dann durch laterale Verbindungen mit den Feature-Maps gleicher räumlicher Dimensionen des Bottom-up Pathway aggregiert. Die Feature-Map des Bottom-up Pathway weist eine geringere semantische Abstraktion auf, jedoch sind ihre Features präziser lokalisiert, da sie weniger oft herunterskaliert wurden. Schließlich wird auf jede zusammengeführte Feature-Map eine Convolution angewendet, um die endgültige Feature-Map zu erzeugen.

2.5 Mini-Batch-Normalisierung

Die Normalisierung jeder Feature-Map zu Beginn jedes Layers ist ein entscheidender Schritt zur Beschleunigung der Konvergenz während des Trainings von neuronalen Netzwerken.

Die Batch-Normalisierung [16] ist eine einfache Form der Normalisierung, die jede Eingangsdimension unabhängig normalisiert.

Die Herausforderung beim Training tiefer neuronaler Netzwerke liegt in der dynamischen Veränderung der Eingabedatenverteilung jedes Layers, verursacht durch Modifikationen der Parameter in den vorgelagerten Schichten. Diese Veränderung der Datenverteilung, bekannt als Internal-Covariate-Shift, verlangsamt den Trainingsprozess erheblich. Sie erfordert niedrigere Lernraten und eine sorgfältige Initialisierung der Parameter, was insbesondere das Training von Modellen mit starken Nichtlinearitäten erschwert. Die Batch-Normalisierung adressiert dieses Problem effektiv, indem sie die Eingaben jedes Layers normalisiert und somit den Internal-Covariate-Shift minimiert.

Für einen Layer mit d-dimensionalem Input $x = (x^{(1)}, \ldots, x^{(d)})$ wird jedes Feature $x^{(k)}$ wie folgt normalisiert:

$$\hat{x}^{(k)} = \frac{x^{(k)} - E[x^{(k)}]}{\sqrt{\mathrm{Var}[x^{(k)}]}} \quad . \tag{2.15}$$

Hierbei sind $E[x^{(k)}]$ und $\mathrm{Var}[x^{(k)}]$ der Erwartungswert und die Varianz, die über das gesamte Trainingsset berechnet werden. Wie in [22] dargestellt, beschleunigt eine solche Normalisierung die Konvergenz, selbst wenn die Features nicht dekorreliert sind.

Die direkte Normalisierung jedes Inputs könnte die Repräsentationsfähigkeit des Layers verändern. Um dies zu adressieren und sicherzustellen, dass die Transformation in das Netzwerk die Identitätstransformation repräsentieren kann, führt die Batch-Normalisierung skalierbare und verschiebbare Parameter für jede Aktivierung $x^{(k)}$ ein:

$$y^{(k)} = \gamma^{(k)} \hat{x}^{(k)} + \beta^{(k)} \quad . \tag{2.16}$$

Diese Parameter $\gamma^{(k)}$ und $\beta^{(k)}$ werden zusammen mit den ursprünglichen Modellparametern gelernt und ermöglichen es, die Repräsentationskraft des Netzwerks wiederherzustellen.

Implementierung in Mini-Batches

In einem idealen Szenario, in dem jeder Trainingsschritt auf dem gesamten Trainingsset basiert, würde die Normalisierung auf dem gesamten Set durchgeführt. Dies ist jedoch in der Praxis bei der Verwendung von stochastischer Optimierung und Mini-Batches nicht praktikabel. Daher wird eine Vereinfachung vorgenommen: Jeder Mini-Batch produziert Schätzungen für den Mittelwert und die Varianz jeder Aktivierung. Diese Statistiken können dann vollständig in die Backpropagation (siehe Kapitel 2.1.2) integriert werden.

Betrachtet wird ein Mini-Batch B der Größe m. Da die Normalisierung auf jede Aktivierung unabhängig angewendet wird, liegt der Fokus auf einer spezifischen Aktivierung $x^{(k)}$ und es wird

k für Klarheit weggelassen. Für diesen Mini-Batch $B = \{x_1, \ldots, x_m\}$ wird der normalisierte Wert $\hat{x}_i$ und dessen lineare Transformation y_i wie folgt berechnet:

$$\mu_B = \frac{1}{m}\sum_{i=1}^{m} x_i \quad \text{(Mini-Batch-Mittel)} \qquad , \tag{2.17}$$

$$\sigma_B^2 = \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_B)^2 \quad \text{(Mini-Batch-Varianz)} \qquad , \tag{2.18}$$

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}} \quad \text{(Normalisierung)} \qquad , \tag{2.19}$$

$$y_i = \gamma\hat{x}_i + \beta \quad \text{(Skalierung und Verschiebung).} \tag{2.20}$$

Die Mini-Batch-Normalisierung kann somit in ein Netzwerk integriert werden, um jede Aktivierung zu manipulieren. Durch diese Normalisierung wird der Internal-Covariate-Shift reduziert, was die Stabilität des Netzwerks während des Trainings verbessert und zu schnellerer Konvergenz führt.

2.6 Layer-Normalisierung

Ähnlich zur Batch-Normalisierung bietet die Layer-Normalisierung eine Methode zur Reduktion der Trainingsdauer neuronaler Netzwerke [23]. Insbesondere bei Layern, die ReLU-Aktivierungsfunktionen nutzen, können Änderungen in den Outputs eines Layers zu stark korrelierten Änderungen in den summierten Inputs der nachfolgenden Layer führen. Diese Beobachtung legt nahe, dass das Problem des Covariate-Shift durch die Stabilisierung des Mittelwerts und der Varianz der summierten Inputs innerhalb jeder einzelnen Layer verringert werden kann.

Die Layer-Normalisierung berechnet die Normalisierungsstatistiken μ und σ für alle Neuronen einer Hidden-Layer wie folgt:

$$\mu^{(l)} = \frac{1}{H}\sum_{i=1}^{H} a_i^{(l)} \qquad , \tag{2.21}$$

$$\sigma^{(l)} = \sqrt{\frac{1}{H}\sum_{i=1}^{H}\left(a_i^{(l)} - \mu^{(l)}\right)^2} \qquad . \tag{2.22}$$

Dabei bezeichnet H die Anzahl der Neuronen in der Layer und $a_i^{(l)}$ die Aktivierung des i-ten Neurons in der l-ten Hidden-Layer. Im Unterschied zur Mini-Batch-Normalisierung, bei der die Normalisierungsterme μ und σ für alle Neuronen einer Hidden-Layer identisch sind und sich zwischen verschiedenen Trainingsinstanzen unterscheiden, eliminiert die Layer-Normalisierung die Abhängigkeit von der Mini-Batch-Größe.

2.7 Die Transformer-Architektur

Der Transformer ist ein leistungsfähiges Modell [41], das ursprünglich für maschinelle Übersetzung eingeführt wurde, mittlerweile aber auch in anderen Bereichen der Künstlichen Intelligenz

wie Textgenerierung, Bilderkennung und Sprachverarbeitung eingesetzt wird. Die Architektur basiert auf Self-Attention-Mechanismen und ist in ihrer ursprünglichen Form aus einem Encoder und einem Decoder aufgebaut.

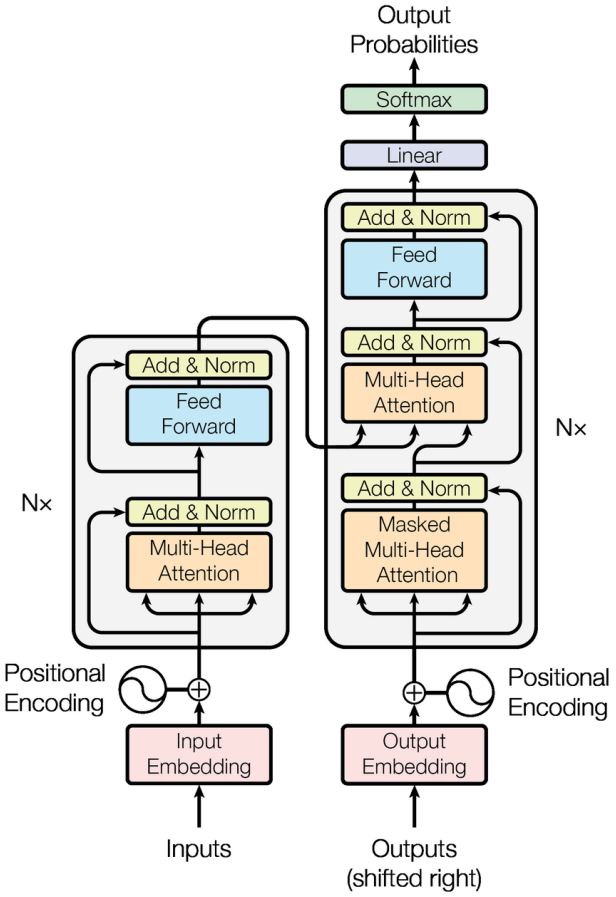


Abb. 2.6: Darstellung der Transformer-Architektur, links der Encoder und rechts der Decoder. Der Encoder besteht aus einer Stapelung von identischen Layern, wobei jede Layer aus einen Multi-Head-Self-Attention-Mechanismus und ein Position-Wise Fully-Connected Feed-Forward Network besteht. Jede Sub-Layer wird durch eine Residual-Verbindung und mit einer Layer-Normalisierung ergänzt. Der Decoder ergänzt die Architektur durch zusätzliche Multi-Headed-Cross-Attention-Schichten (in der Abb. Multi-Head Attention), welche die vom Encoder generierten Repräsentationen einbeziehen [41].

2.7.1 Der Encoder

Der Encoder (wie links in Abb. 2.6 dargestellt) dient dazu, eine Eingabesequenz von Symbolrepräsentationen $x_1, \ldots, x_n$ in eine Sequenz kontinuierlicher Repräsentationen $z = (z_1, \ldots, z_n)$ umzuwandeln. Diese kontinuierlichen Repräsentationen dienen dann als Eingabe für den Decoder, der die endgültige Ausgabesequenz generiert $(y_1, \ldots, y_n)$. Der Encoder eines klassischen Transformer-Modells besteht aus einer Stapelung von identischen Layern, wobei jede Layer zwei grundlegende Sub-Komponenten enthält: einen Multi-Head-Self-Attention-Mechanismus und ein Position-Wise Fully-Connected Feed-Forward Network (wie in Kapitel 2.7.4 dargestellt). Jede Sub-Layer wird durch eine Residual-Verbindung ergänzt, wie in He et al. [14] beschrieben. Diese Verbindung kom-

biniert den Input der Layer mit ihrem Output. Anschließend wird eine Layer-Normalisierung (wie in Kapitel 2.6 dargestellt) angewendet, um die Verteilung der Outputs zu standardisieren und die Konvergenz während des Trainings zu verbessern.

$$Y = L(X + S(X)) \quad , \tag{2.23}$$

wobei X der Input, $\mathrm{S}(X)$ die Funktion ist, die durch die jeweilige Sub-Layer implementiert wird, $\mathrm{L}(\cdot)$ die Layer-Normalisierung und Y den Output darstellt.

2.7.2 Der Decoder

Der Decoder (wie rechts in Abb. 2.6 dargestellt) nimmt die Sequenz kontinuierlicher Repräsentationen vom Encoder und generiert darauf aufbauend die Ausgabesequenz. Auch er besteht aus einer Stapelung von Layern. Jede dieser Layer enthält zusätzlich zur Multi-Head-Self-Attention und dem Position-Wise Fully-Connected Feed-Forward Network eine Multi-Headed-Cross-Attention. Ähnlich wie beim Encoder werden Residual-Verbindungen um jede Sub-Layer verwendet, gefolgt von einer Layer-Normalisierung. Zusätzlich wird die Self-Attention-Layer im Decoder so angepasst, dass Positionen nicht auf nachfolgende Positionen zugreifen können. Diese Maskierung, zusammen mit der positionsweisen Verschiebung der Ausgabeeinbettungen, gewährleistet, dass die Prädiktion für eine Position i ausschließlich auf bekannten Ausgabewerten an Positionen $j < i$ basiert.

2.7.3 Attention

Eine Attention-Funktion kann als eine Abbildung beschrieben werden, die eine Query (Abfrage) und eine Menge von Key-Value-Paaren (Schlüssel-Wert) auf eine Ausgabe abbildet. Query, Keys, Values und die Ausgabe sind allesamt Vektoren. Die Ausgabe wird als gewichtete Summe der Values berechnet, wobei das Gewicht jedes Values durch eine Kompatibilitätsfunktion der Query mit dem entsprechenden Key bestimmt wird.

Scaled Dot-Product Attention

Die Eingaben bestehen aus Query, Keys der Dimension d_k sowie Values der Dimension d_v. Das Skalarprodukt der Queries mit allen Keys wird berechnet, jedes Ergebnis wird durch $\sqrt{d_k}$ geteilt und anschließend wird eine Softmax-Funktion angewendet, um die Gewichte für die Values zu erhalten, wie in Abb. 2.7 dargestellt.

In der Praxis wird die Attention-Funktion auf eine Menge von Queries gleichzeitig angewendet und in die Q zusammengefasst. Die Keys und Values werden ebenfalls in Matrizen K und V zusammengefasst. Die Ausgabe wird wie folgt berechnet:

$$\mathrm{Attention}(Q,K,V) = \mathrm{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad . \tag{2.24}$$

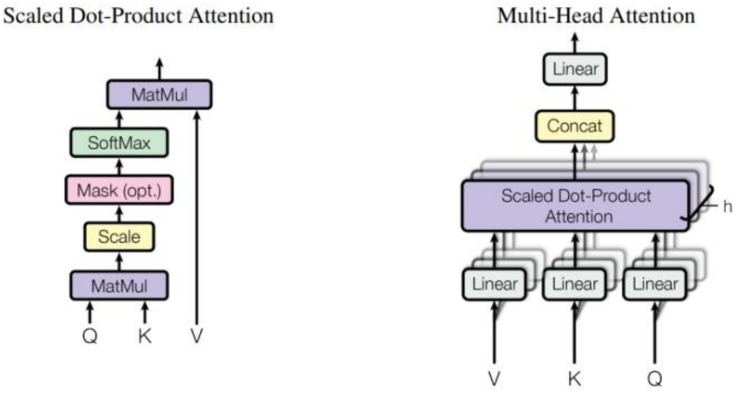


Abb. 2.7: Die Abbildung zeigt die zentralen Mechanismen der Attention im Transformer-Modell. (Links) Die scaled Dot-Product Attention illustriert den Prozess der Berechnung, wie in Gleichung 2.24 dargestellt: Die Eingabematrizen Q (Queries) und K (Keys) werden zunächst durch eine Matrixmultiplikation MatMul kombiniert. Anschließend erfolgt eine Skalierung, optional eine Maskierung und eine Normalisierung durch die Softmax-Funktion. Die berechneten Attention-Werte werden schließlich mit den Werten V (Values) multipliziert, um die Ausgabe zu erzeugen. (Rechts) Die Multi-Head Attention erweitert diesen Mechanismus, indem sie h parallele scaled Dot-Product Attention-Layer nutzt. Zunächst werden Q, K und V durch lineare Transformationen projiziert. Die Ergebnisse aus allen h Attention-Heads (wie in Gleichung 2.26 dargestellt) werden entlang der Feature-Dimension konkateniert Concat (wie in Gleichung 2.25 dargestellt) und durch eine finale lineare Transformation kombiniert, um die endgültige Ausgabe zu erzeugen [41].

Multi-Head Attention

Anstelle einer einzigen Attention-Funktion verwendet der Transformer Multi-Head Attention, bei der h parallele Attention-Mechanismen genutzt werden, wie in Abb. 2.7 dargestellt. Jeder Attention-Head arbeitet in einem anderen Repräsentationsraum:

$$\text{MultiHead}(Q,K,V) = \text{Concat}(\text{head}_1,...,\text{head}_h)W^O \quad , \tag{2.25}$$

wobei

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \quad . \tag{2.26}$$

1. Lineare Projektionen: Die Eingabematrizen Q, K, V (Queries, Keys, Values) werden zunächst mit den Parametermatrizen W_i^Q, W_i^K, und W_i^V linear projiziert, um die spezifischen Eingaben für jeden Attention-Head zu berechnen, siehe Formel 2.26. Dies ermöglicht, dass jeder Head auf unterschiedliche Aspekte der Daten fokussiert.

2. Parallele Berechnung: Die Attention-Funktion wird für jeden Head unabhängig berechnet, siehe Formel 2.24. Dadurch kann jeder Head Informationen aus verschiedenen Teilräumen der Repräsentation extrahieren.

3. Konkatenation und Transformation: Nach der Berechnung der Attention-Funktion für alle h Heads werden die Ausgabematrizen der Heads entlang der Feature-Dimension konkateniert, siehe Formel 2.25. Danach folgt eine finale lineare Transformation mit der Parametermatrix W^O,

um die endgültige Ausgabe der Multi-Head Attention zu erzeugen.

Die Multi-Head Attention ermöglicht es dem Modell, Informationen aus mehreren Teilräumen der Repräsentation gleichzeitig zu extrahieren. Abhängigkeiten zwischen unterschiedlichen Positionen in der Eingabesequenz können effizient erfasst werden. Die Verwendung mehrerer Heads steigert die Modellkapazität, ohne die Berechnungskomplexität drastisch zu erhöhen. Durch die Reduktion der Dimension jedes Head (d_k und d_v) auf d_{model}/h bleibt der Rechenaufwand der Multi-Head Attention vergleichbar mit dem eines einzelnen Attention-Mechanismus.

2.7.4 Position-Wise Fully-Connected Feed-Forward Network

Jede Layer des Encoders und Decoders enthält ein Position-Wise Fully-Connected Feed-Forward Network (ähnlich zu 2.1.1), das wie folgt definiert ist:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2 \quad , \tag{2.27}$$

wobei W_1, W_2, b_1 und b_2 die Parameter der linearen Transformationen sind und x der Eingabevektor.

Das Feedforward-Netzwerk wird auf jede Position unabhängig und identisch angewendet. Es besteht aus zwei linearen Transformationen, die durch eine ReLU-Aktivierungsfunktion miteinander verbunden sind.

2.7.5 Positional Encoding

Da der Transformer weder Recurrence noch Convolution (wie in Kapitel 2.3 dargestellt) verwendet, werden Positionsinformationen explizit durch Positional Encodings eingebracht. Die Dimensionen der Positional Encodings entsprechen d_{model}, wodurch sie direkt mit den Input-Embeddings summiert werden können. Die Input-Embeddings haben ebenfalls die Dimension d_{model} und stellen die numerische Repräsentation der Eingabesequenz dar. Sie sind Vektorrepräsentationen, die semantische Informationen der Eingabe, wie von Texten oder Bildern, enthalten. Durch die Addition der Positional Encodings und der Input-Embeddings kann das Modell sowohl semantische als auch positionsbezogene Informationen in die Berechnungen einbeziehen. Diese summierte Repräsentation bildet die Eingabe für den Encoder und Decoder des Transformers.

Die ursprüngliche Transformer-Architektur verwendet Positional Encodings, die auf sinus- und cosinusförmigen Funktionen basieren. Diese werden wie folgt definiert:

$$\text{PE}(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \quad , \tag{2.28}$$

$$\text{PE}(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \quad , \tag{2.29}$$

wobei *pos* die Position und *i* die Dimension ist. Jede Dimension des Positional Encodings entspricht einer Sinuswelle, deren Wellenlängen eine geometrische Progression von 2π bis $10000 \cdot 2\pi$ bilden. Dies hat den Vorteil, dass es dem Modell ermöglicht, Sequenzen zu verarbeiten, deren Längen über die während des Trainings beobachteten Sequenzen hinausgehen. Dadurch kann das Modell theoretisch auch bei längeren Sequenzen, die es zuvor nicht gesehen hat, Positionen extrapolieren.

Neben den sinus- und cosinusförmigen Positional Encodings gibt es auch gelernte Positional Embeddings. Diese basieren auf trainierbaren Vektoren, die für jede Position separat gelernt werden. Allerdings sind sie bei der Inferenz weniger flexibel als sinusförmige Encodings, da das Modell in der Inferenz nur Positionen verarbeiten kann, die innerhalb der während des Trainings gesehenen Sequenzlängen liegen.

3 Verwandte Arbeiten

In diesem Kapitel werden grundlegende Konzepte der HD-Kartenerstellung (High-Definition Maps) sowie relevante Arbeiten im Bereich der BEV-Darstellung (Bird's Eye View) für autonome Fahrsysteme vorgestellt. Die BEV-Darstellung ist ein zentraler Aspekt in der Entwicklung autonomer Fahrsysteme. Sie ermöglicht eine Darstellung der Verkehrsumgebung aus einer oberen Perspektive, die eine übersichtliche und umfassende Visualisierung der Verkehrssituation bietet. Diese Art der Darstellung ist besonders in komplexen Verkehrsszenarien vorteilhaft, da sie es dem autonomen System ermöglicht, Verkehrsteilnehmer und Hindernisse klar zu erfassen und entsprechend zu reagieren. Ziel dieses Kapitels ist es, den aktuellen Stand der Technik und die wichtigsten Ansätze zu erläutern, um den Kontext der in dieser Arbeit verwendeten Methoden darzulegen.

3.1 Grundlagen und Übersicht zur HD-Kartenerstellung für autonome Fahrzeuge

HD-Karten sind digitale Karten mit einer außergewöhnlich hohen Präzision, die speziell für die Bedürfnisse des autonomen Fahrens entwickelt werden [11]. Sie enthalten geometrische, semantische und topologische Informationen der Fahrumgebung, wie zum Beispiel Fußgängerüberwege, Fahrspurtrennungen und Straßenränder. Die Erstellung von HD-Karten unterscheidet sich in Offline-Verfahren und Online-Verfahren. Bei den Offline-Verfahren kommen SLAM-basierte Methoden (Simultaneous Localization and Mapping) zum Einsatz, die durch umfangreiche und komplexe Verarbeitungspipelines gekennzeichnet sind und erhebliche Wartungsaufwendungen erfordern [39, 40, 47]. Online-Verfahren nutzen Daten von im Fahrzeug integrierten Sensoren wie Kameras, Radar (Radio Detection and Ranging) und LIDAR (Light Detection and Ranging) um in Echtzeit Darstellungen der Umgebung aus der Vogelperspektive zu generieren. Initiale Arbeiten dazu nutzten linienförmige Vorgaben, um offen geformte Fahrspuren auf Basis von Frontansichtsbildern zu generieren [3, 5, 30]. Diese Ansätze weisen Limitationen bei der Detektion von Kartenelementen mit arbiträren Formen auf. Auf Grundlage von BEV-Darstellungen und semantischen Segmentierungen basieren Arbeiten zur Erstellung rasterisierter HD-Karten [6, 15, 26, 48]. Diese HD-Karten weisen Defizite hinsichtlich vektorisierter, instanzspezifischer Informationen auf, wie etwa die Fahrspurstrukturen, welche für nachgelagerte Anwendungen wie Bewegungsvorhersage und Planung kritisch sind. Dies erfordert umfangreiche Nachbearbeitungen, um die HD-Karten für den Einsatz in Navigations- und Entscheidungsprozessen adaptieren zu können.

Um vektorisierte HD-Karte zu erstellen, nutzt HDMapNet [25] die Resultate der pixelweisen Segmentierung und gruppiert diese. Dieser Prozess erfordert komplexe und zeitintensive Nachbearbeitungen. Solche Nachbearbeitungen sind notwendig, um die pixelbasierten Informationen in eine strukturierte, vektorbasierte Form zu überführen. VectorMapNet [31] verwendet eine Darstellung jedes Kartenelements als Punktsequenz. Dieses System implementiert ein kaskadiertes Fra-

mework von grob nach fein und setzt einen auto-regressiven Decoder ein, um Punkte sequenziell vorherzusagen, was zu langen Inferenzzeiten führt. Ein Framework ist eine strukturierte Sammlung von Richtlinien und Bibliotheken, die die Entwicklung komplexer Softwareanwendungen unterstützt. Inferenzzeiten bezeichnen die Dauer, die ein maschinelles Lernmodell benötigt, um Vorhersagen auf Basis neuer Daten zu treffen, nachdem es bereits trainiert wurde. Online-Methoden zur Konstruktion vektorisierter HD-Karten sind durch diese Effizienzeinschränkungen begrenzt und eignen sich nicht für Echtzeitanwendungen.

3.2 MapTR

MapTR [27] ist ein umfassendes Framework zur effizienten Erstellung vektorisierter HD-Karten in Echtzeit. Es bildet die fundamentale Basis der in dieser Arbeit verwendeten Netzwerkarchitektur (wie in Kapitel 4.1 dargestellt). Die MapTR-Architektur [27] basiert auf der Encoder-Decoder-Transformer-Struktur (wie in Kapitel 2.7 dargestellt). Diese Transformer-Struktur basiert auf der DETR-Architektur (End-to-End Object Detection with Transformers) [4]. MapTR verarbeitet Daten von fahrzeugmontierten Kameras und Sensoren wie LIDAR. Die Architektur generiert vektorisierte HD-Karten, die geometrische, semantische und topologische Informationen über die Fahrumgebung enthalten. Aus diesem umfassenden Framework werden als Input nur die sechs Kamerabilder im Folgenden genutzt.

3.2.1 Architektur

MapTR implementiert ein Encoder-Decoder-Transformer. Die gesamte Architektur ist in Abbildung 3.1 dargestellt.

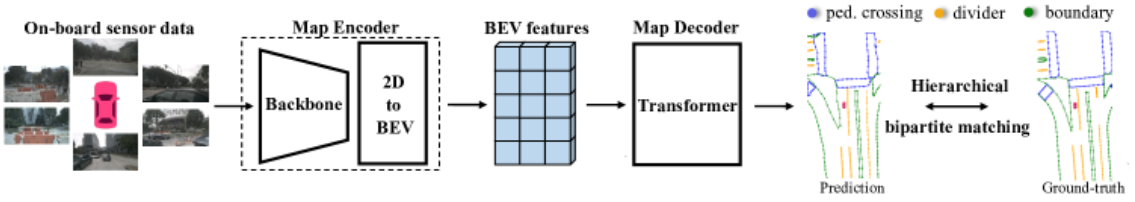


Abb. 3.1: Schematische Darstellung der Pipeline von MapTR. Die Kamerabilder aus Bodenperspektive (On-board-Sensordaten) werden im Map-Encoder von 2D zu BEV-Features transformiert. Die BEV-Features werden anschließend durch einen Map-Decoder verarbeitet. Durch hierarchische bipartite Matching wird die Prädiktion mit den Ground-Truth Daten verglichen, um einen Loss zu berechnen [27].

Der Map-Encoder dieser Architektur verarbeitet sechs Kamerabilder aus der Bodenperspektive, die zunächst durch ein Backbone-Netzwerk weiterverarbeitet werden. Als Backbone dient ein ResNet-50 [14], eine spezielle Form eines Convolutional Neural Networks (CNN), dessen Details in Kapitel 2.3 näher erläutert sind. Nach der initialen Verarbeitung erfolgt eine Transformation dieser Daten durch eine 2D-zu-BEV-Transformation, um BEV-Features zu extrahieren. Für die 2D-zu-BEV-Transformation wird primär der Geometry-Guided Kernel Transformer (GKT) [6] verwendet (wie in Kapitel 3.3 dargelegt). Zusätzlich bietet MapTR Unterstützung für alternative Frameworks für diese spezifische Transformation, zu denen der Cross-view Transformer (CVT)

[48], Lift Splat Shoot (LSS) [37], Deformable Attention [26] und Inverse Perspective Mapping (IPM) [34] zählen.

Die erhaltenen BEV-Features werden später in der modifizierten Netzwerkarchitektur, die als Grundlage dieser Arbeit dient, mit den Aerial-Features fusioniert (wie in Kapitel 4.1 dargelegt).

Der Map-Decoder (wie in Abb. 3.1 dargestellt) verwendet ein hierarchisches Query-Embedding-Schema, um jedes Kartenelement explizit zu kodieren. Es wird eine Menge von Instance-Level Queries $\{q_i^{ins}\}_{i=0}^{N-1}$ und eine Menge von Point-Level Queries $\{q_j^{pt}\}_{j=0}^{N_v-1}$ definiert, die von allen Instanzen geteilt werden. Jedes Kartenelement (mit Index i) entspricht einem Set von hierarchischen Queries $\{q_{ij}^{hie}\}_{j=0}^{N_v-1}$. Die hierarchische Query des j-ten Punktes des i-ten Kartenelements wird formuliert als:

$$q_{ij}^{hie} = q_i^{ins} + q_j^{pt} \qquad . \tag{3.1}$$

Die hierarchischen Queries ermöglichen eine differenzierte Kodierung von Kartenelementen. Beispielsweise wird ein Fußgängerüberweg betrachtet: Die Instance-Level Query $\{q_i^{ins}\}$ spezifiziert die Art des Kartenelements, in diesem Fall einen Fußgängerüberweg, während die Point-Level Queries $\{q_j^{pt}\}$ die genauen Lokalisationen innerhalb der HD-Karte erfassen. Somit wird jeder Punkt j des Kartenelements i durch eine hierarchische Query dargestellt. Der Map-Decoder integriert mehrere kaskadierte Decoder-Layer, die die hierarchischen Queries iterativ aktualisieren. In diesen Layern wird Multi-Head Self-Attention angewendet, wie in Kapitel 2.7.3 erläutert, um einen umfassenden Informationsaustausch zwischen den Queries sowohl auf Inter-Instanz- als auch auf Intra-Instanz-Ebene zu ermöglichen. Folgend wird die Deformable Attention [50] eingesetzt, um eine dynamische Interaktion der hierarchischen Queries mit den BEV-Features zu ermöglichen, die von BEVFormer [26] inspiriert ist. Jede Query q_{ij}^{hie} prädiziert die 2-dimensionale normalisierte BEV-Koordinate (x_{ij}, y_{ij}) des Referenzpunktes p_{ij}. Jedes Kartenelement entspricht einem Satz von Referenzpunkten $\{p_{ij}\}_{j=0}^{N_v-1}$ mit flexibler und dynamischer Verteilung. Diese Referenzpunkte $\{p_{ij}\}_{j=0}^{N_v-1}$ können sich an die willkürliche Form des Kartenelements anpassen und informativen Kontext für das Lernen des Kartenelements erfassen. Anschließend werden BEV-Features um die Referenzpunkte herum gesampelt und die Queries aktualisiert.

3.2.2 Permutationsäquivalente Modellierung

MapTR zielt darauf ab, die HD-Karte in einer einheitlichen Weise zu modellieren und zu lernen. Für die permutationsäquivalente Modellierung abstrahiert MapTR Kartenelemente geometrisch als geschlossene Formen (wie z.B. Fußgängerüberwege) und als offene Formen (wie z.B. Fahrspurmarkierungen), wie in Abb. 3.2 dargestellt.

Durch sequenzielles Erfassen der Punkte entlang der Formgrenze wird das geschlossene Element in ein Polygon und das offene Element in ein Polygonzug überführt. Obwohl sowohl Polygone als auch Polygonzüge als geordnete Punktmengen $VF = [v_0, v_1, \ldots, v_{N_v-1}]$ dargestellt werden können, ist die Permutation der Punktmengen nicht explizit definiert und nicht einzigartig. Es existieren viele äquivalente Permutationen für Polygone und Polygonzüge. Beispielsweise ist bei einer Fahrspurmarkierung (Polygonzug) zwischen zwei entgegengesetzten Fahrspuren, wie links in Abb. 3.2 dargestellt, die Festlegung seiner Richtung schwierig. Beide Endpunkte des Fahr-

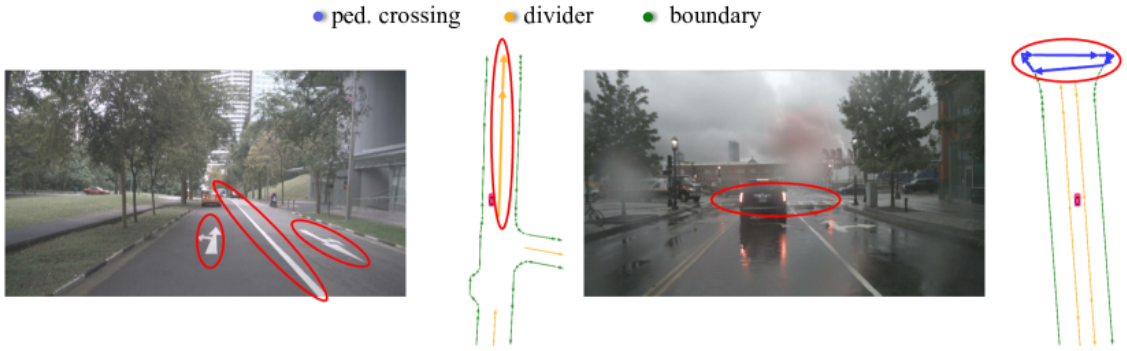


Abb. 3.2: Illustration der geometrischen Abstraktion von Kartenelementen durch MapTR. Links wird eine Fahrspurmarkierungen (lane divider, gelb) als offene Form (rot umkreist) dargestellt und Rechts ist ein Fußgängerüberwege (pedestrian crossing) als geschlossene Form (auch rot umkreist) dargestellt. Diese Visualisierungen verdeutlichen, wie MapTR unterschiedliche Kartenelemente in Form von Polygonen für geschlossene und Polylinien für offene Strukturen modelliert [27].

spurtrenners können als Startpunkt betrachtet werden und das Punkteset kann in zwei Richtungen abstrahiert werden. Rechts in Abb. 3.2 kann für den Fußgängerüberweg (Polygon) das Punkteset in zwei entgegengesetzte Richtungen (im Uhrzeigersinn und gegen den Uhrzeigersinn) abstrahiert werden. Das zirkuläre Ändern der Permutation des Punktesets hat keinen Einfluss auf die geometrische Form des Polygons. Um dieses Problem zu adressieren, modelliert MapTR jedes Kartenelement mit $V = (V, \Gamma)$. $V = \{v_j\}_{j=0}^{N_v-1}$ bezeichnet den Punktsatz des Kartenelements (N_v ist die Anzahl der Punkte). $\Gamma = \{\gamma_k\}$ bezeichnet eine Gruppe äquivalenter Permutationen des Punktsatzes V, die alle möglichen Organisationssequenzen abdeckt.

3.2.3 Hierarchisches Matching

MapTR implementiert die Inferenz einer festgelegten Anzahl N von Kartenelementen in einem Durchgang, folgend dem End-to-End-Paradigma von DETR [4]. Dabei wird N bewusst größer als die typische Anzahl von Kartenelementen in einer Szene gewählt. Die Organisation der Kartenelemente wird wie folgt strukturiert:

- Prädizierte Kartenelemente: Die Menge der N prädizierten Kartenelemente wird durch $\hat{Y} = \{\hat{y}_i\}_{i=0}^{N-1}$ dargestellt.
- Ground-Truth Kartenelemente: Parallel dazu wird die Menge der Ground-Truth (GT) Kartenelemente mit $\emptyset$ (kein Objekt) aufgefüllt, um ebenfalls N Elemente zu umfassen, bezeichnet durch $Y = \{y_i\}_{i=0}^{N-1}$.

Jedes Element y_i aus der GT-Menge wird spezifiziert durch:

- c_i - das Zielklassenlabel,
- V_i - den Punktsatz,
- Γ_i - die Permutationsgruppe des GT Kartenelements.

Entsprechend wird jedes prädizierte Element $\hat{y}_i$ charakterisiert durch:

- $\hat{p}_i$ - die vorhergesagte Klassifikationsscore,
- $\hat{V}_i$ - der vorhergesagte Punktsatz.

MapTR führt hierarchisches bipartites Matching ein, d.h., es wird zunächst ein instanzbasiertes Matching und anschließend ein punktbasiertes Matching durchgeführt. Instanzbasiertes Matching beinhaltet das Finden einer optimalen Zuordnung der prädizierten Kartenelemente zu den entsprechenden GT-Elementen. Nach erfolgreicher Zuordnung der Kartenelemente wird jedes prädizierte Kartenelement einem GT-Element zugewiesen. Anschließend wird ein punktbasiertes Matching durchgeführt, das darauf abzielt, die spezifischen Punkte innerhalb der prädizierten und GT-Kartenelemente korrekt zuzuordnen. Hierbei wird die Permutation berücksichtigt, die in der permutationsäquivalenten Modellierung 3.2.2 beschrieben ist.

Instanzbasiertes Matching

Zuerst muss eine optimale Zuweisung auf Instanzebene $\hat{\pi}$ zwischen den prädizierten Kartenelementen $\{\hat{y}_i\}$ und den GT Kartenelementen $\{y_i\}$ gefunden werden. $\hat{\pi}$ ist eine Permutation von N Elementen ($\hat{\pi} \in \Pi_N$) mit den niedrigsten instanzbasierten Matching-Kosten:

$$\hat{\pi} = \arg\min_{\pi \in \Pi_N} \sum_{i=0}^{N-1} L_{\text{ins match}}(\hat{y}_{\pi(i)}, y_i) \quad . \tag{3.2}$$

$L_{\text{ins match}}(\hat{y}_{\pi(i)}, y_i)$ sind die paarweisen Matching-Kosten zwischen der Prädiktion $\hat{y}_{\pi(i)}$ und GT y_i, die sowohl das Klassenlabel des Kartenelements als auch die Position des Punktsatzes berücksichtigen:

$$L_{\text{ins match}}(\hat{y}_{\pi(i)}, y_i) = L_{\text{Focal}}(\hat{p}_{\pi(i)}, c_i) + L_{\text{position}}(\hat{V}_{\pi(i)}, V_i) \quad . \tag{3.3}$$

$L_{\text{Focal}}(\hat{p}_{\pi(i)}, c_i)$ ist der Klassen-Matching-Kostenanteil, definiert als der Focal Loss [28] zwischen der prädizierten Klassifikationsscore $\hat{p}_{\pi(i)}$ und dem Zielklassenlabel c_i. $L_{\text{position}}(\hat{V}_{\pi(i)}, V_i)$ ist der Position-Matching-Kostenanteil, der die Positionskorrelation zwischen dem prädizierten Punktsatz $\hat{V}_{\pi(i)}$ und dem GT Punktsatz V_i widerspiegelt. Der Algorithmus wird verwendet, um die optimale instanzbasierte Zuweisung $\hat{\pi}$ gemäß DETR zu finden.

Punktbasiertes Matching

Nach dem instanzbasierten Matching wird jedem prädizierten Kartenelement $\hat{y}_{\hat{\pi}(i)}$ ein GT Kartenelement y_i zugewiesen. Dann wird für jede vorhergesagte Instanz, der positive Labels $c_i \neq \emptyset$ zugeordnet wurden, ein punktbasiertes Matching durchgeführt, um eine optimale Punkt-zu-Punkt-Zuweisung $\hat{\gamma} \in \Gamma$ zwischen dem prädizierten Punktsatz $\hat{V}_{\hat{\pi}(i)}$ und dem GT Punktsatz V_i zu finden. $\hat{\gamma}$ wird aus der vordefinierten Permutationsgruppe Γ ausgewählt und hat die niedrigsten punktbasierten Matching-Kosten:

$$\hat{\gamma} = \arg\min_{\gamma \in \Gamma} \sum_{j=0}^{N_v-1} D_{\text{Manhattan}}(\hat{v}_j, v_{\gamma(j)}) \quad . \tag{3.4}$$

$D_{\text{Manhattan}}(\hat{v}_j, v_{\gamma(j)})$ ist der Manhattan-Abstand zwischen dem j-ten Punkt des vorhergesagten Punktsatzes $\hat{V}$ und dem $\gamma(j)$-ten Punkt des GT Punktsatzes V.

3.2.4 Design der Lossfunktion im MapTR Training

MapTR wird auf der Basis der optimalen Zuweisung sowohl auf Instanz- als auch auf Punktebene ($\hat{\pi}$ und $\{\hat{\gamma}_i\}$) trainiert. Die Lossfunktion, wie in Kapitel 2.1.3 beschrieben, setzt sich in MapTR aus drei Komponenten zusammen:

- Classification Loss (L_{cls}),
- Point-to-Point Loss (L_{p2p})
- und Edge Direction Loss (L_{dir}).

Die Lossfunktion wird wie folgt formuliert:

$$L = \lambda L_{\text{cls}} + \alpha L_{\text{p2p}} + \beta L_{\text{dir}} \quad , \tag{3.5}$$

wobei λ, α und β die Gewichtungsfaktoren für die jeweiligen Komponenten sind, die sich wie folgt berechnen:

- Classification Loss (L_{cls}): Der Classification Loss wird mit Hilfe des Focal Loss [28] berechnet und basiert auf der optimalen Instanzenzuordnung $\hat{\pi}$, wie in Gleichung 3.2 modelliert. Jedes prädizierte Kartenelement wird einem Klassenlabel zugewiesen und der Loss wird wie folgt formuliert:

$$L_{\text{cls}} = \sum_{i=0}^{N-1} L_{\text{Focal}}(\hat{p}_{\hat{\pi}(i)}, c_i) \quad , \tag{3.6}$$

- Point-to-Point Loss (L_{p2p}): Dieser Loss bewertet die Position jedes prädizierten Punktes, basierend auf dem optimalen Zuordnungsergebnis auf Punktebene $\hat{\gamma}_i$. Für jede GT-Instanz wird der Loss als Manhattan-Distanz zwischen jedem zugeordneten Punktpaar berechnet:

$$L_{\text{p2p}} = \sum_{i=0}^{N-1} \mathbf{1}_{\{c_i \neq \emptyset\}} \sum_{j=0}^{N_v-1} D_{\text{Manhattan}}(\hat{v}_{\hat{\pi}(i),j}, v_{i,\hat{\gamma}_i(j)}) \quad . \tag{3.7}$$

- Edge Direction Loss (L_{dir}): Der Edge Direction Loss ergänzt den Point-to-Point Loss, indem nicht nur die Punkte der Polygonzüge und des Polygons, sondern auch die Richtungen der Kanten berücksichtigt werden. Der Loss wird als negativer Cosinus der prädizierten und tatsächlichen Kantenrichtungen definiert:

$$L_{\text{dir}} = -\sum_{i=0}^{N-1} \mathbf{1}_{\{c_i \neq \emptyset\}} \sum_{j=0}^{N_v-1} \text{cosine similarity}(\hat{e}_{\hat{\pi}(i),j}, e_{i,\hat{\gamma}_i(j)}) \quad . \tag{3.8}$$

Hierbei sind:

- N: Anzahl der prädizierten Kartenelemente.
- L_{Focal}: Focal Loss-Funktion, die falsch klassifizierte Beispiele stärker gewichtet.
- $\hat{p}_{\hat{\pi}(i)}$: Classification score der optimal zugeordneten prädizierten Instanz $\hat{\pi}(i)$.
- c_i: Zielklassenlabel des i-ten Elements.
- $D_{\text{Manhattan}}(x,y)$: Manhattan-Distanz zwischen den Koordinaten x und y, berechnet als $|x_1 - x_2| + |y_1 - y_2|$.
- $\mathbf{1}_{\{c_i \neq \emptyset\}}$: Indikatorfunktion, die 1 ist, wenn die Instanz c_i existiert, sonst 0.
- N_v: Anzahl der Punkte innerhalb einer Instanz.
- $\hat{v}_{\hat{\pi}(i),j}$: Prädizierte Koordinaten des j-ten Punktes der i-ten Instanz gemäß der Zuordnung $\hat{\pi}$.
- $v_{i,\hat{\gamma}_i(j)}$: Tatsächliche Koordinaten des j-ten Punktes der i-ten Instanz gemäß der Punktzuordnung $\hat{\gamma}_i$.
- $\hat{\gamma}_i$: Optimale Zuordnung der Punkte für die i-te Instanz.
- $\hat{e}_{\hat{\pi}(i),j}$: Prädizierte Kante zwischen benachbarten Punkten innerhalb der i-ten Instanz.
- $e_{i,\hat{\gamma}_i(j)}$: Tatsächliche Kante zwischen benachbarten Punkten innerhalb der i-ten Instanz.
- cosine similarity(a,b): Maß für die Ähnlichkeit zwischen zwei Vektoren a und b, berechnet als $\frac{a \cdot b}{\|a\|\|b\|}$.

3.3 2D-zu-BEV-Transformationsmethoden

Geometry-Guided Kernel Transformers (GKT)

Die Transformation von 2D-Features, extrahiert durch ein Backbone-Netzwerk wie ResNet-50, in BEV-Features, ist eine wesentliche Komponente innerhalb der MapTR-Architektur, wie in Kapitel 3.2 eingeführt und in Abbildung 3.1 visualisiert. Eine zentrale Methode für diese Aufgabe ist der Geometry-Guided Kernel Transformer (GKT) [6], der sich ausschließlich auf diskriminative Regionen fokussiert und diese für die BEV-Feature-Generierung betrachtet. Diese Methode ist in Abb. 3.3 dargestellt.

GKT projiziert jeden BEV-Gitterpunkt auf eine Menge von 2D-Pixelkoordinaten, die durch die intrinsischen und extrinsischen Kameraparameter bestimmt werden. Anschließend verwendet der GKT eine Kernel-Entfaltung. Die Kernel-Entfaltung ist ein Verfahren, bei dem umliegende Pixelbereiche basierend auf vordefinierten Kerneldimensionen aggregiert werden. Auf Basis dessen werden dann BEV-Features extrahiert.

GKT implementiert eine Look-Up-Tabelle (LUT), um die Notwendigkeit der Verarbeitung von Kamerakalibrierungsparametern während der Laufzeit zu eliminieren. Die Kernel-Regionen für

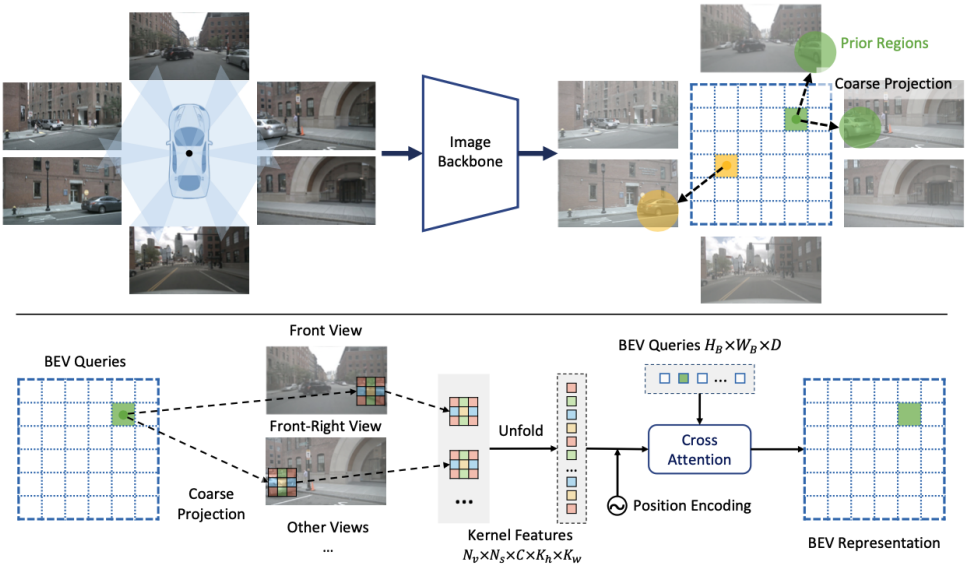


Abb. 3.3: Darstellung des Geometry-Guided Kernel Transformers (GKT). Die Abbildung illustriert, wie ein CNN-Backbone aus Kamerabildern BEV-Features extrahiert. GKT fokussiert sich auf diskriminative Regionen und extrahiert dann die BEV-Features [6].

jedes BEV-Gitter werden fest definiert und können vorab offline berechnet werden. Vor der Ausführung wird eine LUT konfiguriert, die die Zuordnungen zwischen den Indizes der BEV-Queries und den korrespondierenden Pixelindizes speichert. Während der Laufzeit ermöglicht die LUT einen schnellen Zugriff auf die relevanten Pixelindizes für jede BEV-Query, indem die entsprechenden Einträge aus der Tabelle abgerufen werden.

Das GKT Framework wird in Abb. 3.3 dargestellt. Ein CNN-Backbone extrahiert Kamerafeatures aus den Bildern aus Bodenperspektive $I = \{I_v\}$. Diese Kamerafeatures werden durch den Transformer in den BEV-Feature-Raum transformiert, der gleichmäßig in Gitter unterteilt ist. Jedes BEV-Gitter entspricht einem 3D-Koordinatenpunkt $P_i = (x_i, y_i, z)$ und einem lernbaren Query-Embedding q_i. Um den Fokus des Transformers auf den diskriminativen Regionen zu gewährleisten, werden geometrische Vorpriorisierungen genutzt. Mit den Parametern der Kamera wird jedes BEV-Gitter P_i grob in 2D-Pixelkoordinaten $\{Q_i^{sv}\}$ projiziert und dann auf ganze Koordinaten $\{\overline{Q}_i^{sv}\}$ gerundet:

$$Q_i^{sv} = K^{sv} \cdot Rt^{sv} \cdot P_i^{sv} \quad , \tag{3.9}$$

$$\overline{Q}_i^{sv} = \text{round}(Q_i^{sv}) \quad , \tag{3.10}$$

wobei s die Bildskala, v die Ansicht, K^{sv} die intrinsische Kameramatrix und Rt^{sv} die extrinsische Rotationsmatrix beschreibt, die die Orientierung der Kamera angibt.

Um diese vor priorisierten Positionen werden Kernel-Regionen der Größe $K_h \times K_w$ entfaltet. Jedes BEV-Query q_i interagiert mit den entsprechenden entfalteten Kernel-Features $F \in \mathbb{R}^{N_{\text{view}} \times N_{\text{scale}} \times C \times K_h \times K_w}$ und generiert so die BEV-Features.

BEVFormer

Innerhalb der MapTR-Architektur, die primär auf GKT zur Transformation von 2D-Features in BEV-Features setzt, wird ebenfalls BEVFormer [26] unterstützt. BEVFormer [26] transformiert Kamerabilder aus Bodenperspektive in BEV-Features, genau wie GKT.

BEVFormer , in Abb. 3.4 dargestellt, kombiniert räumliche sowie zeitliche Informationen. Eine zentrale Komponente dieses Ansatzes sind die BEV Queries, die lernbare Parameter sind. Diese Queries ermöglichen es, mithilfe von Spatial Cross-Attention, dass nur relevante räumliche Bereiche abgefragt werden, die auch von Interesse sind. Im Gegensatz zu GKT, der sich nur auf räumliche Features aus einzelnen Bildern konzentriert, berücksichtigt BEVFormer zusätzlich zeitliche Features. BEVFormer erreicht dies durch die Integration von Temporal Self-Attention. Dies ermöglicht eine dynamischere und kontextbezogene Interpretation der Umgebung, was insbesondere für Anwendungen wie 3D-Objekterkennung und Kartensegmentierung von Vorteil ist.

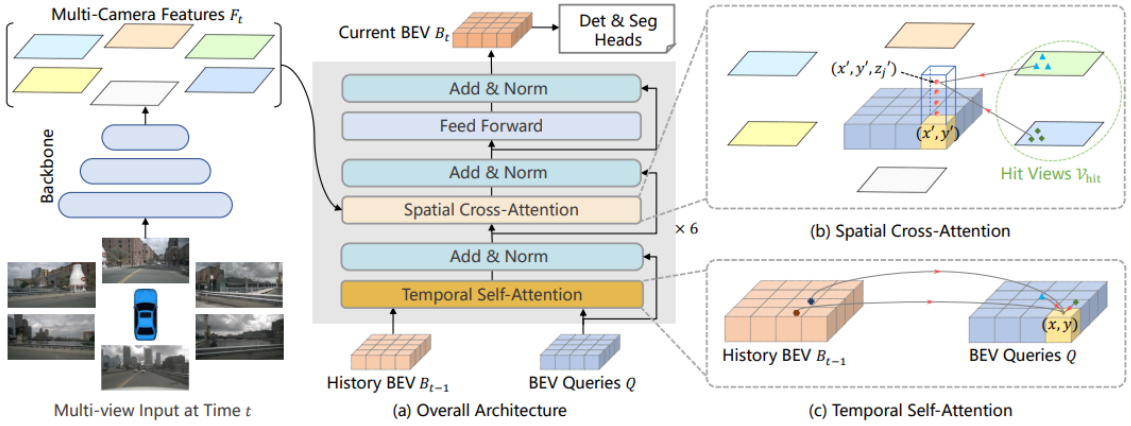


Abb. 3.4: Darstellung der BEVFormer-Architektur. Panel (a) zeigt die Gesamtarchitektur eines Encoder-Layers, welches Features aus Kamerabildern aus Bodenperspektive F_t und historischen BEV-Features B_{t-1} verarbeiten. Die BEV Queries interagieren über Spatial Cross-Attention und Temporal Self-Attention mit den Features F_t, um BEV-Features B_t für die aktuelle Zeiteinheit zu generieren. Panel (b) illustriert die Funktionsweise der Spatial Cross-Attention, die relevante räumliche Informationen aus verschiedenen Kameraperspektiven aggregiert, während Panel (c) die Integration zeitlicher Features durch Temporal Self-Attention verdeutlicht[26].

Die BEVFormer-Architektur besteht aus sechs Encoder-Layern, die an die traditionellen Strukturen von Transformern angelehnt sind. Im Inferenzprozess werden zu jedem Zeitpunkt t Kamerabilder aus Bodenperspektive durch ein Backbone-Netzwerk, etwa ResNet-101 [14], verarbeitet. Diese Methode ermöglicht die Extraktion von Features $F_t = \{F_t^i\}_{i=1}^{N_{\text{view}}}$ aus den verschiedenen Kameraperspektiven, wobei F_t^i die Features aus der i-ten Kameraansicht und N_{view} alle Kameraansichten darstellen. Dies ist links Abb. 3.4 dargestellt.

Parallel dazu werden die BEV-Features B_{t-1} des vorherigen Zeitpunkts gespeichert. In jeder Encoder-Layer erfolgt zunächst der Einsatz von BEV Queries, um mittels Temporal Self-Attention zeitliche Informationen aus den BEV-Features B_{t-1} des vorangegangenen Zeitpunkts zu erfassen. Anschließend werden die BEV Queries zur Anwendung von Spatial Cross-Attention genutzt, die räumliche Informationen aus den Kamerabildern F_t extrahieren. Nach der Bearbeitung durch ein

Feed-Forward-Netzwerk gibt jede Encoder-Layer verfeinerte BEV-Features aus, die als Eingang für die nächste Layer dienen.

Nach dem Durchlaufen der sechs aufeinanderfolgenden Encoder-Layer werden endgültig einheitliche BEV-Features B_t für den aktuellen Zeitpunkt t generiert. Diese dienen dann als Grundlage für weitere Verarbeitungsschritte, wie die 3D-Objekterkennung und Kartensegmentierung.

3.4 Weitere Ansätze zur Erstellung von HD-Karten

Hochaufgelöste digitale Karten (HD-Karten) lassen sich hinsichtlich ihrer Erstellungsweise in offline und online generierte Karten gliedern. Weiterhin können sie in rasterisierte und vektorisierte HD-Karten unterteilt werden. Rasterisierte HD-Karten basieren auf dem Prinzip der semantischen Segmentierung in einer Vogelperspektive (Bird's-Eye View, BEV) und gelten oft als grundlegender Ansatz für die HD-Kartenerstellung. Obwohl dieser Ansatz direkt und relativ einfach ist, mangelt es ihm an der Fähigkeit, instanzspezifische und strukturelle Informationen effektiv zu erfassen. Zudem ist es schwierig, räumliche Konsistenz in den prädizierten rasterisierten Karten zu gewährleisten, was umfangreiche Nachbearbeitungen notwendig macht, um die Karten für nachfolgende Navigations- und Entscheidungsprozesse nutzbar zu machen. Im Gegensatz dazu stehen die vektorisierten HD-Karten, die eine präzisere und informationsreichere Darstellung bieten. Diese Methode konzentriert sich darauf, Elemente der Karte direkt als vektorisierte Punktmengen darzustellen, was eine genauere und schnellere Verarbeitung ermöglicht. Fortschritte in dieser Technik, wie beispielsweise durch das VectorMapNet [31] und MapTR [27], haben gezeigt, dass direkte Vektorisierung zu besseren Genauigkeiten und schnelleren Laufzeiten führen kann. Daneben gibt es auch Ansätze, die beides kombinieren, wie MapVR [46]. Im Folgenden soll auf VectorMapNet [31] und MachMap [38] weiter eingegangen werden.

VectorMapNet ist eine end-to-end Lernpipeline, die von Sensoren erfasste Beobachtungen nutzt, um spärliche Polylines in der Vogelperspektive zu prädizieren (wie in Abb. 3.5 dargestellt). Dieser Ansatz modelliert explizit die räumlichen Beziehungen zwischen Kartenelementen und erzeugt vektorisierte Karten, die für nachgelagerte Aufgaben des autonomen Fahrens geeignet sind. VectorMapNet aggregiert zuerst Features aus verschiedenen Modalitäten (z.B. Kamerabilder und LIDAR) in einem gemeinsamen BEV-Feature-Raum. Anschließend werden Kartenelemente mittels lernbarer Queries und BEV-Features lokalisiert. Schließlich wird jede Query in eine Polylinie dekodiert. Die Aggregation der Sensordaten in einem BEV-Feature-Raum beginnt mit der Extraktion von BEV-Features aus den RGB-Kameradaten unter Verwendung eines Convolutional Neural Networks (CNN), wie zum Beispiel ResNet. Punktwolken aus LIDAR-Sensoren werden mithilfe von PointPillars [20] kodiert. Anschließend werden die extrahierten Features beider Sensormodalitäten in eine gemeinsame BEV-Darstellung transformiert. Dadurch wird ein einheitlicher Feature-Raum geschaffen, der die Grundlage für die weitere Verarbeitung bildet. VectorMapNet nutzt lernbare Queries, die ähnlich den Object Queries im DETR-Modell [50] gezielt nach bestimmten Kartenelementen wie Straßengrenzen, Fahrspurmarkierungen oder Fußgängerüberwegen sucht. Diese Queries interagieren mit den BEV-Features, um die spezifische Position und Formen der Kartenelemente zu identifizieren. Das Modell nutzt dafür eine Kombination aus Attention-Mechanismen,

die es ermöglichen, relevante Bereiche innerhalb der BEV-Features gezielt zu lokalisieren und die relevanten Features herauszufiltern. Nachdem die relevanten Kartenelemente lokalisiert sind, werden die zugehörigen Queries in Polylines dekodiert. Eine Polyline ist eine Folge von verbundenen Punkten, die ein Kartenelement wie eine Straßengrenze oder eine Fahrspurmarkierung beschreiben. Die resultierenden Polylines werden für nachgelagerte autonome Fahrsysteme verwendet, z.B. für die Routenplanung, Hinderniserkennung oder Bewegungsprädiktion.

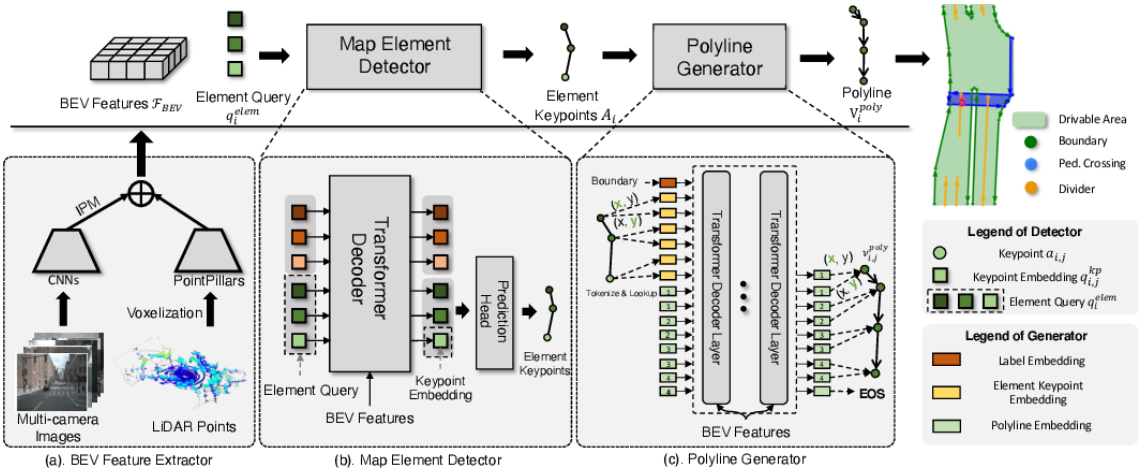


Abb. 3.5: Die Netzwerkarchitektur von VectorMapNet. Das obere Panel zeigt den Ablauf von VectorMapNet bei der Erzeugung von Polylines aus Sensordatenn. Das untere Panel veranschaulicht detaillierte Strukturen und Inferenzverfahren der drei Hauptkomponenten von VectorMapNet: BEV-Feature-Extraktor, Kartenelement-Detektor und Polylinien-Generator [31].

MachMap [38] ist eine end-to-end Lernpipeline zur vektorisierten Erstellung kompakter HD-Karten für autonomes Fahren. Das Modell formuliert die HD-Kartenkonstruktion als Punktdetektionsproblem im Bird-Eye-View (BEV) Raum und nutzt dabei eine Architektur, die eine CNN-Backbone wie InternImage [44], einen zeitbasierten Instanz-Decoder und ein Point-Mask-Kopplungsmodul nutzt. MachMap reduziert die Anzahl der vektorisierten Punkte drastisch, ohne die Darstellungsleistung zu beeinträchtigen. Dies wird durch zwei Schritte erreicht. Durch die Neuordnung aller Elemente (z.B. Fahrspurmarkierungen), welche neu ausgerichtet werden, wird eine konsistente Orientierung geschaffen. Außerdem werden überflüssige Punkte entfernt, wodurch eine sparsamere und effizientere Darstellung der Karteninformationen erreicht wird. Verwendet werden dafür Algorithmen wie der Douglas-Peucker-Algorithmus [43], die eine Polylinie mit weniger Punkten, aber ähnlicher Form erzeugen.

Die Architektur von MachMap, analog zu VactorMap, gliedert sich in drei Hauptkomponenten, wie in Abbildung 3.6 visualisiert. Die erste zentrale Komponente bildet der BEV Feature Extractor. Dieser extrahiert Features und wendet dafür eine deformierbare Convolution [9] an. Während des Downsampling-Prozesses werden eine Reihe von Feature-Maps generiert und anschließend durch ein bidirektionales Feature Pyramid Network (BiFPN) [49] aggregiert. Diese aggregierten Features werden auf Basis der Position vorheriger Features transformiert, entsprechend der Fusionsstrategie in VideoBEV [13]. Dadurch kann die aktuelle Position der Features besser bestimmt werden. Im Instance Decoder des MaskDINO-Frameworks [24] werden Objekterkennung und Segmentierung

gleichzeitig durchgeführt, wobei Inhalts- und Positionsvektoren für die Generierung von Instanzmasken und normierten Koordinaten genutzt werden. Das System verwendet das Query-Design von MapTR für eine präzisere Kartenelementmodellierung. Die Lossfunktion bildet sich durch eine Kombination aus Kreuzentropie, Dice-Loss [35] und L1-Loss.

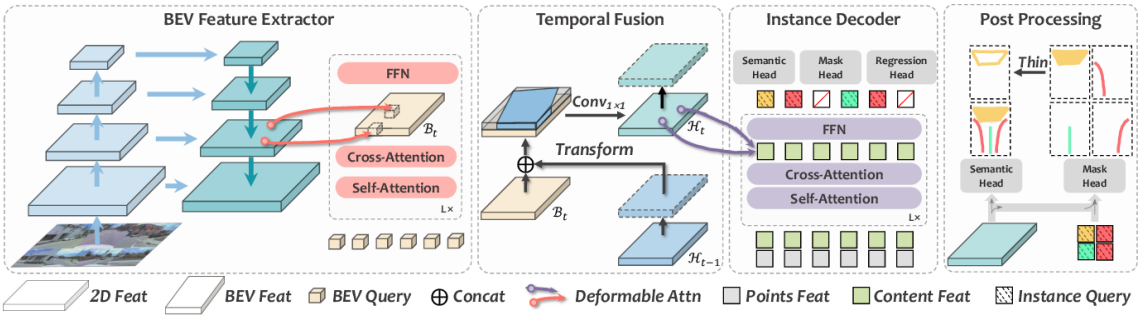


Abb. 3.6: Übersicht über die Architektur von MachMap, die sich in drei Hauptkomponenten gliedert: BEV Feature Extractor, Temporal Fusion und Instance Decoder, ergänzt durch einen Post-Processing-Schritt. Der BEV Feature Extractor nutzt deformierbare Convolutions, um 2D-Features zu BEV-Features zu transformieren, die durch ein BiFPN aggregiert werden. Die Temporal Fusion verwendet transformierte historische BEV-Features H_{t-1} und aktuelle BEV-Features B_t, um über eine $Conv1x1$ Layer aktualisierte Features H_t zu generieren. Der Instance Decoder verarbeitet diese Features zur Objekterkennung und Segmentierung mittels des MaskDINO-Frameworks, wobei ein spezielles Query-Design für präzise Kartenelementmodellierung verwendet wird. Der Post-Processing-Schritt beinhaltet das Verfeinern von Segmentierungs- und Maskierungsergebnissen, gesteuert durch semantische und Maskenkopf-Ausgaben [38].

4 Konzept

In diesem Kapitel wird die verwendete Netzwerkarchitektur beschrieben. Zudem werden verschiedene Strategien zur Fusion der BEV- und Aerial-Features vorgestellt, die eine optimale Nutzung beider Datenquellen ermöglichen.

4.1 Netzwerkarchitektur

Die verwendete Architektur basiert auf dem transformerbasierten Netzwerk MapTR [27] (wie in Kapitel 3.2 dargelegt), das sechs Kamerabilder aus Bodenperspektive eines Fahrzeugs nutzt, um online eine HD-Karte zu generieren. Diese Architektur wird um ein Fusionsmodul erweitert, das Luftbilder (wie in Kapitel 5.6.2 dargestellt) als zusätzliche Datenquelle neben den existierenden Kamerabildern integriert. Es existieren zwei parallele Verarbeitungspipelines, wie in Abb. 4.1 dargestellt.

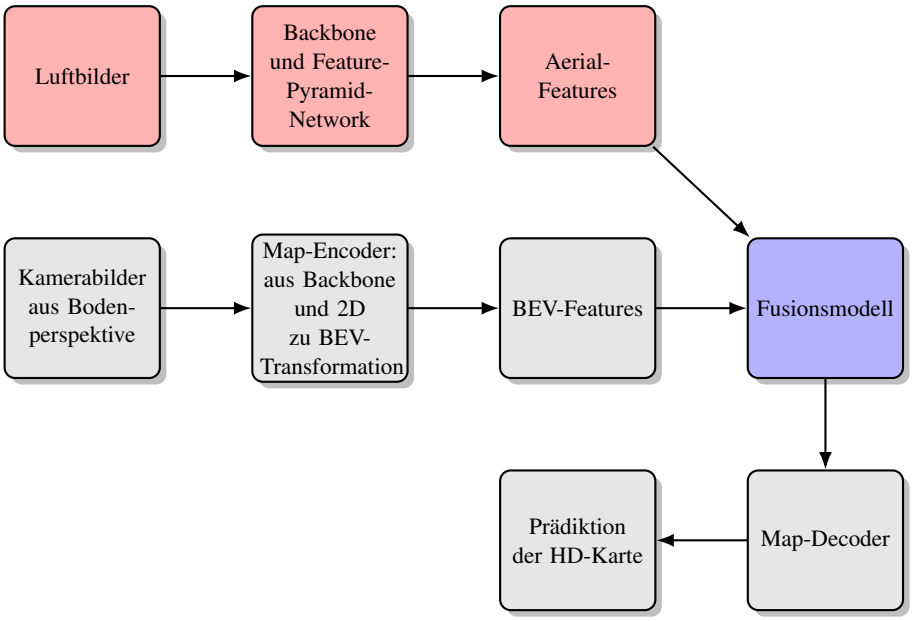


Abb. 4.1: Verwendete Netzwerkarchitektur. Luftbilder werden durch ein Backbone und FPN verarbeitet um Aerial-Features zu generieren. Kamerabilder aus Bodenperspektive werden durch ein Backbone und einem 2D-zu-BEV-Transformer transformiert um BEV-Features zu erhalten. Diese Features werden im Fusionsmodell fusioniert und in einem Map-Decoder decodiert, um dadurch eine HD-Karte zu prädizieren.

Einerseits werden die Luftbilder mittels eines Backbones (ResNet-50 [14]) verarbeitet. Die hieraus extrahierten Features werden anschließend durch ein Feature Pyramid Network (FPN) [29] weiterverarbeitet. Diese Aerial-Features werden nachfolgend in die MapTR-Architektur eingegliedert.

Der Map-Encoder verarbeitet parallel die Bilder aus den Bodenperspektiven, indem er zunächst die Features extrahiert und in eine einheitliche Feature-Darstellung überführt. Dies erfolgt mittels eines ResNet-50 [14]. Die daraus resultierenden Features werden in BEV-Features transformiert, wobei das Geometry-Guided Kernel Transformer (GKT) Modul [6] zum Einsatz kommt. Dies ist ausführlich in Kapitel 3.3 dargelegt. Die umgewandelten BEV-Features werden mit den aus den Luftbildern extrahierten Aerial-Features fusioniert. Details zu den verschiedenen Fusionsmethoden werden in Kapitel 4.2 erläutert.

Die fusionierten Features werden anschließend im Map Decoder weiterverarbeitet. Eine ausführliche Beschreibung der Pipeline findet sich in Kapitel 3.2.1.

4.2 Fusionsstrategien

Es wurden verschiedene Fusionsstrategien untersucht, die sich in statische und dynamische Strategien unterteilen lassen. Ziel ist die Fusion von zwei vierdimensionalen Tensoren, die aus den Dimensionen Mini-Batch-Größe, Channel, Width und Height bestehen. Diese Tensoren repräsentieren unterschiedliche Datenquellen. Der erste Tensor enthält die BEV-Features (Bird's Eye View), die aus Kamerabildern in Bodenperspektive extrahiert werden. Der zweite Tensor beinhaltet die Aerial-Features, welche aus Luftbildern gewonnen werden.

4.2.1 Statische Fusionsstrategien

Bei statischen Fusionsstrategien wird die Fusion der Eingangstensoren während der Validierungsphase durch eine im Training festgelegte Gewichtung realisiert. Dieser gewichtete Kombination durch den Gewichtstensor ist wie folgt definiert:

$$F_{\text{fused}} = w \cdot F_{\text{bev}} + (1 - w) \cdot F_{\text{aerial}} \quad , \tag{4.1}$$

wobei $F_{\text{fused}}, F_{\text{bev}}, F_{\text{aerial}} \in \mathbb{R}^{B \times C \times W \times H}$, F_{fused} die fusionierten Features, F_{bev} die BEV-Features, F_{aerial} die Aerial-Features, B die Mini-Batch-Größe, C die Anzahl der Kanäle, W die Width und H die Height der Eingangstensoren darstellt. Der Gewichtstensor w bestimmt die relative Gewichtung der beiden Tensoren auf die Fusion.

Die statische Fusionsstrategie zeichnet sich durch ihre geringe Komplexität aus. Während der Trainingsphase wird ein Gewichtstensor trainiert, der anschließend während der Testphase unverändert angewendet wird. Die Gewichtung bleibt somit während der Inferenz über jeglichen Input hinweg konstant.

4.2.2 Dynamische Fusionsstrategien

Im Gegensatz dazu wird bei der dynamischen Fusionsstrategie die Gewichtung der Eingangstensoren abhängig von deren Inhalten dynamisch angepasst. Die Fusion erfolgt als gewichtete Kombination der beiden Tensoren:

$$F_{\text{fused}} = w \cdot F_{\text{bev}} + \big(1 - w\big) \cdot F_{\text{aerial}} \quad , \tag{4.2}$$

$$w(F_{\text{bev}}) = \mathcal{N}(F_{\text{bev}}) \qquad , \tag{4.3}$$

wobei $F_{\text{fused}}, F_{\text{bev}}, F_{\text{aerial}} \in \mathbb{R}^{B \times C \times W \times H}$, F_{fused} die fusionierten Features, F_{bev} die BEV-Features, F_{aerial} die Aerial-Features, B die Mini-Batch-Größe, C die Anzahl der Channels, W die Width, H die Height der Eingangstensoren und w die Gewichtung darstellt, welche von einem neuronalen Netzwerk erzeugt wird.

Das neuronale Netz dient der dynamischen Steuerung der Beiträge der BEV- und Aerial-Features zur Fusion. Dabei wird die Gewichtung dynamisch auf Grundlage der aktuellen Features in den Eingangstensoren berechnet. Diese dynamische Gewichtung ermöglicht eine flexiblere Fusion der unterschiedlichen Informationsquellen, je nachdem, welche Features in den Daten vorhanden sind. Beispielsweise könnten Sichtbereiche am Fahrzeug durch vorbeifahrende Fahrzeuge, schlechte Wetterbedingungen oder andere Hindernisse verdeckt sein. Die dynamische Strategie könnte in solchen Fällen die Gewichtung dynamisch anpassen, um die unverdeckten Features aus den Luftbildern zu priorisieren.

Ebenso könnten Luftbilder durch Überdachungen, Bäume oder andere große Strukturen teilweise verdeckt sein. Daher wäre die Fähigkeit, die Gewichtung dynamisch zu justieren und mehr Gewicht auf die unverdeckten Features der Kamerabilder zu legen, von großem Vorteil. Dies ermöglicht dem System, eine umfassendere und genauere Interpretation der verfügbaren Daten, um sich nahtlos an die gegebenen visuellen Einschränkungen anzupassen.

5 Umsetzung

Dieses Kapitel beschäftigt sich mit der Implementierung des in Kapitel 4 dargelegten theoretischen Konzepts. Hierbei wird speziell der verwendete Datensatz detailliert erläutert und die Auswahl sowie Einstellung der Hyperparameter begründet. Besonderes Augenmerk liegt auf dem Einsatz des Optimierungsalgorithmus AdamW [32] in Kombination mit dem Cosine Annealing Schedule zur Anpassung der Lernrate, welche eine zentrale Rolle in der Strategie des Netzwerktrainierens spielt. Eine detaillierte Erklärung zum Cosine Annealing Schedule findet sich in Kapitel 5.3.

5.1 Fusionsmodelle

In diesem Abschnitt werden verschiedene Ansätze zur Implementierung von Fusionsmodellen detailliert betrachtet.

Feature Scaling

Bei manchen Fusionsmodellen wurde Feature Scaling angewendet. Dabei wurden beide Eingangstensoren auf eine gemeinsame Verteilung gebracht. Dieses Feature-Scaling ist entscheidend, da es Unterschiede in den Verteilungen der Eingangstensoren ausgleicht. Dadurch kann das Netzwerk direkt die optimalen Fusionsgewichte lernen, ohne zusätzliche Anpassungen zur Angleichung der Features vornehmen zu müssen.

Die Verteilungen der Einträge in den Tensoren werden wie folgt definiert:

$$F_{\text{bev}} \sim N(\mu_{\text{bev}}, \sigma^2_{\text{bev}}) \quad , \tag{5.1}$$

$$F_{\text{aerial}} \sim N(\mu_{\text{aerial}}, \sigma^2_{\text{aerial}}) \quad . \tag{5.2}$$

Um den Tensor F_{aerial} an dieselbe Verteilung wie F_{bev} anzupassen, erfolgt folgendes Feature-Scaling:

$$F_{\text{aerial, scaled}} = \sigma_{\text{bev}} \cdot \left(\frac{x_{\text{aerial}} - \mu_{\text{aerial}}}{\sigma_{\text{aerial}}} \right) + \mu_{\text{bev}} \quad , \tag{5.3}$$

wobei jeder Eintrag x_{aerial} des Tensors F_{aerial} gemäß dem Mittelwert μ_{aerial} und der Standardabweichung σ_{aerial} normalisiert und daraufhin derart reskaliert wird, dass die Verteilung der Einträge in $F_{\text{aerial, scaled}}$ denselben Mittelwert und dieselbe Standardabweichung wie die Einträge in Tensor F_{bev} aufweist. Der Tensor F_{bev} hat den Mittelwert μ_{bev} und die Standardabweichung σ_{bev}.

5.1.1 Statische Fusionsmodelle

Bei den statischen Fusionsstrategien sind unterschiedliche Implementierungsvarianten zu unterscheiden. Grundsätzlich wird zwischen der Verwendung von pre-trained Modellen (vortrainierten) und non pre-trained Modellen (nicht vortrainierten) differenziert. Bei den pre-trained Ansätzen wurden die Modellgewichte aus einem bereits trainierten MapTR-Modell übernommen. Lediglich das Feature-Pyramid-Network (FPN) sowie das Fusionsmodell werden während des Trainings weiter angepasst. Der Vorteil dieses Ansatzes besteht darin, dass nicht alle Gewichte trainiert werden müssen. Dadurch kann sich das Training auf die spezifischen Anpassungen des Fusionsmodells und der FPN konzentrieren. (Erklärung der Netzwerkarchitektur findet sich in Kapitel 4.1 und zum Feature-Pyramid-Network in Kapitel 2.5)

Im Gegensatz dazu sind die non pre-trained Modelle ohne vorinitialisierte Modellparameter. Hier werden alle Modellparameter, einschließlich der Feature-Pyramide und des Fusionsmodells, während des Trainings gelernt.

Die statischen Fusionsmodelle folgen alle der Logik der Gleichung 4.1, zur besseren Übersichtlichkeit ist diese hier nochmals dargestellt:

$$F_{\text{fused}} = w \cdot F_{\text{bev}} + (1 - w) \cdot F_{\text{aerial}} \quad , \tag{5.4}$$

wobei $F_{\text{fused}}, F_{\text{bev}}, F_{\text{aerial}} \in \mathbb{R}^{B \times C \times W \times H}$, F_{fused} die fusionierten Features, F_{bev} die BEV-Features, F_{aerial} die Aerial-Features, B die Mini-Batch-Größe, C die Anzahl der Channel, W die Width und H die Height der Eingangstensoren darstellt. Der Gewichtstensor w bestimmt den relativen Einfluss der beiden Tensoren auf die Fusion.

Non pre-trained Ansätze

Im Rahmen der non pre-trained Ansätze wurden folgende Implementierungen vorgenommen:

- Fusion der Tensoren über die Channel (**GD: C non pre-trained 0.5**): Es wurden die Tensoren über die Channel-Achse fusioniert und der Gewichtstensor $w \in \mathbb{R}^C$ für jeden Eintrag mit dem Wert 0.5 initialisiert.

- Fusion der Tensoren über die Channel, Width und Height mit Xavier initialisiertem Gewichtstensor (**GD: CWH non pre-trained Xavier**): Bei diesem Ansatz wird die Gewichtung der beiden Eingabetensoren durch den Gewichtstensor $w \in \mathbb{R}^{C \times W \times H}$ gestert, der mittels Xavier-Uniform-Initialisierung (wie in Kapitel 5.7 dargelegt) initialisiert wird.

Pre-trained Ansätze

Bei den pre-trained Ansätzen wurden die Modellgewichte aus einem bereits trainierten MapTR-Modell übernommen. Lediglich das Feature-Pyramid-Network sowie das Fusionsmodell werden während des Trainings weitertrainiert.

Im Rahmen dieser statischen Fusionsstrategie wurden folgende Fusionsmodelle implementiert:

- Fusion der Tensoren über Channel, Width und Height mit Xavier-Initialisierung (**GD: CWH pre-trained Xavier**): Der Gewichtstensor weist die Dimension $w \in \mathbb{R}^{C \times W \times H}$ auf. Es wurde kein Feature-Scaling der Eingangstensoren vorgenommen.

- Fusion der Tensoren über Channel, Width und Height mit Xavier-Initialisierung und mit Feature-Scaling entlang der Mini-Batch, Width und Height-Achse (**GD: CWH pre-trained Xavier FS: BWH**): Der Gewichtstensor hat die Dimension $w \in \mathbb{R}^{C \times W \times H}$. Die Features des Aerial-Tensors wurden auf dieselbe Verteilung wie die des BEV-Tensors gebracht, beschränkt auf die Mini-Batch, Width, Height-Achse. Dieses Feature Scaling ist in Gleichung 5.3 dargelegt.

- Fusion der Tensoren über Channel, Width und Height mit Xavier-Initialisierung und Reskalierung entlang der Mini-Batch, Channel, Width, Height-Achse (**GD: CWH pre-trained Xavier FS: BCWH**): Der Gewichtstensor besitzt die Dimension $w \in \mathbb{R}^{C \times W \times H}$. Die Aerial-Features wurden auf dieselbe Verteilung wie die BEV-Features angepasst, dies erfolgt über die Mini-Batch, Channel, Width, Height-Achse. Dieses Feature Scaling ist in Gleichung 5.3 dargelegt.

5.1.2 Dynamische Fusionsmodelle

Im Bereich der dynamischen Fusionsmodelle wurden drei unterschiedliche Implementierungsvarianten implementiert. Zwei dieser Varianten basieren auf dem in Kapitel 4.2.2 dargelegten Konzept und folgen eng der in Gleichung 4.2 formulierten Fusionsmethode. Die beiden Ansätze, die Dynamic Summed Fusion und die Dynamic Convolutional Fusion, nutzen dabei ein pre-trained Modell, genau wie einige der statischen Fusionsmodelle. Im Gegensatz dazu verwendet die Concatenation Fusion ein non pre-trained Modell und wurde von Scratch trainiert.

Dynamic Summed Fusion pre-trained FS: BCWH

Die Dynamic Summed Fusion ist eine pre-trained dynamische Fusionsstrategie. Die Fusion addiert die gewichtete Kombination der beiden Tensoren durch eine einfache Addition. Beide Eingangstensoren haben durch Feature Scaling 5.3 die gleiche Verteilung. Das Feature Scaling wurde über Mini-Batch-Size, Channel, Width und Height-Achse vollzogen.

Die Fusion erfolgt durch die folgende Operation:

$$F_{\text{fused}} = w \cdot F_{\text{bev}} + \big(1 - w\big) \cdot F_{\text{aerial, scaled}} \quad , \tag{5.5}$$

Das gewichtsberechnende neuronale Netzwerk w bestimmt die Gewichtung basierend auf den BEV-Features wie folgt:

$$w(F_{\text{bev}}) = \sigma(\mathcal{N}(F_{\text{bev}})) \quad , \tag{5.6}$$

wobei $\mathcal{N}$ ein Convolutional Neural Network darstellt, das den Tensor F_{bev} als Eingabe verarbeitet. Es wird eine Convolutional Layer mit einem 3x3 Kernel, Stride von 1 und Padding von 1

verwendet. Anschließend wird $\mathscr{N}$ durch die Sigmoid-Funktion σ normalisiert, um einen Gewichtungswert im Bereich zwischen 0 und 1 zu erzeugen.

Concatenation Fusion non pre-trained

Als weitere dynamische Fusionsstrategie ist die Konkatenation der Tensoren mit anschließender Convolution zu betrachten. Dafür wurde ein non pre-trained Modell verwendet. Die Konkatenation der Tensoren F_{bev} und F_{aerial} entlang der Channel-Achse ermöglicht eine direkte Verknüpfung der Features aus beiden Datenquellen: Luftbildern und Kamerabildern aus Bodenperspektive.

$$F_{\text{cat}} = F_{\text{bev}} \circ F_{\text{aerial}} \quad , \tag{5.7}$$

wobei $\circ$ den Konkatenationsoperator entlang der Channel-Achse darstellt. Die Tensoren F_{bev} und F_{aerial} haben jeweils die Dimensionen $\mathbb{R}^{B\times C\times W\times H}$, wobei B die Mini-Batch-Größe, C die Anzahl der Channels, W die Width und H die Height der Eingangstensoren darstellen. Der resultierende Tensor F_{cat} hat somit die Dimensionen:

$$F_{\text{cat}} \in \mathbb{R}^{B\times(2C)\times W\times H} \quad . \tag{5.8}$$

Eine Convolutional Layer h_{conv} mit Kernel 3x3 , Padding von 1 und keinem hinzugefügten Bias-Term wird auf den Tensor F_{cat} angewendet, dies ist in Kapitel 2.3 dargelegt:

$$F_{\text{conv}} = h_{\text{conv}}(F_{\text{cat}}) \quad , \tag{5.9}$$

wobei Tensor $F_{\text{conv}} \in \mathbb{R}^{B\times N\times W\times H}$. Anschließend erfolgt eine Mini-Batch-Normalisierung h_{bn}, wie in Kapitel 2.5 dargestellt:

$$F_{\text{norm}} = h_{\text{bn}}(F_{\text{conv}}) \quad , \tag{5.10}$$

Der normalisierte Tensor F_{norm} wird anschließend durch die ReLU-Funktion h_{relu} geleitet, welche eine elementweise Nichtlinearität einführt, indem sie alle negativen Werte auf Null setzt. Dies ist in Kapitel 2.1.1 dargelegt:

$$F_{\text{relu}} = h_{\text{relu}}(F_{\text{norm}}) \quad . \tag{5.11}$$

Dynamic Convolutional Fusion pre-trained FS: BCWH

Bei der Convolutional Fusion erfolgt die Fusion der Eingangstensoren ähnlich wie bei der Dynamic Summed Fusion. Der wesentliche Unterschied besteht darin, dass die gewichtete Kombination der beiden Tensoren nicht durch eine Addition erfolgt, sondern durch ein Convolutional Neural Network der gewichteten Tensoren. Sie ähnelt auch sehr stark der Concatenation Fusion, nur dass sie gewichtete Tensoren in die Convolution gibt. Beide Eingangstensoren haben durch Feature Scaling 5.3 die gleiche Verteilung. Außerdem wird ein pre-trained Modell verwendet. Das Feature Scaling wurde über Mini-Batch-Size, Channel, Width und Height-Achse vollzogen.

Die Fusion erfolgt durch die folgende Operation:

$$F_{\text{fused}} = h_{\text{conv}}\big(w \cdot F_{\text{bev}}, \big(1-w\big) \cdot F_{\text{aerial, scaled}}\big) \quad , \tag{5.12}$$

wobei h_{conv} eine Convolutional-Layer darstellt, die auf die gewichtete Kombination der beiden Tensoren angewendet wird. Die eine Convolutional-Layer hat einen 3x3-Kernel mit Stride 1 und Padding 1.

Das gewichtsberechnende neuronale Netzwerk w bestimmt die Gewichtung basierend auf den BEV-Features wie folgt:

$$w(F_{\text{bev}}) = \sigma(\mathcal{N}(F_{\text{bev}})) \quad , \tag{5.13}$$

wobei $\mathcal{N}$ ein Convolutional Neural Network darstellt, das den Tensor F_{bev} als Eingabe verarbeitet, der anschließend durch die Sigmoid-Funktion σ normalisiert wird, um einen Gewichtungswert im Bereich zwischen 0 und 1 zu erzeugen.

5.2 Der Optimierungsalgorithmus AdamW

Ein Optimierungsalgorithmus ist ein mathematisches Verfahren, das darauf abzielt, die besten Lösungen innerhalb eines gegebenen Rahmens zu finden, oft durch Minimierung oder Maximierung einer Zielfunktion unter Berücksichtigung bestimmter Einschränkungen. In der Welt der neuronalen Netze werden Optimierungsalgorithmen verwendet, um die Gewichte eines Modells so anzupassen, dass die Lossfunktion minimiert wird. Dieser Prozess wird als Training des Modells bezeichnet. Der Optimierungsalgorithmus spielt eine entscheidende Rolle, da er bestimmt, wie effizient und effektiv ein Modell aus den verfügbaren Daten lernen kann.

Adaptive Gradientenalgorithmen wie Adam [18] haben sich als Standard beim Training neuronaler Netze etabliert. Diese Algorithmen sind dafür bekannt, dass sie die Gradienten basierend auf ihren historischen Größen skalieren. Bei der herkömmlichen Nutzung von Adam in Verbindung mit L2-Regularisierung werden Gewichte, die große Gradienten aufweisen, jedoch nicht so stark regularisiert, wie es bei einer entkoppelten Weight Decay-Regularisierung der Fall wäre. Dies liegt daran, dass der Gradient der Regularisierung zusammen mit dem Gradienten der Lossfunktion skaliert wird, was zu einer Unäquivalenz zwischen L2-Regularisierung und decoupled Weight Decay-Regularisierung führt.

Aus diesem Grund haben Ilya Loshchilov und Frank Hutter eine verbesserte Version von Adam entwickelt, bekannt als AdamW [32]. AdamW adressiert genau diese Problematik, indem es die Weight Decay-Regularisierung von den verlustbasierten Gradientenaktualisierungen entkoppelt, wie in Zeile 21 des Algorithmus 1 gezeigt. Dies führt zu einer Weiterentwicklung von Adam mit decoupled Weight Decay-Regularisierung, die speziell darauf ausgerichtet ist, die Effektivität der Weight Decay-Regularisierung zu erhöhen und dadurch die Generalisierungsleistung des Modells zu verbessern. MapTR [27] verwendet den Optimierungsalgorithmus AdamW für das Training, um von dieser Verbesserung zu profitieren.

Algorithm 1 Adam with L2 regularization and AdamW mit decoupled weight decay

Gegeben:
$\alpha = 0.0006$ ▷ Anfängliche Lernrate, bestimmt die Schrittgröße der Updates
$\beta_1 = 0.95$ ▷ Momentumfaktor für das erste Moment, beeinflusst die Glättung des Gradienten
$\beta_2 = 0.99$ ▷ Faktor für das zweite Moment, beeinflusst die Glättung des Gradienten
$\varepsilon = 10^{-8}$ ▷ Kleine Zahl zur Vermeidung von Division durch Null
$\lambda \in \mathbb{R}$ ▷ Weight Decay/L2-Regularisierungsfaktor zur Vermeidung von Overfitting
Initialisiere:
Zeitschritt $t \leftarrow 0$, Parametervektor $\theta_{t=0} \in \mathbb{R}^n$ ▷ Startwerte für Parameter und Zeitschritt
Ersten Momentenvektor initialisieren $m_{t=0} \leftarrow 0$ ▷ Initialisierung des ersten Momentenvektors
Zweiten Momentenvektor initialisieren $v_{t=0} \leftarrow 0$ ▷ Initialisierung des zweiten Momentenvektors
Schedule Multiplier initialisieren $\eta_{t=0} \in \mathbb{R}$ ▷ Initialisierung des Zeitplanmultiplikators
repeat
    $t \leftarrow t + 1$ ▷ Inkrementierung des Zeitschritts
    $\nabla f_t(\theta_{t-1}) \leftarrow \text{SelectBatch}(\theta_{t-1})$ ▷ Selektiere Mini-Batch und berechne Gradienten
    $g_t \leftarrow \nabla f_t(\theta_{t-1}) + \lambda \theta_{t-1}$ ▷ Aktualisiere Gradienten um L2-Regularisierung
    $m_t \leftarrow \beta_1 m_{t-1} + (1-\beta_1) g_t$ ▷ Update des ersten Moments
    $v_t \leftarrow \beta_2 v_{t-1} + (1-\beta_2) g_t^2$ ▷ Update des zweiten Moments
    $\hat{m}_t \leftarrow \frac{m_t}{1-\beta_1^t}$ ▷ Bias-Korrektur des ersten Moments
    $\hat{v}_t \leftarrow \frac{v_t}{1-\beta_2^t}$ ▷ Bias-Korrektur des zweiten Moments
    $\eta_t \leftarrow \text{SetScheduleMultiplier}(t)$ ▷ Anpassung des Schedule Multipliers
    $\theta_t \leftarrow \theta_{t-1} - \eta_t \left(\alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon} + \lambda \theta_{t-1} \right)$ ▷ Parameteraktualisierung mit AdamW
until stopping criterion is met ▷ Wiederholen bis zum Stoppkriterium
return θ_t ▷ Rückgabe der optimierten Parameter

Konfiguration des AdamW-Optimierers für die Netzwerkarchitektur

Im Rahmen von MapTR wurde der AdamW-Optimierer bereits mit spezifischen Parametern konfiguriert, um optimale Ergebnisse bei der Fusion verschiedener Datensätze zu erzielen. Diese Parameterwahl basiert auf den vorangegangenen Erfahrungen und etablierten Praktiken innerhalb von MapTR.

- Lernrate (α): Die anfängliche Lernrate wurde auf $\alpha = 6 \times 10^{-4}$ festgelegt.
- Momentumfaktoren (β_1, β_2): Die Betas für das erste und zweite Moment wurden auf $\beta_1 = 0.95$ und $\beta_2 = 0.99$ gesetzt.
- Weight Decay (λ): Der Weight Decay wurde auf $\lambda = 0.01$ festgelegt.

Die Trainingsdauer war ein wesentlicher Faktor bei der Wahl dieser spezifischen Einstellungen, da der Fokus des Projekts auf anderen Aspekten der Modellentwicklung lag. Diese Parameterkonfiguration unterstützt das Hauptziel von MapTR, robustes und generalisierendes Modell zu entwickeln und somit auch von der daraus entwickelten Netzwerkarchitektur.

5.3 Konfiguration der Lernrate im Trainingsprozess

Im Trainingsprozess der Netzwerkarchitektur wird die Lernrate mittels eines Cosine Annealing Schedules [33] angepasst. Den Verlauf der Lernrate von der verwendetet Netzwerkarchitektur ist in Abbildung 5.1 dargestellt.

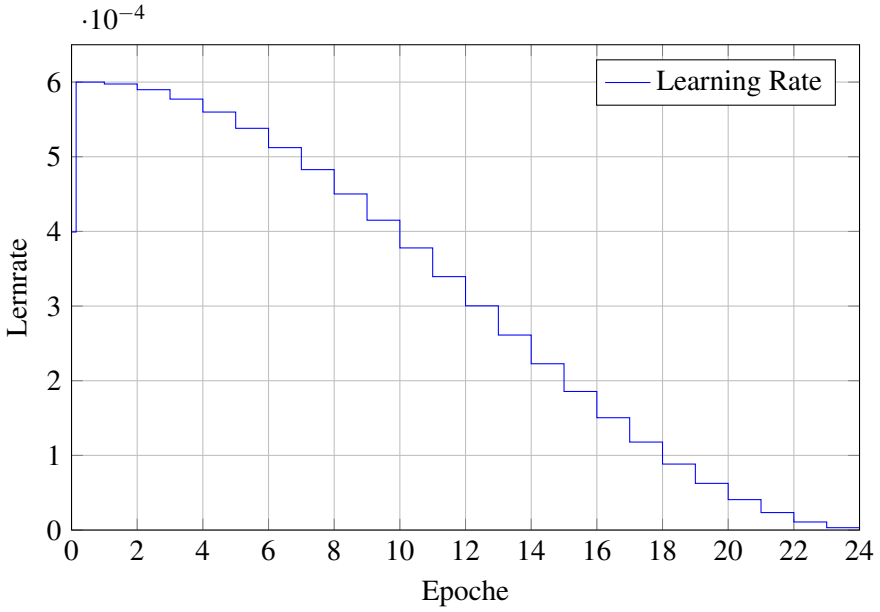


Abb. 5.1: Diese Abbildung zeigt die Anpassung der Lernrate über 24 Epochen, beginnend mit einem Warmup, das die Lernrate innerhalb der ersten 500 Iterationen auf ihr Maximum bringt, gefolgt von einer stufenweisen Reduktion, die durch das Cosine Annealing Schedule und das Verhältnis der minimalen Lernrate (`min_lr_ratio`) gesteuert wird.

Der Trainingsprozess wird durch eine spezifische Warmup-Phase eingeleitet. Das Warmup initiiert die Lernrate bei 4×10^{-4}, entsprechend einem Warmup-Ratio von $1/3$. Dieser Wert stellt ein Drittel des Weges zur maximal festgelegten Lernrate von 6×10^{-4} dar. Über 500 Iterationen steigt die Lernrate linear auf diesen Maximalwert an.

Nach dem Warmup wird die Lernrate gemäß des Parameters `min_lr_ratio` stufenweise reduziert. Dieser ist auf 0.001 eingestellt, dadurch reduziert sich die Lernrate immer um Tausendstel. Dieser ist auf 0.001 eingestellt, was bedeutet, dass sich die Lernrate nach Erreichen des Maximums bei jedem Schritt um ein Tausendstel ihrer jeweils letzten Lernrate reduziert.

Die Lernrate wurde mit folgenden Parametern initialisiert:

- Maximale Lernrate (`lr`): 6×10^{-4}
- Warmup-Typ (`warmup`): 'linear'
- Anzahl der Warmup-Iterationen (`warmup_iters`): 500
- Warmup-Verhältnis (`warmup_ratio`): $1/3$
- Verhältnis der minimalen Lernrate (`min_lr_ratio`): 1×10^{-3}

5.4 Berechnungsinfrastruktur und Cluster-Details

Für die Durchführung der in dieser Arbeit beschriebenen Experimente wurde eine NVIDIA A100 Tensor Core GPU verwendet [1], welche auf der Ampere-Architektur basiert. Speziell wurde das Modell mit 80 GB High-Bandwidth Memory (HBM2e) eingesetzt. Die Berechnungen für das Training über 24 Epochen erstreckten sich über einen Zeitraum von etwa 3,5 Tagen.

5.5 Weiteres Hyperparameter-Tuning

In diesem Abschnitt wird auf weiterführende Anpassungen weiterer Hyperparameter eingegangen, die aufgrund ihrer positiven Auswirkungen auf die Modellergebnisse implementiert wurden.

Einstellung der Mini-Batch-Größe

Die Mini-Batch-Größe ist ein zentraler Hyperparameter im maschinellen Lernen, der die Anzahl der Trainingsbeispiele bestimmt, die gemeinsam in einem Durchgang zur Berechnung der Gradienten verwendet werden.

- Effektive Batch-Größe: Die effektive Mini-Batch-Größe wurde auf 32 festgelegt.

Kumulative Gradientenakkumulation

Die kumulative Gradientenakkumulation ermöglicht es, effektiv eine größere Mini-Batch-Größe zu simulieren. Dies ist besonders bei begrenztem VRAM vorteilhaft. Die Berechnung und Speicherung der Gradienten benötigt viel VRAM. Die kumulative Gradientenakkumulation nicht alle

Samples eines großen Mini-Batches gleichzeitig verarbeiten zu müssen. Dies reduziert die Belastung des VRAMs, da die Gradienten schrittweise über mehrere Mini-Batches hinweg gesammelt werden, bevor ein Gewichtsupdate im Netzwerk durchgeführt wird.

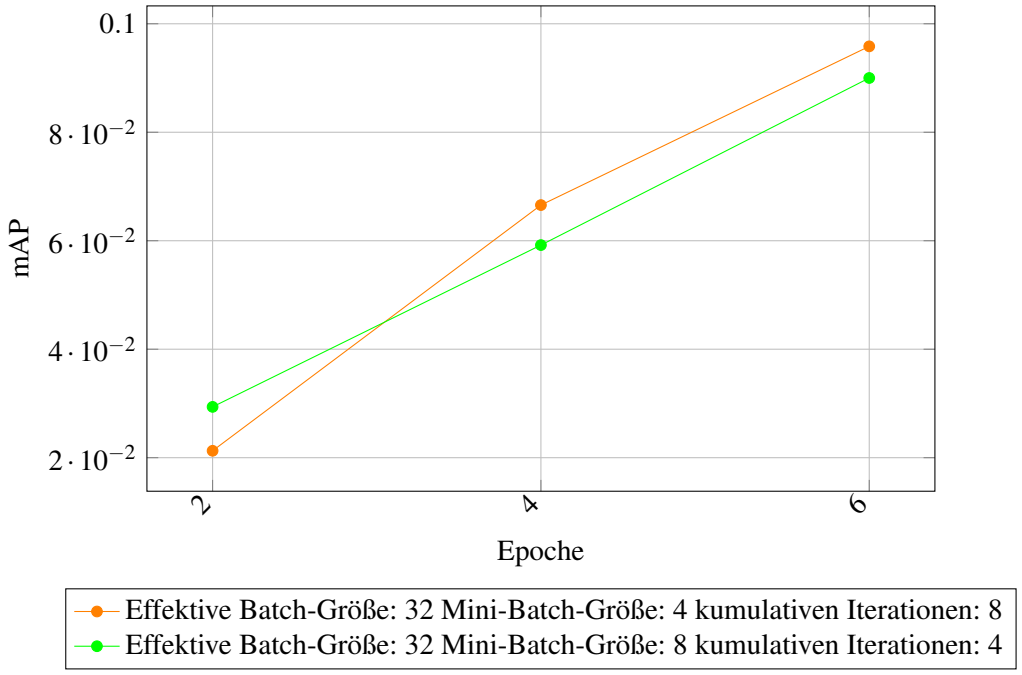


Abb. 5.2: Vergleich der Validierungsmetriken (mAP, siehe Kapitel 6.1) für zwei unterschiedliche Trainingskonfigurationen der Netzwerkarchitektur. Es zeigt sich, dass die Konfiguration mit einer Mini-Batch-Größe von 4 und 8 kumulativen Iterationen ein besseres Ergebnis erzielt.

- Kumulative Iterationen: Die Anzahl der kumulativen Iterationen wurde auf 8 festgelegt.
- Begründung: Abbildung 5.2 zeigt die Validierungsmetriken, speziell den Mean-Average-Precision (mAP), für zwei Trainingskonfigurationen der Netzwerkarchitektur. Es wurden Konfigurationen mit einer Mini-Batch-Größe von 4 und kumulativen Iterationen von 8 im Vergleich zu einer Mini-Batch-Größe von 8 mit kumulativen Iterationen von 4 untersucht, wobei beide Konfigurationen eine effektive Batch-Größe von 32 haben. Die grafische Darstellung verdeutlicht, dass die Konfiguration mit der kleineren Mini-Batch-Größe und den erhöhten kumulativen Iterationen zu einer verbesserten Modellperformance führt. Jedoch werden die Batch-Normalisierungen nur auf die Samples jeder Akkumulation und nicht auf die gesamte effektive Batch-Größe angewendet.

Gradient Clipping

Gradient Clipping ist eine Technik zur Begrenzung der Größe von Gradienten, um das Problem des Explodierens der Gradienten zu verhindern, welches besonders in tiefen neuronalen Netzwerken auftritt.

- Maximale Norm: 35

- Norm-Typ: 2 (L2-Norm)

- **Zweck**: Durch die Festlegung einer maximalen Norm von 35 und der Verwendung der L2-Norm wird sichergestellt, dass keine übermäßig großen Gradienten die Aktualisierungen der Gewichte dominieren. Dies hilft, die Parameteraktualisierungen innerhalb vernünftiger Grenzen zu halten und erhöht die allgemeine Stabilität während des Trainings.

5.6 Datensatz

Im Rahmen dieser Arbeit werden zwei Datenquellen genutzt: der nuScenes-Datensatz [2], der sechs Kamerabilder aus der Bodenperspektive sowie die zugehörigen Ground-Truth-Daten bereitstellt, und Apple Maps, das Luftbilder der relevanten Gebiete liefert [42]. Ground-Truth-Daten (GT) bezeichnen dabei manuell oder algorithmisch erstellte Annotationen, die als Referenzstandard für die Validierung und Evaluierung von Modellen dienen.

5.6.1 Der nuScenes-Datensatz

Der Datensatz umfasst die vollständige Sensorik eines autonomen Fahrzeugs, darunter sechs Kameras, fünf Radare und ein LIDAR, die zusammen ein 360°-Blickfeld abdecken (Abb. 5.3). Der Datensatz besteht aus 1.000 Szenen mit jeweils 20 Sekunden Länge, was in Summe etwa 15 Stunden Fahrzeit und eine zurückgelegte Strecke von 242 Kilometern bei einer durchschnittlichen Geschwindigkeit von 16 km/h entspricht. Die Datenerhebung fand in Boston (Seaport und South Boston) sowie in Singapur (One North, Holland Village und Queenstown) statt, um eine hohe Diversität in Bezug auf Vegetation, Bebauung, Verkehrsaufkommen sowie Fahrbedingungen (z. B. Rechts- und Linksverkehr) sicherzustellen. Zwei identisch ausgerüstete Renault Zoe Elektrofahrzeuge wurden verwendet, wobei die Sensorik präzise synchronisiert wurde.

Eine hochgenaue Lokalisierung des Fahrzeugs wurde durch eine Kombination aus LIDAR und Odometrie mittels eines Monte-Carlo-Lokalisierungsalgorithmus [8] erreicht, wodurch eine Genauigkeit von bis zu 10 cm erzielt werden konnte. Zusätzlich wurden hochauflösende semantische Karten (HD-Karten) erstellt. Diese Karten enthalten die GT-Daten für das vorgestellte Modell. Sie bestehen aus elf semantischen Klassen wie Straßen, Gehwege und anderen relevanten Elementen.

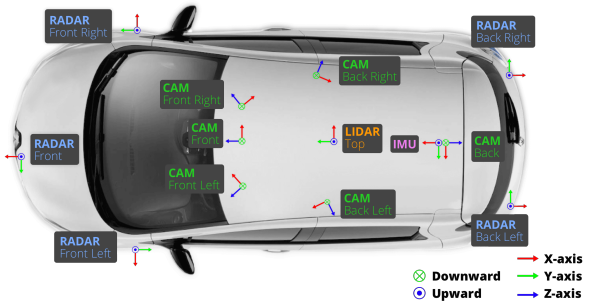


Abb. 5.3: Sensoraufbau der Datenerhebungsplattform. Für diese Arbeit sind ausschließlich die Kameras relevant, während andere Sensorik wie LIDAR und Radar nicht berücksichtigt wird [2].

Für diese Arbeit werden aus dem umfangreichen Angebot des nuScenes-Datensatzes lediglich drei der elf zur Verfügung gestellten semantischen Klassen der HD-Karten berücksichtigt. Diese Klassen umfassen:

- Fußgängerüberwege,
- Fahrspurmarkierungen,
- Straßengrenzen.

Ein Beispiel einer HD-Karte der Ground-Truth-Daten mit den semantischen Klassen ist in Abbildung 5.4 zu sehen.

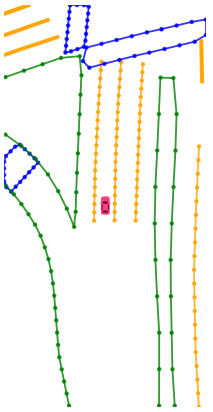


Abb. 5.4: Durch MapTR modifizierten Ground-Truth-Daten aus dem nuScenes-Datensatz. Gezeigt werden Fußgängerüberwege, Fahrspurmarkierungen und Straßengrenzen [2].

Darüber hinaus wird ausschließlich die Bildgebung der sechs Kameras verwendet. In Abbildung 5.5 sind die sechs Kamerabilder einer Beispielszene zu sehen. Daten von Radaren und dem LIDAR-Sensor werden nicht genutzt. Die Kameras des Systems besitzen folgende Spezifikationen:

- RGB-Bildgebung mit einer Erfassungsfrequenz von 12 Hz,
- 1/1.8” CMOS-Sensor,
- Auflösung von 1600×900 Pixel,
- automatische Belichtungsregelung,
- JPEG-komprimierte Speicherung.

Diese Einschränkungen wurden vorgenommen, um die Komplexität der Analyse zu reduzieren und den Fokus auf die wesentlichen Aspekte der Fragestellung zu legen. Darüber hinaus orientiert sich diese Auswahl an der Arbeit von MapTR, die ebenfalls ausschließlich diese drei semantischen Klassen verwendet.



Abb. 5.5: Kamerabilder einer Szene aus dem nuScenes-Datensatz. Diese ermöglicht eine umfassende visuelle Erfassung der Fahrzeugumgebung [2].

Datensplits in Trainings- und Validierungsdatensatz

Die aktuelle Strategie für den Datensplit in MapTR, die aus dem standardmäßigen nuScenes-Datensplit übernommen wurde, führt zu räumlichen Überschneidungen zwischen Trainings- und Validierungsdatensätzen. Dies resultiert aus dem ursprünglichen Design des nuScenes-Splits, der für Objekterkennungsaufgaben konzipiert wurde, wo räumliche Trennung weniger kritisch ist. Für Aufgaben wie die HD-Karten-Prädiktion, bei denen räumliche Unabhängigkeit zwischen Trainings- und Validierungsdaten entscheidend ist, stellt dies jedoch ein Problem dar.

Trotz dieser Problematik wurde entschieden, den vorhandenen Datensplit von MapTR zu übernehmen, um die Ergebnisse direkt vergleichbar zu machen.

5.6.2 Apple Maps als ergänzende Datenquelle

Zusätzlich zum nuScenes-Datensatz wurden Luftbilder aus Apple Maps verwendet, um den Datensatz zu ergänzen und als zusätzliche Datenbasis für die Netzwerkarchitektur zu dienen. Die Bilder wurden aus der Apple Satellite View [42] bezogen.

Luftbilder bieten eine Vogelperspektive auf die relevanten Gebiete und ermöglichen eine kontextuelle Erweiterung der Bodendaten aus einer von oben betrachteten Perspektive. Im Vergleich zu bodennahen Sensoren wie Kameras zeichnen sich Luftbilder durch eine größere räumliche Reichweite aus. Verdeckungen durch Verkehrsteilnehmer werden minimiert und beschränken sich lediglich auf die darunterliegende Bodenebene. Zudem sind Luftbilder für viele Regionen der Welt bereits verfügbar, was ihre Integration und Nutzung erheblich vereinfacht. Obwohl Luftbilder, die durch Flugzeuge, Drohnen oder Satelliten aufgenommen wurden, häufig für die Kartenerstellung genutzt werden, können Einschränkungen wie eine geringe räumliche Auflösung oder veraltetes Material die Qualität beeinflussen. Dennoch bieten sie eine wertvolle Ergänzung.

Der Zugriff auf die Luftbilder erfolgte über Apple Maps, da diese Plattform eine effiziente Handhabung von API-Anfragen erlaubte, was für das Herunterladen tausender Bilder erforderlich ist. Für die Bildaufnahme wurde der höchste verfügbare Zoomlevel (19 für Apple Maps) verwendet, um eine optimale Detailgenauigkeit der Luftbilder zu gewährleisten.

Die Welt ist in ein Gitter (Grid) von Kacheln (Tiles) unterteilt, die in verschiedenen Zoomstufen verfügbar sind. Auf Zoomlevel 0 ist die gesamte Erde in einer einzigen Kachel dargestellt. Auf Zoomlevel 1 sind es 4 Kacheln, auf Zoomlevel 2 sind es 16 Kacheln, und für einen beliebigen Zoomlevel n ergeben sich 2^{2n} Kacheln [45]. Jede Kachel hat eine eindeutige x- und y-Position im Gitter sowie eine Zoomstufe z. Diese Kacheln korrespondieren eindeutig mit geographischen Koordinaten (Breitengrad und Längengrad), wodurch spezifische Kacheln für bestimmte reale Positionen identifiziert werden können. Eine Beispielkachel aus Boston findet sich in Abbildung 5.6.

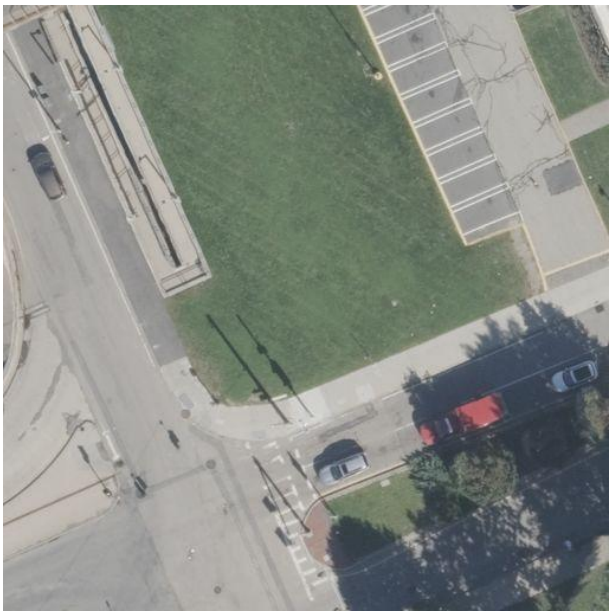


Abb. 5.6: Unverarbeitete Kachel aus Apple Maps, die eine Kreuzung in Boston zeigt, bevor sie für die Verwendung in der Netzwerkarchitektur vorverarbeitet wurde. Die ursprüngliche Auflösung dieser Kachel beträgt 0,1103 m/Pixel.

Die Berechnung der Kachelnummern x und y basierend auf den Koordinaten sowie die Umkehrung, also die Berechnung der geographischen Koordinaten aus den Kachelnummern, erfolgt mit den Formeln, die in der Dokumentation der Slippy Maps beschrieben sind [45]. Mithilfe der berechneten x-, y- und z-Werte konnten die entsprechenden Kacheln direkt von den Apple-Servern heruntergeladen werden.

Für die Extraktion der Kacheln wurde ein modifiziertes Python-Skript verwendet, das auf dem Projekt jimutmap basiert [17]. Das ursprüngliche Skript war nicht mehr funktionsfähig, daher wurde es angepasst, um den Anforderungen dieses Projekts zu entsprechen. Mit dem modifizierten Skript wurden einzelne Kacheln heruntergeladen, um die gesamten vier nuScenes-Regionen (drei in Singapur und eine in Boston) abzudecken.

Die Kacheln werden den entsprechenden Szenen aus dem nuScenes-Datensatz zugeordnet und derart rotiert, dass die Fahrtrichtung stets nach oben zeigt. Ein Beispiel einer solchen verarbeiteten Kachel, die für die Netzwerkarchitektur verwendet wird, ist in Abbildung 5.7 dargestellt. Diese Anpassungen stellen sicher, dass die Daten optimal für die Netzwerkarchitektur aufbereitet sind.

In der Netzwerkarchitektur ist es von entscheidender Bedeutung, dass die Daten aus verschiedenen Standorten dieselbe räumliche Auflösung aufweisen. Ursprünglich wiesen die hochauflösen-

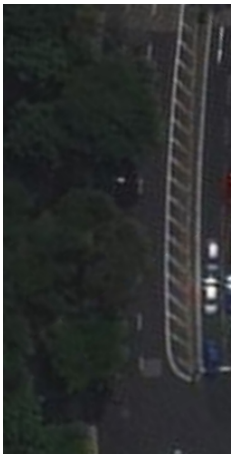


Abb. 5.7: Verarbeitetes Luftbild aus Apple Maps, wie es in der Netzwerkarchitektur verwendet wird. Jedes Luftbild deckt eine Fläche von jeweils 30 × 60 Metern ab.

den Bilder aus Boston und Singapur unterschiedliche Auflösungen auf 0,1103 m/Pixel für Boston und 0,14925 m/Pixel für Singapur. Um eine einheitliche Verarbeitung in der Netzwerkarchitektur sicherzustellen, wurden die Bilder aus Boston herunterskaliert, sodass beide Standorte dieselbe Auflösung haben.

5.7 Gewichtsinitialisierung Xavier

In der Forschung zu tiefen neuronalen Netzen hat sich gezeigt, dass die Wahl der Initialisierungsmethode für Gewichte eine bedeutende Rolle bei der Konvergenz des Trainings spielt. Eine der effektiven Methoden zur Gewichtsinitialisierung ist die uniforme Xavier-Initialisierung, auch bekannt als Glorot-Initialisierung [12]. Diese Methode zielt darauf ab, die Varianz der Aktivierungen über die Layer hinweg zu stabilisieren, um das Verschwinden oder Explodieren von Gradienten zu verhindern.

Die Grundidee der Xavier-Initialisierung basiert auf der Annahme, dass die Gewichte W eines jeden Neurons innerhalb einer Layer gleichmäßig aus einer Verteilung gezogen werden sollten, deren Breite so angepasst ist, dass die Varianz der Ausgaben jedes Neurons konstant bleibt. Dies soll unabhängig von der Anzahl der Eingänge n sein. Für die uniforme Xavier-Initialisierung wird die Breite des Intervalls für die gleichmäßige Verteilung wie folgt definiert:

$$W_{ij} \sim U\left[-\frac{1}{\sqrt{n}}, \frac{1}{\sqrt{n}}\right] \quad , \tag{5.14}$$

wobei $U[-a,a]$ die gleichmäßige Verteilung im Intervall $(-a,a)$ und n die Anzahl der Eingangseinheiten des Neurons (d.h. die Anzahl der Spalten von W) bezeichnet. Die Xavier-Initialisierung wird oft bei Netzen mit Sigmoid- oder Tanh-Aktivierungsfunktionen verwendet.

6 Evaluation

6.1 Validierungsmetrik

In diesem Abschnitt wird erläutert, wie die Netzwerkarchitektur validiert wird. Anstelle der Lossfunktion (siehe Kapitel 3.2.4) wird zur Validierung die Mean-Average-Precision-Metrik (mAP, Erklärung zu mAP in Kapitel 2.2) verwendet.

Für die Evaluierung wurden die folgenden Klassen von Kartenobjekten berücksichtigt:

- Fußgängerüberwege (pedestrian crossing),
- Fahrspurmarkierungen (lane divider),
- Straßengrenzen (road boundary).

Die Kartenkonstruktionsqualität wird anhand der mAP-Metrik bewertet. Die Ground-Truth-Daten (GT) stellen die tatsächlichen, manuell annotierten Kartenobjekte mit ihrer Position dar, die als Referenzstandard für die Evaluierung dienen. Die Prädiktionen (P) stammen aus der Netzwerkarchitektur, die während des Trainings darauf optimiert wurde, Kartenobjekte zu erkennen und zu lokalisieren. Diese Prädiktionen basieren auf den Eingabedaten, wie den Kamerabildern aus Bodenperspektive und Luftbildern. Die Genauigkeit der Prädiktionen (P) wird durch den Vergleich mit den Ground-Truth-Daten (GT) mittels der bidirektionalen Chamfer-Distanz (CD) [25] beurteilt.

Die bidirektionale Chamfer-Distanz wird aus der gerichteten Chamfer-Distanz ($\mathrm{CD}_{\mathrm{dir}}$) berechnet und definiert sich wie folgt:

$$CD_{\mathrm{dir}}(P,GT) = \frac{1}{|P|} \sum_{x \in P} \min_{y \in GT} \|x - y\|_2, \tag{6.1}$$

$$CD(P,GT) = CD_{\mathrm{dir}}(P,GT) + CD_{\mathrm{dir}}(GT,P). \tag{6.2}$$

Anhand vordefinierter Schwellenwerte ($\tau \in T, T = \{0.5, 1.0, 1.5\}$) wird entschieden, ob eine Prädiktion als true positive (TP), false positive (FP), true negative (TN) oder false negative (FN) klassifiziert wird. Anschließend wird die Mean-Average-Precision (mAP) gemäß Kapitel 2.2 berechnet. Ist die bidirektionale Chamfer-Distanz einer Prädiktion kleiner als der definierte Schwellenwert, wird diese als true positive (TP) gewertet, andernfalls als false positive (FP). Im letzten Schritt werden alle Precision-Recall-Paare aggregiert, um die Average-Precision (AP) für verschiedene Schwellenwerte zu bestimmen.

6.2 Ergebnisse

Dieses Kapitel präsentiert die Ergebnisse der durchgeführten Experimente. Es werden Vergleiche zwischen der MapTR-Baseline und den entwickelten statischen sowie dynamischen Fusionsmethoden dargestellt.

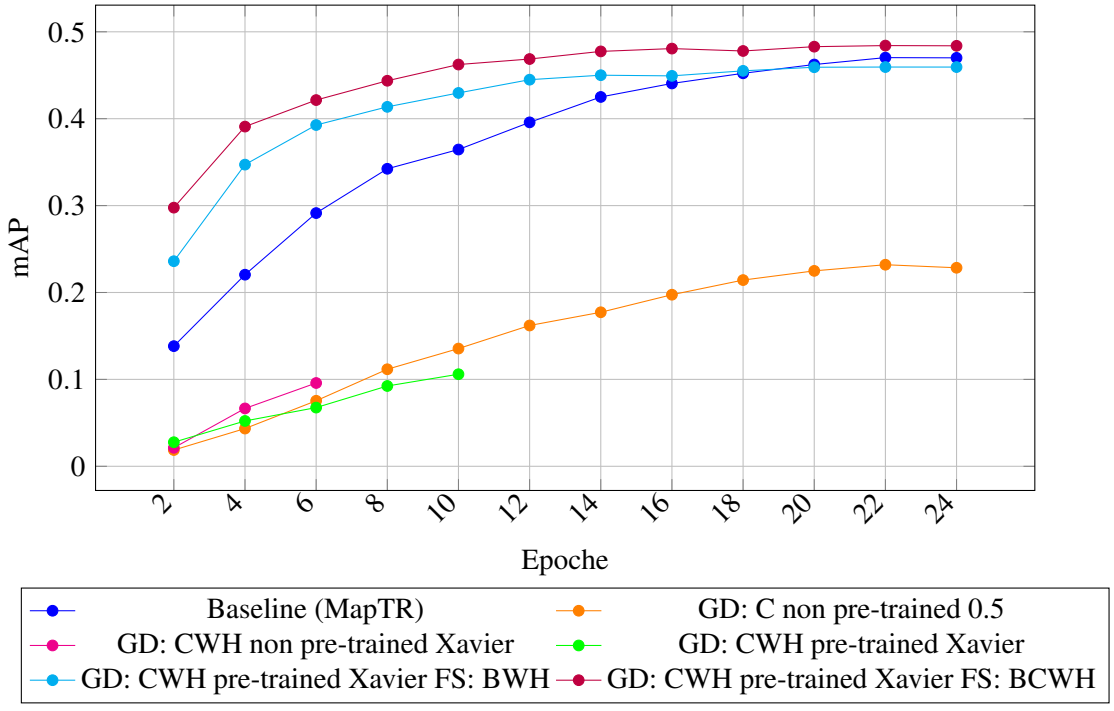
Baseline

Im Rahmen dieser Experimente wurden alle Fusionsmodelle mit der Baseline der MapTR-Architektur verglichen. Diese Vergleiche wurden unter Verwendung der am FZI verfügbaren Recheninfrastruktur durchgeführt, wie in Abschnitt 5.4 beschrieben.

Die MapTR-Architektur [27], die ein ResNet-50 als Backbone und einen GKT für die Transformation von 2D-Features in BEV (Bird's Eye View)-Features verwendet, erreicht laut MapTR einen Mean Average Precision (mAP) von 50,3% nach 24 Trainingsepochen. Unter denselben Bedingungen wurde die MapTR-Architektur auf dem FZI-Cluster trainiert, wobei ein mAP von 47,03% erzielt wurde, wie in Tabelle 6.1 dokumentiert.

6.2.1 Übersicht und Vergleich der statischen Fusionensmodelle

Dieser Abschnitt bietet eine detaillierte Übersicht über die Ergebnisse statischer Fusionsmodelle im Vergleich zur Baseline, wie sie in Kapitel 5.1.1 beschrieben werden. Um die Darstellung der Daten in den Tabellen und Diagrammen zu vereinfachen und gleichzeitig die Lesbarkeit zu erhöhen, wurden Abkürzungen verwendet. Diese Abkürzungen sind im genannten Kapitel fett hervorgehoben und können zudem der Tabelle 6.1 entnommen werden, welche die Ergebnisse zusammenfasst und vergleicht.



Gewichtstensor Dimension		C	CWH	CWH	CWH	CWH
pre-trained		non	non	pre-trained	pre-trained	pre-trained
Initialization		0.5	Xavier	Xavier	Xavier	Xavier
Feature Scaled Dimension		–	–	–	BWH	BCWH
Epoche	Baseline					
2	0.1383	0.0187	0.0213	0.0277	0.2360	0.2976
4	0.2205	0.0434	0.0666	0.0522	0.3471	0.3909
6	0.2914	0.0754	**0.0958**	0.0675	0.3927	0.4214
8	0.3424	0.1117	–	0.0924	0.4136	0.4437
10	0.3645	0.1355	–	**0.1059**	0.4296	0.4623
12	0.3958	0.1620	–	–	0.4449	0.4687
14	0.4250	0.1772	–	–	0.4500	0.4774
16	0.4406	0.1974	–	–	0.4493	0.4807
18	0.4522	0.2143	–	–	0.4551	0.4779
20	0.4623	0.2249	–	–	0.4592	0.4829
22	**0.4703**	**0.2320**	–	–	**0.4594**	**0.4841**
24	0.4700	0.2285	–	–	0.4594	0.4839

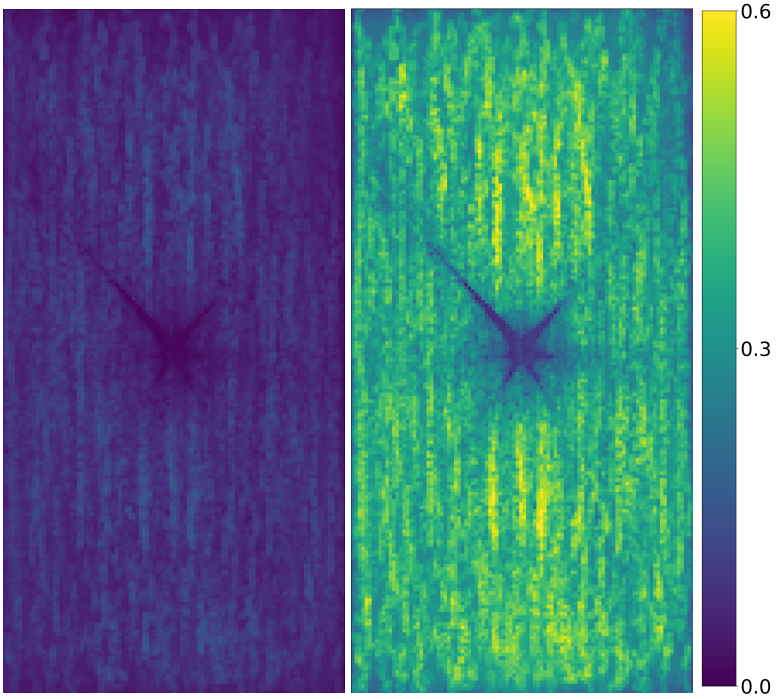
Tab. 6.1: In dieser Tabelle werden die Mean Average Precision (mAP)-Werte für jede Epoche und Modell dargestellt, wobei das Maximum für jedes Fusionsmodell besonders hervorgehoben ist. Die Tabelle zeigt ausschließlich auf die statischen Fusionsmodelle im Vergleich zur Baseline. Die Trainingsdauer wurde auf maximal 24 Epochen festgelegt. Sollte sich ein Trainingslauf als nicht erfolgversprechend erweisen, wurde dieser vorzeitig abgebrochen. Die Dauer für das Training einer einzelnen Epoche betrug etwa ca. 3,5 Stunden, während das vollständige Training über 24 Epochen insgesamt ungefähr 3,5 Tage in Anspruch nahm.

Die vorliegenden Untersuchungen der Fusionsmodelle im Vergleich zur Baseline zeigen anfänglich die Anwendung der Fusion ausschließlich über die Channel-Dimension. Hierbei erwies sich die Xavier-Initialisierung als überlegen gegenüber einer uniformen Initialisierung der Gewichtstensoren mit dem Wert 0,5, welche konsistent schlechtere Ergebnisse lieferte. Aufgrund dieser Erkenntnisse wurde in nachfolgenden Experimenten ausschließlich die Xavier-Initialisierung für statische Fusionsmodelle verwendet.

Die anfänglichen Resultate lagen deutlich unter den Baseline-Werten, was zur Entscheidung führte, alle Gewichte aus der MapTR-Pipeline zu übernehmen und lediglich das Aerial Backbone, die Feature-Pyramide-Network sowie das Fusionsmodell weiter zu trainieren. Diese Modifikation brachte jedoch zuerst keine signifikanten Verbesserungen. Eine eingehende Analyse der Verteilungen der einzelnen Eingangstensoren zeigte erhebliche Unterschiede in den Erwartungswerten und Standardabweichungen. Daraus resultierte die Entscheidung, beide Eingangstensoren auf dieselbe Verteilung zu normieren. Zunächst erfolgte dieses Feature Scaling lediglich über die Mini-Batch Width und Height, ähnlich der Mini-Batch-Normalisierung, was eine merkliche Verbesserung der Ergebnisse zur Folge hatte. In weiteren Schritten wurde zusätzlich über die Channel-Dimension reskaliert, eine Kombination aus Mini-Batch- und Layer-Normalisierung, was schließlich die Baseline übertraf.

Das Fusionsmodell ‘GD: CWH pre-trained Xavier FS: BCWH‘ erreichte mit einem mAP von 0.4841 die besten Ergebnisse und konnte die Baseline um 2,9 Prozentpunkte übertreffen. De-

taillierte Analysen dieses Modells, insbesondere des dreidimensionalen Gewichtstensors, wurden durchgeführt. Zur Visualisierung wurden die Werte über die Channels gemittelt und anschließend Width und Height dargestellt, wie in Abb. 6.1 zu sehen.

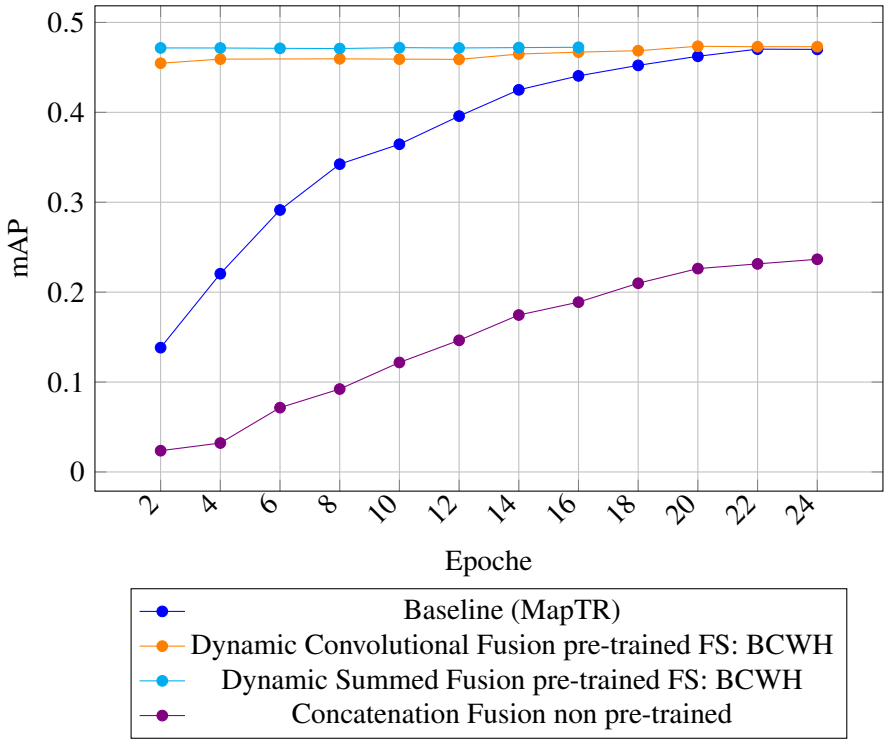


(a) Epoch 1 Heatmap (b) Epoch 24 Heatmap

Abb. 6.1: Visualisierung der Gewichtsverteilung in ‘GD: CWH pre-trained Xavier FS: BCWH‘ Fusionsmodell nach 1 Epoche und 24. Die Farben Dunkellila in der Grafik repräsentieren den durchschnittlichen Einfluss der BEV-Features auf die resultierenden Features über die Channels: Dunkellila (Wert 0) zeigt minimalen Einfluss und Gelb (Wert 0,6) einen moderaten Einfluss. Die Visualisierung veranschaulicht die relativ konstante Struktur der Gewichtung über die Zeit.

Die Visualisierung nach der ersten und nach vierundzwanzigsten Epochen zeigt, dass die Struktur relativ konstant bleibt, wobei sich lediglich die Gewichtung ändert. Dunkellila repräsentiert einen Durchschnittswert von 0 über die Channels, was darauf hinweist, dass die BEV-Features wenig Einfluss auf die resultierenden Features haben. Gelb hingegen deutet darauf hin, dass im Durchschnitt ein Einfluss von 0,6 der BEV-Features auf die resultierenden Features besteht. Diese Interpretation sollte jedoch mit Vorsicht betrachtet werden, da die Gewichtungen über alle Kanäle hinweg gemittelt wurden. Des Weiteren ist das Fahrzeug in der Mitte der Visualisierung deutlich erkennbar. Die sechs Kamerarichtungen sind sichtbar, und an den Schnittstellen der Kameras nimmt das Netzwerk tendenziell mehr Features aus den Luftbildern (Aerial-Features) auf. Zusätzlich ist am Rand der Visualisierung ein blau-lilaner Farbverlauf zu beobachten, was darauf hindeutet, dass in diesen Bereichen verstärkt Aerial-Features verwendet werden.

6.2.2 Übersicht und Vergleich der dynamsichen Fusionmodelle



Fusion		Dynamic Convolutional	Dynamic Summed	Concatenation
pre-trained		pre-trained	pre-trained	non
Feature Scaled		BCWH	BCWH	–
Dimension				
Epoche	Baseline			
2	0.1383	0.4546	0.4716	0.0237
4	0.2205	0.4592	0.4716	0.0322
6	0.2914	–	0.4712	0.0715
8	0.3424	0.4595	0.4710	0.0922
10	0.3645	0.4592	0.4720	0.1218
12	0.3958	0.4589	0.4716	0.1464
14	0.4250	0.4648	0.4720	0.1745
16	0.4406	0.4670	**0.4723**	0.1889
18	0.4522	0.4685	–	0.2099
20	0.4623	**0.4735**	–	0.2262
22	**0.4703**	0.4730	–	0.2314
24	0.4700	0.4730	–	**0.2366**

Tab. 6.2: mAP-Werte pro Epoche, das Maximum jeder Fusionsmethode ist hervorgehoben. Diese Tabelle zeigt nur die dynamisch Fusionsmethoden im Vergleich zur Baseline. Es wurden maximal 24 Epochen trainiert. Wenn der Run nicht vielversprechend war, wurde er abegbrochen. Das Training einer Epche hat ca. 3,5 Stunden gedauert. Das Training von 24 Epochen dauerte ungefähr 3,5 Tage.

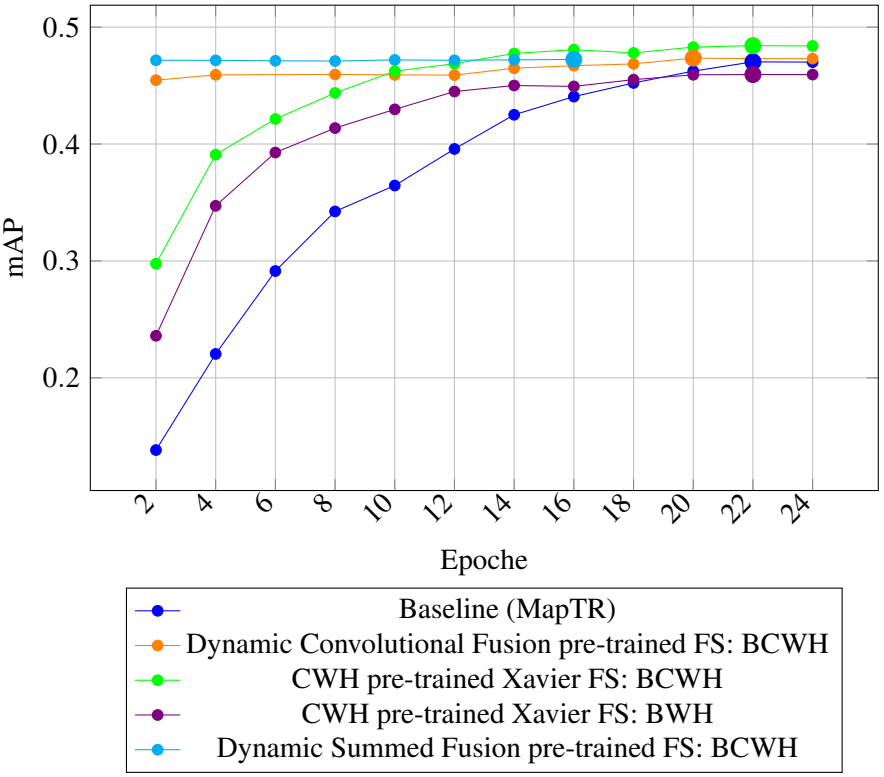
In diesem Abschnitt werden die Ergebnisse von den drei dynamischen Fusionsstrategien präsentiert und mit der Baseline verglichen. Die Baseline, mit einem Mean Average Precision (mAP)

von 0.4703, diente als Referenzwert für die Evaluierung der Leistungsfähigkeit der dynamischen Modelle.

Zwei der untersuchten Modelle, die Dynamic Convolutional Fusion und die Dynamic Summed Fusion, zeigten eine Verbesserung gegenüber der Baseline. Die Dynamic Convolutional Fusion erreichte ein mAP von 0.4735, was einer marginalen Verbesserung von 0,7 Prozent gegenüber der Baseline entspricht. Ähnlich dazu erzielte die Dynamic Summed Fusion ein mAP von 0.4723, was eine geringfügige Steigerung von 0,4 Prozent darstellt. Beide Modelle waren pre-trained, was ihnen half, die Baseline leicht zu übertreffen.

Im Gegensatz dazu wurde auch ein non pre-trained Modell, die Concatenation Fusion, getestet, dessen Ergebnisse deutlich hinter denen der Baseline zurückblieben. Mit einem mAP von 0.2366 lag dieses Modell erheblich unter der Baseline, was die Herausforderungen von nicht vortrainierten Ansätzen in diesem spezifischen Anwendungskontext verdeutlicht. Da sie zusätzlich immer noch die Parameter der Fusion lernen müssen.

6.3 Schlussfolgerungen aus den Ergebnissen



Die Analyse der verschiedenen Fusionsmodelle hat gezeigt, dass das Feature Scaling eine entscheidende Rolle für den Erfolg dieser Modelle spielt. Alle vier leistungsstärksten Fusionsmodelle implementierten Feature-Scaling, was deren Ergebnisse unterstreicht. Das Scaling der Features ermöglicht eine ausgewogene Berücksichtigung der Input-Features, wodurch die Mean Average Precision verbessert werden konnte.

Des Weiteren profitieren Modelle, die mit pre-trained Modellparametern initialisiert sind, da sie sich besser auf die spezifischen Feinabstimmungen des Fusionsmodells fokussieren.

Fusion		Dynamic Convolutional	Static	Static	Dynamic Summed
Gewichtstensor Dim.			CWH	CWH	
pre-trained		pre-trained	pre-trained	pre-trained	non
Feature Scaled		BCWH	BCWH	BCWH	BCWH
Epoche	Baseline				
2	0.1383	0.4546	0.2976	0.2360	0.4716
4	0.2205	0.4592	0.3909	0.3471	0.4716
6	0.2914	–	0.4214	0.3927	0.4712
8	0.3424	0.4595	0.4437	0.4136	0.4710
10	0.3645	0.4592	0.4623	0.4296	0.4720
12	0.3958	0.4589	0.4687	0.4449	0.4716
14	0.4250	0.4648	0.4774	0.4500	0.4720
16	0.4406	0.4670	0.4807	0.4493	**0.4723**
18	0.4522	0.4685	0.4779	0.4551	–
20	0.4623	**0.4735**	0.4829	0.4592	–
22	**0.4703**	0.4730	**0.4841**	**0.4594**	–
24	0.4700	0.4730	0.4839	0.4594	–

Tab. 6.3: Diese Tabelle zeigt die Besten Fusionmodelle. Die Epoche mit dem größten mAP für jedes Fusionsmodell ist markiert.

6.3.1 Qualitative Analyse der Fusionsstrategien

Zur Evaluation der Fusionsstrategien wurden fünf Szenen aus dem Validierungsdatensatz ausgewählt, um qualitativ spezifische Herausforderungen wie Reichweite und Verdeckungen zu untersuchen.

Reichweite und Erkennbarkeit von Verdeckungen

In Abbildung 6.2 ist die Reichweite der Luftbildaufnahme ersichtlich, die eine klare Sicht auf eine Kreuzung in der linken oberen Ecke bietet. Während die MapTR-Baseline aufgrund der Verdeckung von Verkehrszeichen Schwierigkeiten bei der Erkennung hat, ermöglichen die Luftbilder eine deutlich verbesserte Situationsbeurteilung. In Abb. 6.5 verdeckt ein Fahrzeug die Mittelspurmarkierung des Versuchsträgers, wodurch diese in der MapTR-Baseline nicht dargestellt wird. Durch die Integration von Luftbildern kann die Netzwerkarchitektur jedoch die Fahrspurmarkierung erkennen, da sie im Luftbild sichtbar ist, obwohl die Straße teilweise aufgrund starken Bewuchses kaum einsehbar bleibt. Diese Beobachtung verdeutlicht die Leistungsfähigkeit der Netzwerkarchitektur bei der Fusion von Luft- und Kamerabildern, da sie in der Lage ist, Verdeckungen im Kamerabild erfolgreich zu überwinden.

Limitationen der Luftbilder

Jedoch sind auch Luftbilder nicht frei von Einschränkungen. Beispiele hierfür sind in den Abbildungen 6.3 und 6.4 dargestellt, wo die Sicht durch Bäume oder abstehende Gebäude teilweise

blockiert wird. Diese Verdeckungen beeinträchtigen die Effektivität der Luftbilder als zuverlässige Datenquelle für die HD-Kartierung.

Nutzung von Kamerabildern bei Verdeckungen

Kamerabilder sind oft durch andere Fahrzeuge verdeckt, wie in Abbildung 6.6 ersichtlich. Trotz dieser Herausforderungen gelingt es der Baseline manchmal, relevante Informationen der Fahrumgebung richtig zu prädizieren. Was Overfitting nahelegt. Die Kombination von Luftbildern sollte die Erkennungsrate weiter verbessern.

Fünf zufällige Szenen aus dem Validierungsdatensatz

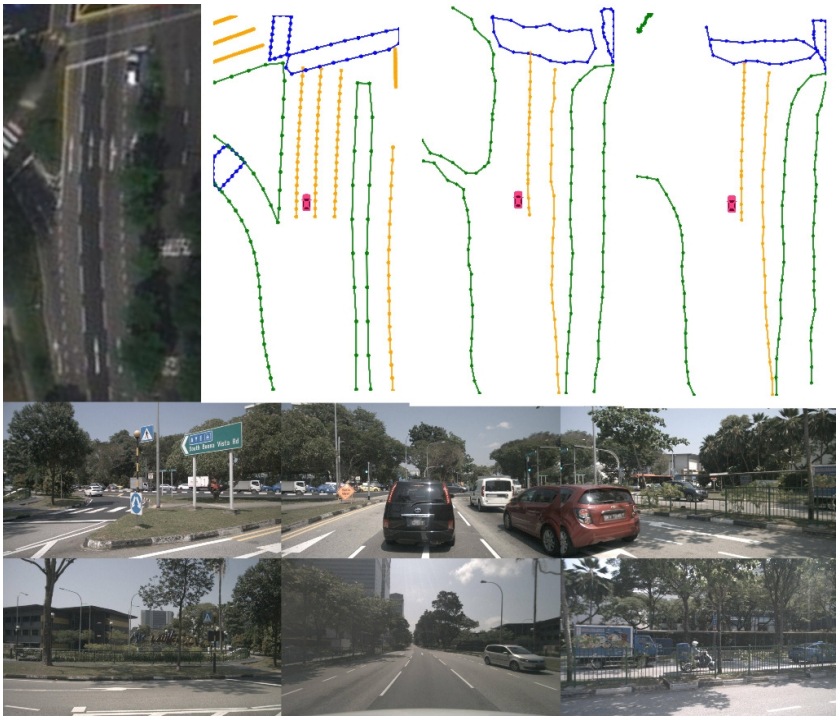


Abb. 6.2: Szene aus dem Validierungsdatensatz. Oben links: Luftaufnahme aus Apple Maps. Daneben: Das Label, Prädiktion des besten Modells Epoche 22 'GD: CWH pre-trained Xavier FS: BCWH' und der Baseline Epoche 22. Unten: Die Kamerabildern aus Bodenperspektive passend zur Szene.

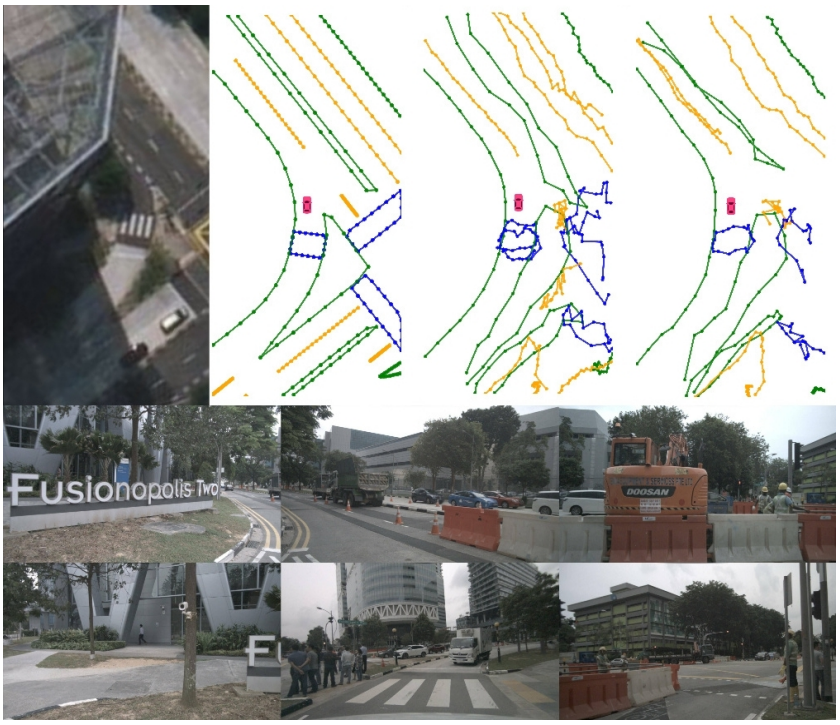


Abb. 6.3: Szene aus dem Validierungsdatensatz. Oben links: Luftaufnahme aus Apple Maps. Daneben: Das Label, Prädiktion des besten Modells Epoche 22 'GD: CWH pre-trained Xavier FS: BCWH' und der Baseline Epoche 22. Unten: Die Kamerabildern aus Bodenperspektive passend zur Szene.

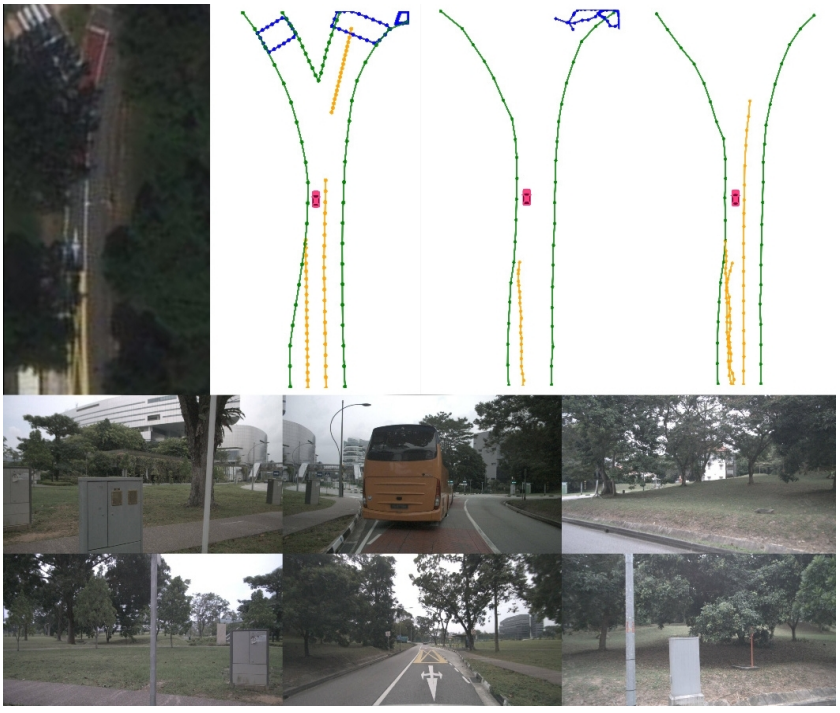


Abb. 6.4: Szene aus dem Validierungsdatensatz. Oben links: Luftaufnahme aus Apple Maps. Daneben: Das Label, Prädiktion des besten Modells Epoche 22 'GD: CWH pre-trained Xavier FS: BCWH' und der Baseline Epoche 22. Unten: Die Kamerabildern aus Bodenperspektive passend zur Szene.

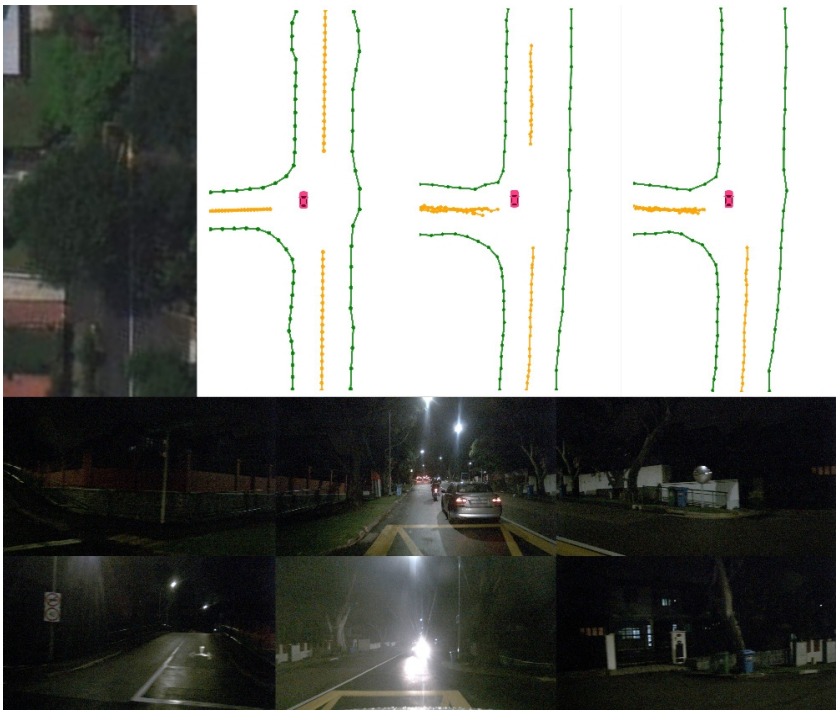


Abb. 6.5: Szene aus dem Validierungsdatensatz. Oben links: Luftaufnahme aus Apple Maps. Daneben: Das Label, Prädiktion des besten Modells Epoche 22 'GD: CWH pre-trained Xavier FS: BCWH' und der Baseline Epoche 22. Unten: Die Kamerabildern aus Bodenperspektive passend zur Szene.

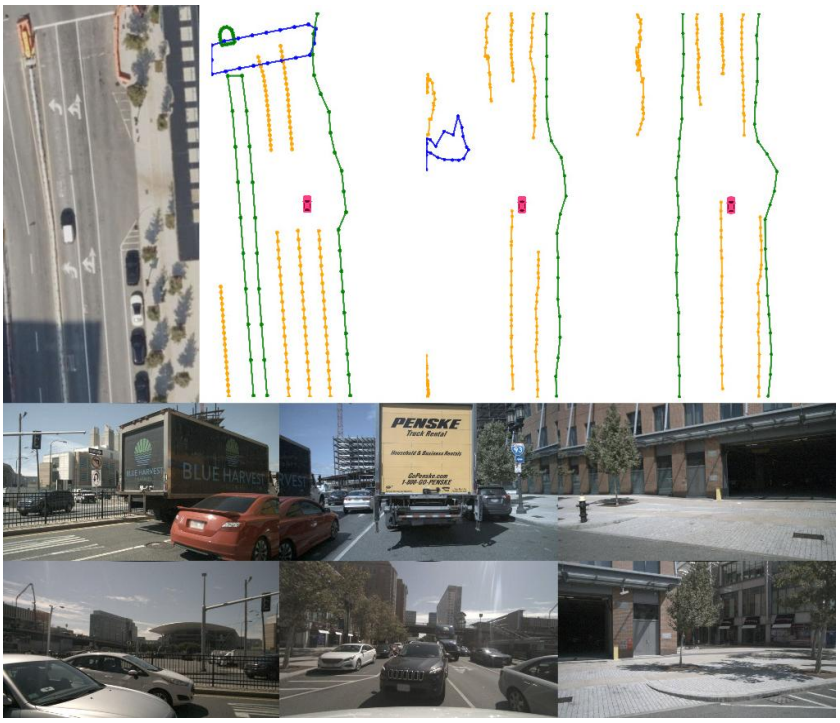


Abb. 6.6: Szene aus dem Validierungsdatensatz. Oben links: Luftaufnahme aus Apple Maps. Daneben: Das Label, Prädiktion des besten Modells Epoche 22 'GD: CWH pre-trained Xavier FS: BCWH' und der Baseline Epoche 22. Unten: Die Kamerabildern aus Bodenperspektive passend zur Szene.

7 Zusammenfassung und Ausblick

Die Arbeit beschäftigte sich mit der Konzeption, Umsetzung und Evaluation einer transformerbasierten Netzwerkarchitektur, welche Luftbilder und Kamerabilder aus Bodenperspektive eines Fahrzeugs fusioniert und daraus online eine HD-Karte der Umgebung erzeugt. Im Fokus stand die Erweiterung der existierenden MapTR-Architektur [27] um ein Fusionsmodell, das die Datenquellen fusioniert. Es wurden statische und dynamische Fusionsmodelle konzeptioniert und implementiert, dabei konnte die Genauigkeit der existierenden MapTR-Architektur übertroffen werden. Die Evaluierung der unterschiedlichen Fusionsansätze zeigte, dass vor allem Modelle, die Feature Scaling implementieren und auf einer pre-trained Netzwerkarchitektur basieren die besten Ergebnisse erzielen. Die Ergebnisse unterstreichen die Potenziale der kombinierten Nutzung von Luft- und Kamerabildern, insbesondere in Bezug auf die Überwindung von Sichtbehinderungen durch Verdeckungen, die bei der Nutzung nur einer Bildquelle auftreten können.

Die Ergebnisse dieser Arbeit bieten eine solide Grundlage für weitere Forschungen im Bereich der Fusionsstrategien für die HD-Kartenerstellung. Zukünftige Arbeiten könnten sich auf mehrere Aspekte konzentrieren:

1. **Integration zeitbasierter Informationen:** Eine Methode, die in Forschungsarbeiten wie MachMap [38] verwendet wird, ist die zeitliche Fusion, die historische BEV-Features in die Berechnung der aktuellen BEV-Features einbezieht, um die Prädiktion der HD-Karte zu verbessern.

2. **Verfeinerung der Fusionsmethoden:** Weiterentwicklung und Optimierung der Fusionsstrategien, wie beispielsweise die Verwendung von spezielleren Netzen zur dynamischen Gewichtung.

3. **Optimierung des Datensplits:** Die aktuelle Strategie für den Datensplit in MapTR führt zu Überschneidungen zwischen Trainings- und Validierungsdatensätzen in räumlicher Hinsicht. Dies könnte die hohen Prädiktionsgenauigkeiten in einigen Szenen erklären, in denen selbst nicht sichtbare Spuren prädiziert werden und sollte für eine objektivere Bewertung angepasst werden.

4. **Quantitative Bewertung der Reichweite:** Eine systematische Untersuchung der Prädiktionsfähigkeit über die Reichweite könnte durchgeführt werden, indem Labels und Prädiktonen in einem definierten Raum um das Fahrzeug analysiert werden. In dieser Arbeit wurde ein Bereich von 30 Metern x 60 Metern betrachtet in denen die Fahrzeugkameras gute Einsicht haben. Weiter entfernte Straßenmarkierungen wurden in der MapTR-Baseline und dieser Arbeit nicht prädiziert. Durch eine Vergrößerung des zu prädizierenden Bereichs könnte man den Mehrwert von Luftbildern in der Fusion besser quantifizieren.

A Abbildungsverzeichnis

2.1 Schematische Darstellung eines Multi-Layer-Perceptron 4
2.2 Precision-Recall-Kurve 8
2.3 Beispiel einer Convolution. 9
2.4 Max-Pooling-Operation 11
2.5 Schematische Darstellung eines Feature-Pyramid-Network (FPN) 12
2.6 Transformer-Architektur 15
2.7 Attention-Mechanismen im Transformer 17

3.1 Schematische Darstellung der MapTR-Pipeline 22
3.2 Geometrische Abstraktion von Kartenelementen durch MapTR 24
3.3 Darstellung des Geometry-Guided Kernel Transformers 28
3.4 Architektur des BEVFormer-Encoders 29
3.5 Netzwerkarchitektur von VectorMapNet 31
3.6 Architekturübersicht von MachMap 32

4.1 Erweiterte Netzwerkarchitektur MapTR durch Fusionsmodell 33

5.1 Darstellung der Lernrate über 24 Epochen. 43
5.2 Vergleich der Validierungsmetriken für Trainingskonfigurationen 45
5.3 Sensoraufbau der Datenerhebungsplattform 46
5.4 Modifizierte Ground-Truth-Daten aus nuScenes 47
5.5 Kamerabilder aus dem nuScenes-Datensatz 48
5.6 Unverarbeitete Kachel aus Apple Maps 49
5.7 Verarbeitetes Luftbild aus Apple Maps 50

6.1 Visualisierung der Gewichtsverteilung im Fusionsmodell 54
6.2 Szene aus dem Validierungsdatensatz – Bild 1 58
6.3 Szene aus dem Validierungsdatensatz – Bild 2 59
6.4 Szene aus dem Validierungsdatensatz – Bild 3 59
6.5 Szene aus dem Validierungsdatensatz – Bild 3 60
6.6 Szene aus dem Validierungsdatensatz – Bild 5 60

B Tabellenverzeichnis

6.1 mAP-Werte für statische Fusionsmodelle im Vergleich zur Baseline 53
6.2 mAP-Werte für dynamische Fusionsmethoden im Vergleich zur Baseline 55
6.3 Beste Fusionsmodelle mit mAP 57

C Literaturverzeichnis

[1] NVIDIA A100 Tensor Core GPU. `https://www.nvidia.com/en-us/data-center/a100/`, 2024. Zugriff: 5. Januar 2025.

[2] H. Caesar, V. Bankiti, A. H. Lang, S. Vora, V. E. Liong, Q. Xu, A. Krishnan, Y. Pan, G. Baldan, and O. Beijbom. nuScenes: A multimodal dataset for autonomous driving. In *CVPR*, 2020.

[3] Y. B. Can, A. Liniger, D. P. Paudel, and L. Van Gool. Structured bird's-eye-view traffic scene understanding from onboard images. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 15661–15670, 2021.

[4] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko. End-to-end object detection with transformers. In *European conference on computer vision*, pages 213–229. Springer, 2020.

[5] L. Chen, C. Sima, Y. Li, Z. Zheng, J. Xu, X. Geng, H. Li, C. He, J. Shi, Y. Qiao, et al. Persformer: 3d lane detection via perspective transformer and the openlane benchmark. In *European Conference on Computer Vision*, pages 550–567. Springer, 2022.

[6] S. Chen, T. Cheng, X. Wang, W. Meng, Q. Zhang, and W. Liu. Efficient and robust 2d-to-bev representation learning via geometry-guided kernel transformer. *arXiv preprint arXiv:2206.04584*, 2022.

[7] D. Chicco. Ten quick tips for machine learning in computational biology. *BioData mining*, 10(1):35, 2017.

[8] Z. J. Chong, B. Qin, T. Bandyopadhyay, M. H. Ang, E. Frazzoli, and D. Rus. Synthetic 2d lidar for precise vehicle localization in 3d urban environment. In *2013 IEEE International Conference on Robotics and Automation*, pages 1554–1559. IEEE, 2013.

[9] J. Dai, H. Qi, Y. Xiong, Y. Li, G. Zhang, H. Hu, and Y. Wei. Deformable convolutional networks. In *Proceedings of the IEEE international conference on computer vision*, pages 764–773, 2017.

[10] V. Dumoulin and F. Visin. A guide to convolution arithmetic for deep learning. *arXiv preprint arXiv:1603.07285*, 2016.

[11] G. Elghazaly, R. Frank, S. Harvey, and S. Safko. High-definition maps: Comprehensive survey, challenges and future perspectives. *IEEE Open Journal of Intelligent Transportation Systems*, 2023.

[12] X. Glorot and Y. Bengio. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the thirteenth international conference on artificial intelligence and statistics*, pages 249–256. JMLR Workshop and Conference Proceedings, 2010.

[13] C. Han, J. Yang, J. Sun, Z. Ge, R. Dong, H. Zhou, W. Mao, Y. Peng, and X. Zhang. Exploring recurrent long-term temporal fusion for multi-view 3d perception. *IEEE Robotics and Automation Letters*, 2024.

[14] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.

[15] A. Hu, Z. Murez, N. Mohan, S. Dudas, J. Hawke, V. Badrinarayanan, R. Cipolla, and A. Kendall. Fiery: Future instance prediction in bird's-eye view from surround monocular cameras. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 15273–15282, 2021.

[16] S. Ioffe. Batch normalization: Accelerating deep network training by reducing internal covariate shift. *arXiv preprint arXiv:1502.03167*, 2015.

[17] G. R. Jimut123. jimutmap. `https://github.com/Jimut123/jimutmap`. Accessed: 2024-12-26.

[18] D. P. Kingma. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.

[19] A. Krizhevsky, I. Sutskever, and G. E. Hinton. Imagenet classification with deep convolutional neural networks. *Advances in neural information processing systems*, 25, 2012.

[20] A. H. Lang, S. Vora, H. Caesar, L. Zhou, J. Yang, and O. Beijbom. Pointpillars: Fast encoders for object detection from point clouds. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 12697–12705, 2019.

[21] Y. Le Cun, L. Bottou, and Y. Bengio. Reading checks with multilayer graph transformer networks. In *1997 IEEE International Conference on Acoustics, Speech, and Signal Processing*, volume 1, pages 151–154. IEEE, 1997.

[22] Y. LeCun, L. Bottou, G. B. Orr, and K.-R. Müller. Efficient backprop. In *Neural networks: Tricks of the trade*, pages 9–50. Springer, 2002.

[23] J. Lei Ba, J. R. Kiros, and G. E. Hinton. Layer normalization. *ArXiv e-prints*, pages arXiv–1607, 2016.

[24] F. Li, H. Zhang, H. Xu, S. Liu, L. Zhang, L. M. Ni, and H.-Y. Shum. Mask dino: Towards a unified transformer-based framework for object detection and segmentation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 3041–3050, 2023.

[25] Q. Li, Y. Wang, Y. Wang, and H. Zhao. Hdmapnet: An online hd map construction and evaluation framework. In *2022 International Conference on Robotics and Automation (ICRA)*, pages 4628–4634. IEEE, 2022.

[26] Z. Li, W. Wang, H. Li, E. Xie, C. Sima, T. Lu, Y. Qiao, and J. Dai. Bevformer: Learning bird's-eye-view representation from multi-camera images via spatiotemporal transformers. In *European conference on computer vision*, pages 1–18. Springer, 2022.

[27] B. Liao, S. Chen, X. Wang, T. Cheng, Q. Zhang, W. Liu, and C. Huang. MapTR: Structured Modeling and Learning for Online Vectorized HD Map Construction. In *International Conference on Learning Representations*, 2023.

[28] T. Lin. Focal loss for dense object detection. *arXiv preprint arXiv:1708.02002*, 2017.

[29] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie. Feature pyramid networks for object detection. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 2117–2125, 2017.

[30] R. Liu, Z. Yuan, T. Liu, and Z. Xiong. End-to-end lane shape prediction with transformers. In *Proceedings of the IEEE/CVF winter conference on applications of computer vision*, pages 3694–3702, 2021.

[31] Y. Liu, T. Yuan, Y. Wang, Y. Wang, and H. Zhao. Vectormapnet: End-to-end vectorized hd map learning. In *International Conference on Machine Learning*. PMLR, 2023.

[32] I. Loshchilov. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017.

[33] I. Loshchilov and F. Hutter. Sgdr: Stochastic gradient descent with warm restarts. *arXiv preprint arXiv:1608.03983*, 2016.

[34] H. A. Mallot, H. H. Bülthoff, J. J. Little, and S. Bohrer. Inverse perspective mapping simplifies optical flow computation and obstacle detection. *Biological cybernetics*, 64(3):177–185, 1991.

[35] F. Milletari, N. Navab, and S.-A. Ahmadi. V-net: Fully convolutional neural networks for volumetric medical image segmentation. In *2016 fourth international conference on 3D vision (3DV)*, pages 565–571. Ieee, 2016.

[36] K. O'Shea. An introduction to convolutional neural networks. *arXiv preprint arXiv:1511.08458*, 2015.

[37] J. Philion and S. Fidler. Lift, splat, shoot: Encoding images from arbitrary camera rigs by implicitly unprojecting to 3d. In *Computer Vision–ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part XIV 16*, pages 194–210. Springer, 2020.

[38] L. Qiao, Y. Zheng, P. Zhang, W. Ding, X. Qiu, X. Wei, and C. Zhang. Machmap: End-to-end vectorized solution for compact hd-map construction. *arXiv preprint arXiv:2306.10301*, 2023.

[39] T. Shan and B. Englot. Lego-loam: Lightweight and ground-optimized lidar odometry and mapping on variable terrain. In *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 4758–4765. IEEE, 2018.

[40] T. Shan, B. Englot, D. Meyers, W. Wang, C. Ratti, and D. Rus. Lio-sam: Tightly-coupled lidar inertial odometry via smoothing and mapping. In *2020 IEEE/RSJ international conference on intelligent robots and systems (IROS)*, pages 5135–5142. IEEE, 2020.

[41] A. Vaswani. Attention is all you need. *Advances in Neural Information Processing Systems*, 2017.

[42] A. S. View. Satellites pro. `https://satellites.pro/`. Accessed: 2024-12-26.

[43] M. Visvalingam and J. D. Whyatt. The douglas-peucker algorithm for line simplification: re-evaluation through visualization. In *Computer graphics forum*, volume 9, pages 213–225. Wiley Online Library, 1990.

[44] W. Wang, J. Dai, Z. Chen, Z. Huang, Z. Li, X. Zhu, X. Hu, T. Lu, L. Lu, H. Li, et al. Internimage: Exploring large-scale vision foundation models with deformable convolutions. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 14408–14419, 2023.

[45] O. Wiki. Slippy map tilenames. `https://wiki.openstreetmap.org/wiki/Slippy_map_tilenames`. Accessed: 2024-12-26.

[46] G. Zhang, J. Lin, S. Wu, Z. Luo, Y. Xue, S. Lu, Z. Wang, et al. Online map vectorization for autonomous driving: A rasterization perspective. *Advances in Neural Information Processing Systems*, 36, 2024.

[47] J. Zhang, S. Singh, et al. Loam: Lidar odometry and mapping in real-time. In *Robotics: Science and systems*, volume 2, pages 1–9. Berkeley, CA, 2014.

[48] B. Zhou and P. Krähenbühl. Cross-view transformers for real-time map-view semantic segmentation. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 13760–13769, 2022.

[49] L. Zhu, Z. Deng, X. Hu, C.-W. Fu, X. Xu, J. Qin, and P.-A. Heng. Bidirectional feature pyramid network with recurrent attention residual modules for shadow detection. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 121–136, 2018.

[50] X. Zhu, W. Su, L. Lu, B. Li, X. Wang, and J. Dai. Deformable detr: Deformable transformers for end-to-end object detection. *arXiv preprint arXiv:2010.04159*, 2020.